# Certified Data Engineer Professional

**Exam A**

**QUESTION 1**
An upstream system has been configured to pass the date for a given batch of data to the Databricks Jobs API as a parameter. The notebook to be scheduled will use this parameter to load data with the following code:

```
df = spark.read.format("parquet").load(f"/mnt/source/(date)")
```

Which code block should be used to create the date Python variable used in the above code block?

A.
```
date = spark.conf.get("date")
```
B.
```
input_dict = input()
date= input_dict["date"]
```
C.
```
import sys
date = sys.argv[1]
```
D.
```
date = dbutils.notebooks.getParam("date")
```
E.
```
dbutils.widgets.text("date", "null")
date = dbutils.widgets.get("date")
```

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 2**
The Databricks workspace administrator has configured interactive clusters for each of the data engineering groups. To control costs, clusters are set to terminate after 30 minutes of inactivity. Each user should be able to execute workloads against their assigned clusters at any time of the day.

Assuming users have been added to a workspace but not granted any permissions, which of the following describes the minimal permissions a user would need to start and attach to an already configured cluster.

A. "Can Manage" privileges on the required cluster
B. Workspace Admin privileges, cluster creation allowed, "Can Attach To" privileges on the required cluster
C. Cluster creation allowed, "Can Attach To" privileges on the required cluster
D. "Can Restart" privileges on the required cluster
E. Cluster creation allowed, "Can Restart" privileges on the required cluster

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 3**
When scheduling Structured Streaming jobs for production, which configuration automatically recovers from query failures and keeps costs low?

A. Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: Unlimited
B. Cluster: New Job Cluster;
Retries: None;
Maximum Concurrent Runs: 1
C. Cluster: Existing All-Purpose Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1

D. Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1

E. Cluster: Existing All-Purpose Cluster;
Retries: None;
Maximum Concurrent Runs: 1

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 4**
The data engineering team has configured a Databricks SQL query and alert to monitor the values in a Delta Lake table. The **`recent_sensor_recordings`** table contains an identifying **`sensor_id`** alongside the **`timestamp`** and **`temperature`** for the most recent 5 minutes of recordings.

The below query is used to create the alert:

```
SELECT MEAN(temperature), MAX(temperature), MIN(temperature)
FROM recent_sensor_recordings
GROUP BY sensor_id
```

The query is set to refresh each minute and always completes in less than 10 seconds. The alert is set to trigger when **`mean (temperature) > 120`**. Notifications are triggered to be sent at most every 1 minute.

If this alert raises notifications for 3 consecutive minutes and then stops, which statement must be true?

A. The total average temperature across all sensors exceeded 120 on three consecutive executions of the query
B. The **`recent_sensor_recordings`** table was unresponsive for three consecutive runs of the query
C. The source query failed to update properly for three consecutive minutes and then restarted
D. The maximum temperature recording for at least one sensor exceeded 120 on three consecutive executions of the query
E. The average temperature recordings for at least one sensor exceeded 120 on three consecutive executions of the query

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 5**
A junior developer complains that the code in their notebook isn't producing the correct results in the development environment. A shared screenshot reveals that while they're using a notebook versioned with Databricks Repos, they're using a personal branch that contains old logic. The desired branch named **`dev-2.3.9`** is not available from the branch selection dropdown.

Which approach will allow this developer to review the current logic for this notebook?

A. Use Repos to make a pull request use the Databricks REST API to update the current branch to **`dev-2.3.9`**
B. Use Repos to pull changes from the remote Git repository and select the **`dev-2.3.9`** branch.
C. Use Repos to checkout the **`dev-2.3.9`** branch and auto-resolve conflicts with the current branch
D. Merge all changes back to the main branch in the remote Git repository and clone the repo again

E. Use Repos to merge the current branch and the **`dev-2.3.9`** branch, then make a pull request to sync with the remote repository

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 6**
The security team is exploring whether or not the Databricks secrets module can be leveraged for connecting to an external database.

After testing the code with all Python variables being defined with strings, they upload the password to the secrets module and configure the correct permissions for the currently active user. They then modify their code to the following (leaving all other variables unchanged).

```
password = dbutils.secrets.get(scope="db_creds", key="jdbc_password")

print(password)

df = (spark
  .read
  .format("jdbc")
  .option("url", connection)
  .option("dbtable", tablename)
  .option("user", username)
  .option("password", password)
  )
```

Which statement describes what will happen when the above code is executed?

A. The connection to the external table will fail; the string "**`REDACTED`**" will be printed.
B. An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the encoded password will be saved to DBFS.
C. An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the password will be printed in plain text.
D. The connection to the external table will succeed; the string value of **`password`** will be printed in plain text.
E. The connection to the external table will succeed; the string "**`REDACTED`**" will be printed.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 7**
The data science team has created and logged a production model using MLflow. The following code correctly imports and applies the production model to output the predictions as a new DataFrame named **`preds`** with the schema "**`customer_id LONG, predictions DOUBLE, date DATE`**".

```
from pyspark.sql.functions import current_date

model = mlflow.pyfunc.spark_udf(spark, model_uri="models:/churn/prod")
df = spark.table("customers")
columns = ["account_age", "time_since_last_seen", "app_rating"]
preds = (df.select(
  "customer_id",
  model(*columns).alias("predictions"),
  current_date().alias("date")
  )
```

The data science team would like predictions saved to a Delta Lake table with the ability to compare all predictions across time. Churn predictions will be made at most once per day.

Which code block accomplishes this task while minimizing potential compute costs?

A. `preds.write.mode("append").saveAsTable("churn_preds")`

B. `preds.write.format("delta").save("/preds/churn_preds")`

C.
```
(preds.writeStream
  .outputMode("overwrite")
  .option("checkpointPath", "/_checkpoints/churn_preds")
  .start("/preds/churn_preds")
)
```

D.
```
(preds.write
  .format("delta")
  .mode("overwrite")
  .saveAsTable("churn_preds")
)
```

E.
```
(preds.writeStream
  .outputMode("append")
  .option("checkpointPath", "/_checkpoints/churn_preds")
  .table("churn_preds")
)
```

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 8**
An upstream source writes Parquet data as hourly batches to directories named with the current date. A nightly batch job runs the following code to ingest all data from the previous day as indicated by the **`date`** variable:

```
(spark.read
  .format("parquet")
  .load(f"/mnt/raw_orders/{date}")
  .dropDuplicates(["customer_id", "order_id"])
  .write
  .mode("append")
  .saveAsTable("orders")
)
```

Assume that the fields **`customer_id`** and **`order_id`** serve as a composite key to uniquely identify each order.

If the upstream system is known to occasionally produce duplicate entries for a single order hours apart, which statement is correct?

A. Each write to the **`orders`** table will only contain unique records, and only those records without duplicates in the target table will be written.
B. Each write to the **`orders`** table will only contain unique records, but newly written records may have duplicates already present in the target table.
C. Each write to the **`orders`** table will only contain unique records; if existing records with the same key are present in the target table, these records will be overwritten.
D. Each write to the **`orders`** table will only contain unique records; if existing records with the same key are present in the target table, the operation will fail.
E. Each write to the **`orders`** table will run deduplication over the union of new and existing records, ensuring no duplicate records are present.



**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 9**
A junior member of the data engineering team is exploring the language interoperability of Databricks notebooks. The intended outcome of the below code is to register a view of all sales that occurred in countries on the continent of Africa that appear in the **`geo_lookup`** table.

Before executing the code, running **`SHOW TABLES`** on the current database indicates the database contains only two tables: **`geo_lookup`** and **`sales`**.

```
Cmd 1
%python
countries_af = [x[0] for x in
spark.table("geo_lookup").filter("continent='AF'").select("country").collect()]

Cmd 2
%sql
CREATE VIEW sales_af AS
  SELECT *
  FROM sales
  WHERE city IN countries_af
  AND CONTINENT = "AF"
```

Which statement correctly describes the outcome of executing these command cells in order in an interactive notebook?

A. Both commands will succeed. Executing show tables will show that **`countries_af`** and **`sales_af`** have been registered as views.
B. Cmd 1 will succeed. Cmd 2 will search all accessible databases for a table or view named **`countries_af`**: if this entity exists, Cmd 2 will succeed.
C. Cmd 1 will succeed and Cmd 2 will fail. **`countries_af`** will be a Python variable representing a PySpark DataFrame.
D. Both commands will fail. No new variables, tables, or views will be created.
E. Cmd 1 will succeed and Cmd 2 will fail. **`countries_af`** will be a Python variable containing a list of strings.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 10**
A Delta table of weather records is partitioned by date and has the below schema:

```
date DATE, device_id INT, temp FLOAT, latitude FLOAT, longitude FLOAT
```

To find all the records from within the Arctic Circle, you execute a query with the below filter:

```
latitude > 66.3
```

Which statement describes how the Delta engine identifies which files to load?

A. All records are cached to an operational database and then the filter is applied
B. The Parquet file footers are scanned for min and max statistics for the **`latitude`** column
C. All records are cached to attached storage and then the filter is applied
D. The Delta log is scanned for min and max statistics for the **`latitude`** column
E. The Hive metastore is scanned for min and max statistics for the **`latitude`** column

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 11**
The data engineering team has configured a job to process customer requests to be forgotten (have their data deleted). All user data that needs to be deleted is stored in Delta Lake tables using default table settings.

The team has decided to process all deletions from the previous week as a batch job at 1am each Sunday. The total duration of this job is less than one hour. Every Monday at 3am, a batch job executes a series of **`VACUUM`** commands on all Delta Lake tables throughout the organization.

The compliance officer has recently learned about Delta Lake's time travel functionality. They are concerned that this might allow continued access to deleted data.

Assuming all delete logic is correctly implemented, which statement correctly addresses this concern?

A. Because the **`VACUUM`** command permanently deletes all files containing deleted records, deleted records may be accessible with time travel for around 24 hours.
B. Because the default data retention threshold is 24 hours, data files containing deleted records will be retained until the **`VACUUM`** job is run the following day.
C. Because Delta Lake time travel provides full access to the entire history of a table, deleted records can always be recreated by users with full admin privileges.

D. Because Delta Lake's delete statements have ACID guarantees, deleted records will be permanently purged from all storage systems as soon as a delete job completes.
E. Because the default data retention threshold is 7 days, data files containing deleted records will be retained until the **`VACUUM`** job is run 8 days later.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 12**
A junior data engineer has configured a workload that posts the following JSON to the Databricks REST API endpoint **`2.0/jobs/create`**.

```
{
  "name": "Ingest new data",
  "existing_cluster_id": "6015-954420-peace720",
  "notebook_task": {
    "notebook_path": "/Prod/ingest.py"
  }
}
```

Which statement describes the result of executing this workload three times assuming that all configurations and referenced resources are available?

A. Three new jobs named "Ingest new data" will be defined in the workspace, and they will each run once daily.
B. The logic defined in the referenced notebook will be executed three times on new clusters with the configurations of the provided cluster ID.
C. Three new jobs named "Ingest new data" will be defined in the workspace, but no jobs will be executed.
D. One new job named "Ingest new data" will be defined in the workspace, but it will not be executed.
E. The logic defined in the referenced notebook will be executed three times on the referenced existing all purpose cluster.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 13**
An upstream system is emitting change data capture (CDC) logs that are being written to a cloud object storage directory. Each record in the log indicates the change type (insert, update, or delete) and the values for each field after the change. The source table has a primary key identified by the field **`pk_id`**.

For analytical purposes, only the most recent value for each record needs to be recorded in the target Delta Lake table in the Lakehouse. The Databricks job to ingest these records occurs once per hour, but each individual record may have changed multiple times over the course of an hour.

Which solution meets these requirements?

A. Create a separate history table for each **`pk_id`** resolve the current state of the table by running a **`UNION ALL`** filtering the history tables for the most recent state.
B. Use **`MERGE INTO`** to insert, update, or delete the most recent entry for each **`pk_id`** into a bronze table, then propagate all changes throughout the system.
C. Iterate through an ordered set of changes to the table, applying each in turn; rely on Delta Lake's versioning ability to create an audit log.

D. Use Delta Lake's change data feed to automatically process CDC data from an external system, propagating all changes to all dependent tables in the Lakehouse.
E. Ingest all log information into a bronze table; use **`MERGE INTO`** to insert, update, or delete the most recent entry for each **`pk_id`** into a silver table to recreate the current table state.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 14**
An hourly batch job is configured to ingest data files from a cloud object storage container where each batch represent all records produced by the source system in a given hour. The batch job to process these records into the Lakehouse is sufficiently delayed to ensure no late-arriving data is missed. The **`user_id`** field represents a unique key for the data, which has the following schema:

```
user_id BIGINT, username STRING, user_utc STRING, user_region STRING,
last_login BIGINT, auto_pay BOOLEAN, last_updated BIGINT
```

New records are all ingested into a table named **`account_history`** which maintains a full record of all data in the same schema as the source. The next table in the system is named **`account_current`** and is implemented as a Type 1 table representing the most recent value for each unique **`user_id`**.

Assuming there are millions of user accounts and tens of thousands of records processed hourly, which implementation can be used to efficiently update the described **`account_current`** table as part of each hourly batch job?

A. Use Auto Loader to subscribe to new files in the **`account_history`** directory; configure a Structured Streaming trigger once job to batch update newly detected files into the **`account_current`** table.
B. Overwrite the **`account_current`** table with each batch using the results of a query against the **`account_history`** table grouping by **`user_id`** and filtering for the max value of **`last_updated`**.
C. Filter records in **`account_history`** using the **`last_updated`** field and the most recent hour processed, as well as the max **`last_iogin`** by **`user_id`** write a merge statement to update or insert the most recent value for each **`user_id`**.
D. Use Delta Lake version history to get the difference between the latest version of **`account_history`** and one version prior, then write these records to **`account_current`**.
E. Filter records in **`account_history`** using the **`last_updated`** field and the most recent hour processed, making sure to deduplicate on **`username`**; write a merge statement to update or insert the most recent value for each **`username`**.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 15**
A table in the Lakehouse named **`customer_churn_params`** is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.

The churn prediction model used by the ML team is fairly stable in production. The team is only interested in making predictions on records that have changed in the past 24 hours.

Which approach would simplify the identification of these changed records?

A. Apply the churn model to all rows in the **`customer_churn_params`** table, but implement logic to perform an upsert into the predictions table that ignores rows where predictions have not changed.

B. Convert the batch job to a Structured Streaming job using the **`complete`** output mode; configure a Structured Streaming job to read from the **`customer_churn_params`** table and incrementally predict against the churn model.

C. Calculate the difference between the previous model predictions and the current **`customer_churn_params`** on a key identifying unique customers before making new predictions; only make predictions on those customers not in the previous predictions.

D. Modify the overwrite logic to include a field populated by calling **`spark.sql.functions.current_timestamp()`** as data are being written; use this field to identify records written on a particular date.

E. Replace the current overwrite logic with a merge statement to modify only those records that have changed; write logic to make predictions on the changed records identified by the change data feed.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 16**
A table is registered with the following code:

```
CREATE TABLE recent_orders AS (
  SELECT a.user_id, a.email, b.order_id, b.order_date
  FROM
    (SELECT user_id, email
    FROM users) a
    INNER JOIN
    (SELECT user_id, order_id, order_date
    FROM orders
    WHERE order_date >= (current_date() - 7)) b
    ON a.user_id = b.user_id
)
```

Both **`users`** and **`orders`** are Delta Lake tables. Which statement describes the results of querying **`recent_orders`**?

A. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query finishes.

B. All logic will execute when the table is defined and store the result of joining tables to the DBFS; this stored data will be returned when the table is queried.

C. Results will be computed and cached when the table is defined; these cached results will incrementally update as new records are inserted into source tables.

D. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.

E. The versions of each source table will be stored in the table transaction log; query results will be saved to DBFS with each query.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 17**
A production workload incrementally applies updates from an external Change Data Capture feed to a Delta Lake table as an always-on Structured Stream job. When data was initially migrated for this table,

OPTIMIZE was executed and most data files were resized to 1 GB. Auto Optimize and Auto Compaction were both turned on for the streaming production job. Recent review of data files shows that most data files are under 64 MB, although each partition in the table contains at least 1 GB of data and the total table size is over 10 TB.

Which of the following likely explains these smaller file sizes?

A. Databricks has autotuned to a smaller target file size to reduce duration of MERGE operations
B. Z-order indices calculated on the table are preventing file compaction
C. Bloom filter indices calculated on the table are preventing file compaction
D. Databricks has autotuned to a smaller target file size based on the overall size of data in the table
E. Databricks has autotuned to a smaller target file size based on the amount of data in each partition

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 18**
Which statement regarding stream-static joins and static Delta tables is correct?

A. Each microbatch of a stream-static join will use the most recent version of the static Delta table as of each microbatch.
B. Each microbatch of a stream-static join will use the most recent version of the static Delta table as of the job's initialization.
C. The checkpoint directory will be used to track state information for the unique keys present in the join.
D. Stream-static joins cannot use static Delta tables because of consistency issues.
E. The checkpoint directory will be used to track updates to the static Delta table.



**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 19**
A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame **`df`**. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Events are recorded once per minute per device.

Streaming DataFrame **`df`** has the following schema:

```
"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"
```

Code block:

```
df.withWatermark("event_time", "10 minutes")
    .groupBy(
        _____,
        "device_id"
    )
    .agg(
        avg("temp").alias("avg_temp"),
        avg("humidity").alias("avg_humidity")
    )
    .writeStream
    .format("delta")
    .saveAsTable("sensor_avg")
```

Choose the response that correctly fills in the blank within the code block to complete this task.

A. `to_interval("event_time", "5 minutes").alias("time")`
B. `window("event_time", "5 minutes").alias("time")`
C. `"event_time"`
D. `window("event_time", "10 minutes").alias("time")`
E. `lag("event_time", "10 minutes").alias("time")`

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 20**
A data architect has designed a system in which two Structured Streaming jobs will concurrently write to a single bronze Delta table. Each job is subscribing to a different topic from an Apache Kafka source, but they will write data with the same schema. To keep the directory structure simple, a data engineer has decided to nest a checkpoint directory to be shared by both streams.

The proposed directory structure is displayed below:


Which statement describes whether this checkpoint directory structure is valid for the given scenario and why?

A. No; Delta Lake manages streaming checkpoints in the transaction log.
B. Yes; both of the streams can share a single checkpoint directory.
C. No; only one stream can write to a Delta Lake table.
D. Yes; Delta Lake supports infinite concurrent writers.
E. No; each of the streams needs to have its own checkpoint directory.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 21**
A Structured Streaming job deployed to production has been experiencing delays during peak hours of the day. At present, during normal execution, each microbatch of data is processed in less than 3 seconds. During peak hours of the day, execution time for each microbatch becomes very inconsistent, sometimes exceeding 30 seconds. The streaming write is currently configured with a trigger interval of 10 seconds.

Holding all other variables constant and assuming records need to be processed in less than 10 seconds, which adjustment will meet the requirement?

A. Decrease the trigger interval to 5 seconds; triggering batches more frequently allows idle executors to begin processing the next batch while longer running tasks from previous batches finish.
B. Increase the trigger interval to 30 seconds; setting the trigger interval near the maximum execution time observed for each batch is always best practice to ensure no records are dropped.
C. The trigger interval cannot be modified without modifying the checkpoint directory; to maintain the current stream state, increase the number of shuffle partitions to maximize parallelism.
D. Use the trigger once option and configure a Databricks job to execute the query every 10 seconds; this ensures all backlogged records are processed with each batch.
E. Decrease the trigger interval to 5 seconds; triggering batches more frequently may prevent records from backing up and large batches from causing spill.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 22**
Which statement describes Delta Lake Auto Compaction?

A. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an `OPTIMIZE` job is executed toward a default of 1 GB.
B. Before a Jobs cluster terminates, `OPTIMIZE` is executed on all tables modified during the most recent job.
C. Optimized writes use logical partitions instead of directory partitions; because partition boundaries are only represented in metadata, fewer small files are written.
D. Data is queued in a messaging bus instead of committing data directly to memory; all data is committed from the messaging bus in one batch once the job is complete.
E. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an `OPTIMIZE` job is executed toward a default of 128 MB.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 23**
Which statement characterizes the general programming model used by Spark Structured Streaming?

A. Structured Streaming leverages the parallel processing of GPUs to achieve highly parallel data throughput.
B. Structured Streaming is implemented as a messaging bus and is derived from Apache Kafka.
C. Structured Streaming uses specialized hardware and I/O streams to achieve sub-second latency for data transfer.
D. Structured Streaming models new data arriving in a data stream as new rows appended to an unbounded table.
E. Structured Streaming relies on a distributed network of nodes that hold incremental state values for cached stages.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 24**
Which configuration parameter directly affects the size of a spark-partition upon ingestion of data into Spark?

A. `spark.sql.files.maxPartitionBytes`
B. `spark.sql.autoBroadcastJoinThreshold`
C. `spark.sql.files.openCostInBytes`
D. `spark.sql.adaptive.coalescePartitions.minPartitionNum`
E. `spark.sql.adaptive.advisoryPartitionSizeInBytes`



**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 25**
A Spark job is taking longer than expected. Using the Spark UI, a data engineer notes that the Min, Median, and Max Durations for tasks in a particular stage show the minimum and median time to complete a task as roughly the same, but the max duration for a task to be roughly 100 times as long as the minimum.

Which situation is causing increased duration of the overall job?

A. Task queueing resulting from improper thread pool assignment.
B. Spill resulting from attached volume storage being too small.
C. Network latency due to some cluster nodes being in different regions from the source data
D. Skew caused by more data being assigned to a subset of spark-partitions.
E. Credential validation errors while pulling data from an external system.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 26**
Each configuration below is identical to the extent that each cluster has 400 GB total of RAM, 160 total cores and only one Executor per VM.

Given a job with at least one wide transformation, which of the following cluster configurations will result in maximum performance?

A. • Total VMs: 1
   • 400 GB per Executor
   • 160 Cores / Executor
B. • Total VMs: 8
   • 50 GB per Executor
   • 20 Cores / Executor
C. • Total VMs: 16
   • 25 GB per Executor
   • 10 Cores/Executor
D. • Total VMs: 4
   • 100 GB per Executor
   • 40 Cores/Executor
E. • Total VMs: 2
   • 200 GB per Executor
   • 80 Cores / Executor

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 27**
A junior data engineer has implemented the following code block.

```
MERGE INTO events
USING new_events
ON events.event_id = new_events.event_id
WHEN NOT MATCHED
  INSERT *
```

The view **`new_events`** contains a batch of records with the same schema as the **`events`** Delta table. The **`event_id`** field serves as a unique key for this table.

When this query is executed, what will happen with new records that have the same **`event_id`** as an existing record?

A. They are merged.
B. They are ignored.
C. They are updated.
D. They are inserted.
E. They are deleted.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 28**
A junior data engineer seeks to leverage Delta Lake's Change Data Feed functionality to create a Type 1 table representing all of the values that have ever been valid for all rows in a **`bronze`** table created with

the property **`delta.enableChangeDataFeed = true`**. They plan to execute the following code as a daily job:

```
from pyspark.sql.functions import col

(spark.read.format("delta")
  .option("readChangeFeed", "true")
  .option("startingVersion", 0)
  .table("bronze")
  .filter(col("_change_type").isin(["update_postimage", "insert"]))
  .write
  .mode("append")
  .table("bronze_history_type1")
 )
```

Which statement describes the execution and results of running the above query multiple times?

A. Each time the job is executed, newly updated records will be merged into the target table, overwriting previous values with the same primary keys.
B. Each time the job is executed, the entire available history of inserted or updated records will be appended to the target table, resulting in many duplicate entries.
C. Each time the job is executed, the target table will be overwritten using the entire history of inserted or updated records, giving the desired result.
D. Each time the job is executed, the differences between the original and current versions are calculated; this may result in duplicate entries for some records.
E. Each time the job is executed, only those records that have been inserted or updated since the last execution will be appended to the target table, giving the desired result.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 29**
A new data engineer notices that a critical field was omitted from an application that writes its Kafka source to Delta Lake. This happened even though the critical field was in the Kafka source. That field was further missing from data written to dependent, long-term storage. The retention threshold on the Kafka service is seven days. The pipeline has been in production for three months.

Which describes how Delta Lake can help to avoid data loss of this nature in the future?

A. The Delta log and Structured Streaming checkpoints record the full history of the Kafka producer.
B. Delta Lake schema evolution can retroactively calculate the correct value for newly added fields, as long as the data was in the original source.
C. Delta Lake automatically checks that all fields present in the source data are included in the ingestion layer.
D. Data can never be permanently dropped or deleted from Delta Lake, so data loss is not possible under any circumstance.
E. Ingesting all raw data and metadata from Kafka to a bronze Delta table creates a permanent, replayable history of the data state.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 30**
A nightly job ingests data into a Delta Lake table using the following code:

```
from pyspark.sql.functions import current_timestamp, input_file_name, col
from pyspark.sql.column import Column

def ingest_daily_batch(time_col: Column, year:int, month:int, day:int):
    (spark.read
        .format("parquet")
        .load(f"/mnt/daily_batch/{year}/{month}/{day}")
        .select("*",
                time_col.alias("ingest_time"),
                inpute_file_name().alias("source_file")
               )
        .write
        .mode("append")
        .saveAsTable("bronze")
    )
```

The next step in the pipeline requires a function that returns an object that can be used to manipulate new records that have not yet been processed to the next table in the pipeline.

Which code snippet completes this function definition?

```
def new_records():
```

A. `return spark.readStream.table("bronze")`
B. `return spark.readStream.load("bronze")`
C.
```
return (spark.read
    .table("bronze")
    .filter(col("ingest_time") == current_timestamp())
)
```
D. `return spark.read.option("readChangeFeed", "true").table ("bronze")`
E.
```
return (spark.read
    .table("bronze")
    .filter(col("source_file") == f"/mnt/daily_batch/{year}/{mon
)
```

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 31**
A junior data engineer is working to implement logic for a Lakehouse table named **`silver_device_recordings`**. The source data contains 100 unique fields in a highly nested JSON

structure.

The `silver_device_recordings` table will be used downstream to power several production monitoring dashboards and a production model. At present, 45 of the 100 fields are being used in at least one of these applications.

The data engineer is trying to determine the best approach for dealing with schema declaration given the highly-nested structure of the data and the numerous fields.

Which of the following accurately presents information about Delta Lake and Databricks that may impact their decision-making process?

A. The Tungsten encoding used by Databricks is optimized for storing string data; newly-added native support for querying JSON strings means that string types are always most efficient.
B. Because Delta Lake uses Parquet for data storage, data types can be easily evolved by just modifying file footer information in place.
C. Human labor in writing code is the largest cost associated with data engineering workloads; as such, automating table declaration logic should be a priority in all migration workloads.
D. Because Databricks will infer schema using types that allow all observed data to be processed, setting types manually provides greater assurance of data quality enforcement.
E. Schema inference and evolution on Databricks ensure that inferred types will always accurately match the data types used by downstream systems.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**



**QUESTION 32**
The data engineering team maintains the following code:

```
accountDF = spark.table("accounts")
orderDF = spark.table("orders")
itemDF = spark.table("items")

orderWithItemDF = (orderDF.join(
    itemDF,
    orderDF.itemID == itemDF.itemID)
  .select(
    orderDF.accountID,
    orderDF.itemID,

    itemDF.itemName))

finalDF = (accountDF.join(
    orderWithItemDF,
    accountDF.accountID == orderWithItemDF.accountID)
  .select(
    orderWithItemDF["*"],

    accountDF.city))

(finalDF.write
  .mode("overwrite")
  .table("enriched_itemized_orders_by_account"))
```



Assuming that this code produces logically correct results and the data in the source tables has been de-duplicated and validated, which statement describes what will occur when this code is executed?

A. A batch job will update the **`enriched_itemized_orders_by_account`** table, replacing only those rows that have different values than the current version of the table, using **`accountID`** as the primary key.
B. The **`enriched_itemized_orders_by_account`** table will be overwritten using the current valid version of data in each of the three tables referenced in the join logic.
C. An incremental job will leverage information in the state store to identify unjoined rows in the source tables and write these rows to the **`enriched_itemized_orders_by_account`** table.
D. An incremental job will detect if new rows have been written to any of the source tables; if new rows are detected, all results will be recalculated and used to overwrite the **`enriched_itemized_orders_by_account`** table.
E. No computation will occur until **`enriched_itemized_orders_by_account`** is queried; upon query materialization, results will be calculated using the current valid version of data in each of the three tables referenced in the join logic.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 33**
The data engineering team is migrating an enterprise system with thousands of tables and views into the Lakehouse. They plan to implement the target architecture using a series of bronze, silver, and gold tables. Bronze tables will almost exclusively be used by production data engineering workloads, while silver tables will be used to support both data engineering and machine learning workloads. Gold tables will largely serve business intelligence and reporting purposes. While personal identifying information (PII) exists in all tiers of data, pseudonymization and anonymization rules are in place for all data at the silver and gold levels.

The organization is interested in reducing security concerns while maximizing the ability to collaborate across diverse teams.

Which statement exemplifies best practices for implementing this system?

A. Isolating tables in separate databases based on data quality tiers allows for easy permissions management through database ACLs and allows physical separation of default storage locations for managed tables.
B. Because databases on Databricks are merely a logical construct, choices around database organization do not impact security or discoverability in the Lakehouse.
C. Storing all production tables in a single database provides a unified view of all data assets available throughout the Lakehouse, simplifying discoverability by granting all users view privileges on this database.
D. Working in the default Databricks database provides the greatest security when working with managed tables, as these will be created in the DBFS root.
E. Because all tables must live in the same storage containers used for the database they're created in, organizations should be prepared to create between dozens and thousands of databases depending on their data isolation requirements.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**



**QUESTION 34**
The data architect has mandated that all tables in the Lakehouse should be configured as external Delta Lake tables.

Which approach will ensure that this requirement is met?

A. Whenever a database is being created, make sure that the `LOCATION` keyword is used.
B. When configuring an external data warehouse for all table storage, leverage Databricks for all ELT.
C. Whenever a table is being created, make sure that the `LOCATION` keyword is used.
D. When tables are created, make sure that the `EXTERNAL` keyword is used in the `CREATE TABLE` statement.
E. When the workspace is being configured, make sure that external cloud object storage has been mounted.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 35**
To reduce storage and compute costs, the data engineering team has been tasked with curating a series of aggregate tables leveraged by business intelligence dashboards, customer-facing applications, production machine learning models, and ad hoc analytical queries.

The data engineering team has been made aware of new requirements from a customer-facing application, which is the only downstream workload they manage entirely. As a result, an aggregate table

used by numerous teams across the organization will need to have a number of fields renamed, and additional fields will also be added.

Which of the solutions addresses the situation while minimally interrupting other teams in the organization without increasing the number of tables that need to be managed?

A. Send all users notice that the schema for the table will be changing; include in the communication the logic necessary to revert the new table schema to match historic queries.
B. Configure a new table with all the requisite fields and new names and use this as the source for the customer-facing application; create a view that maintains the original data schema and table name by aliasing select fields from the new table.
C. Create a new table with the required schema and new fields and use Delta Lake's deep clone functionality to sync up changes committed to one table to the corresponding table.
D. Replace the current table definition with a logical view defined with the query logic currently writing the aggregate table; create a new table to power the customer-facing application.
E. Add a table comment warning all users that the table schema and field names will be changing on a given date; overwrite the table in place to the specifications of the customer-facing application.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 36**
A Delta Lake table representing metadata about content posts from users has the following schema:

```
user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude
FLOAT, post_time TIMESTAMP, date DATE
```

This table is partitioned by the date column. A query is run with the following filter:

```
longitude < 20 & longitude > -20
```

Which statement describes how data will be filtered?

A. Statistics in the Delta Log will be used to identify partitions that might Include files in the filtered range.
B. No file skipping will occur because the optimizer does not know the relationship between the partition column and the longitude.
C. The Delta Engine will use row-level statistics in the transaction log to identify the flies that meet the filter criteria.
D. Statistics in the Delta Log will be used to identify data files that might include records in the filtered range.
E. The Delta Engine will scan the parquet file footers to identify each row that meets the filter criteria.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 37**
A small company based in the United States has recently contracted a consulting firm in India to implement several new data engineering pipelines to power artificial intelligence applications. All the company's data is stored in regional cloud storage in the United States.

The workspace administrator at the company is uncertain about where the Databricks workspace used by the contractors should be deployed.

Assuming that all data governance considerations are accounted for, which statement accurately informs this decision?

A. Databricks runs HDFS on cloud volume storage; as such, cloud virtual machines must be deployed in the region where the data is stored.
B. Databricks workspaces do not rely on any regional infrastructure; as such, the decision should be made based upon what is most convenient for the workspace administrator.
C. Cross-region reads and writes can incur significant costs and latency; whenever possible, compute should be deployed in the same region the data is stored.
D. Databricks leverages user workstations as the driver during interactive development; as such, users should always use a workspace deployed in a region they are physically near.
E. Databricks notebooks send all executable code from the user's browser to virtual machines over the open internet; whenever possible, choosing a workspace region near the end users is the most secure.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 38**
The downstream consumers of a Delta Lake table have been complaining about data quality issues impacting performance in their applications. Specifically, they have complained that invalid **`latitude`** and **`longitude`** values in the **`activity_details`** table have been breaking their ability to use other geolocation processes.

A junior engineer has written the following code to add **`CHECK`** constraints to the Delta Lake table:

```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
  latitude >= -90 AND
  latitude <= 90 AND
  longitude >= -180 AND
  longitude <= 180);
```

A senior engineer has confirmed the above logic is correct and the valid ranges for latitude and longitude are provided, but the code fails when executed.

Which statement explains the cause of this failure?

A. Because another team uses this table to support a frequently running application, two-phase locking is preventing the operation from committing.
B. The **`activity_details`** table already exists; **`CHECK`** constraints can only be added during initial table creation.
C. The **`activity_details`** table already contains records that violate the constraints; all existing data must pass **`CHECK`** constraints in order to add them to an existing table.
D. The **`activity_details`** table already contains records; **`CHECK`** constraints can only be added prior to inserting values into a table.
E. The current table schema does not contain the field **`valid_coordinates`**; schema evolution will need to be enabled before altering the table to add a constraint.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 39**
Which of the following is true of Delta Lake and the Lakehouse?

A. Because Parquet compresses data row by row. strings will only be compressed when a character is repeated multiple times.
B. Delta Lake automatically collects statistics on the first 32 columns of each table which are leveraged in data skipping based on query filters.
C. Views in the Lakehouse maintain a valid cache of the most recent versions of source tables at all times.
D. Primary and foreign key constraints can be leveraged to ensure duplicate values are never entered into a dimension table.
E. Z-order can only be applied to numeric values stored in Delta Lake tables.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 40**
The view **`updates`** represents an incremental batch of all newly ingested data to be inserted or updated in the **`customers`** table.

The following logic is used to process these records.

```
MERGE INTO customers
USING (
  SELECT updates.customer_id as merge_ey, updates.*
  FROM updates

  UNION ALL

  SELECT NULL as merge_key, updates.*
  FROM updates JOIN customers
  ON updates.customer_id = customers.customer_id
  WHERE customers.current = true AND updates.address <> customers.address
) staged_updates
ON customers.customer_id = mergeKey
WHEN MATCHED AND customers.current = true AND customers.address <> staged_updates.address THEN
  UPDATE SET current = false, end_date = staged_updates.effective_date
WHEN NOT MATCHED THEN
  INSERT(customer_id, address, current, effective_date, end_date)
  VALUES(staged_updates.customer_id, staged_updates.address, true, staged_updates.effective_date,
null)
```

Which statement describes this implementation?

A. The **`customers`** table is implemented as a Type 3 table; old values are maintained as a new column alongside the current value.
B. The **`customers`** table is implemented as a Type 2 table; old values are maintained but marked as no longer current and new values are inserted.
C. The **`customers`** table is implemented as a Type 0 table; all writes are append only with no changes to existing values.
D. The **`customers`** table is implemented as a Type 1 table; old values are overwritten by new values and no history is maintained.
E. The **`customers`** table is implemented as a Type 2 table; old values are overwritten and new customers are appended.

**Correct Answer:** B

**Section:** (none)

**Explanation/Reference:**

**QUESTION 41**
The DevOps team has configured a production workload as a collection of notebooks scheduled to run daily using the Jobs UI. A new data engineering hire is onboarding to the team and has requested access to one of these notebooks to review the production logic.

What are the maximum notebook permissions that can be granted to the user without allowing accidental changes to production code or data?

A. Can Manage
B. Can Edit
C. No permissions
D. Can Read
E. Can Run

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 42**
A table named **`user_ltv`** is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.

The **`user_ltv`** table has the following schema:

```
email STRING, age INT, ltv INT
```

The following view definition is executed:

```
CREATE VIEW email_ltv AS
SELECT
CASE WHEN
  is_member('marketing') THEN email
  ELSE 'REDACTED'
END AS email,
ltv
FROM user_ltv
```

An analyst who is not a member of the marketing group executes the following query:

```
SELECT * FROM email_ltv
```

Which statement describes the results returned by this query?

A. Three columns will be returned, but one column will be named "**`REDACTED`**" and contain only null values.
B. Only the **`email`** and **`ltv`** columns will be returned; the **`email`** column will contain all null values.

C. The **email** and **ltv** columns will be returned with the values in **user_ltv**.
D. The **email.age**, and **ltv** columns will be returned with the values in **user_ltv**.
E. Only the **email** and **ltv** columns will be returned; the **email** column will contain the string "**REDACTED**" in each row.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 43**
The data governance team has instituted a requirement that all tables containing Personal Identifiable Information (PII) must be clearly annotated. This includes adding column comments, table comments, and setting the custom table property **"contains_pii" = true**.

The following SQL DDL statement is executed to create a new table:

```
CREATE TABLE dev.pii_test
(id INT, name STRING COMMENT "PII")
COMMENT "Contains PII"
TBLPROPERTIES ('contains_pii' = True)
```

Which command allows manual confirmation that these three requirements have been met?

A. **DESCRIBE EXTENDED dev.pii_test**
B. **DESCRIBE DETAIL dev.pii_test**
C. **SHOW TBLPROPERTIES dev.pii_test**
D. **DESCRIBE HISTORY dev.pii_test**
E. **SHOW TABLES dev**

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 44**
The data governance team is reviewing code used for deleting records for compliance with GDPR. They note the following logic is used to delete records from the Delta Lake table named **users**.

```
DELETE FROM users
WHERE user_id IN
   (SELECT user_id FROM delete_requests)
```

Assuming that **user_id** is a unique identifying key and that **delete_requests** contains all users that have requested deletion, which statement describes whether successfully executing the above logic guarantees that the records to be deleted are no longer accessible and why?

A. Yes; Delta Lake ACID guarantees provide assurance that the **DELETE** command succeeded fully and permanently purged these records.
B. No; the Delta cache may return records from previous versions of the table until the cluster is restarted.

C. Yes; the Delta cache **immediately** updates to reflect the latest data files recorded to disk.
D. No; the Delta Lake **DELETE** command only provides ACID guarantees when combined with the **MERGE INTO** command.
E. No; files containing deleted records may still be accessible with time travel until a **VACUUM** command is used to remove invalidated data files.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 45**
An external object storage container has been mounted to the location `/mnt/finance_eda_bucket`.

The following logic was executed to create a database for the finance team:

```
CREATE DATABASE finance_eda_db
LOCATION '/mnt/finance_eda_bucket';
GRANT USAGE ON DATABASE finance_eda_db TO finance;
GRANT CREATE ON DATABASE finance_eda_db TO finance;
```

After the database was successfully created and permissions configured, a member of the finance team runs the following code:

```
CREATE TABLE finance_eda_db.tx_sales AS
SELECT *
FROM sales
WHERE state = "TX";
```

If all users on the finance team are members of the **finance** group, which statement describes how the **tx_sales** table will be created?

A. A logical table will persist the query plan to the Hive Metastore in the Databricks control plane.
B. An external table will be created in the storage container mounted to **/mnt/finance_eda_bucket**.
C. A logical table will persist the physical plan to the Hive Metastore in the Databricks control plane.
D. An managed table will be created in the storage container mounted to **/mnt/finance_eda_bucket**.
E. A managed table will be created in the DBFS root storage container.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 46**
Although the Databricks Utilities Secrets module provides tools to store sensitive credentials and avoid accidentally displaying them in plain text users should still be careful with which credentials are stored here and which users have access to using these secrets.

Which statement describes a limitation of Databricks Secrets?

A. Because the SHA256 hash is used to obfuscate stored secrets, reversing this hash will display the

value in plain text.

B. Account administrators can see all secrets in plain text by logging on to the Databricks Accounts console.
C. Secrets are stored in an administrators-only table within the Hive Metastore; database administrators have permission to query this table by default.
D. Iterating through a stored secret and printing each character will display secret contents in plain text.
E. The Databricks REST API can be used to list secrets in plain text if the personal access token has proper credentials.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 47**
What statement is true regarding the retention of job run history?

A. It is retained until you export or delete job run logs
B. It is retained for 30 days, during which time you can deliver job run logs to DBFS or S3
C. It is retained for 60 days, during which you can export notebook run results to HTML
D. It is retained for 60 days, after which logs are archived
E. It is retained for 90 days or until the run-id is re-used through custom run configuration

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**



**QUESTION 48**
A data engineer, User A, has promoted a new pipeline to production by using the REST API to programmatically create several jobs. A DevOps engineer, User B, has configured an external orchestration tool to trigger job runs through the REST API. Both users authorized the REST API calls using their personal access tokens.

Which statement describes the contents of the workspace audit logs concerning these events?

A. Because the REST API was used for job creation and triggering runs, a Service Principal will be automatically used to identify these events.
B. Because User B last configured the jobs, their identity will be associated with both the job creation events and the job run events.
C. Because these events are managed separately, User A will have their identity associated with the job creation events and User B will have their identity associated with the job run events.
D. Because the REST API was used for job creation and triggering runs, user identity will not be captured in the audit logs.
E. Because User A created the jobs, their identity will be associated with both the job creation events and the job run events.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 49**
A user new to Databricks is trying to troubleshoot long execution times for some pipeline logic they are working on. Presently, the user is executing code cell-by-cell, using `display()` calls to confirm code is producing the logically correct results as new transformations are added to an operation. To get a measure

of average time to execute, the user is running each cell multiple times interactively.

Which of the following adjustments will get a more accurate measure of how code is likely to perform in production?

A. Scala is the only language that can be accurately tested using interactive notebooks; because the best performance is achieved by using Scala code compiled to JARs, all PySpark and Spark SQL logic should be refactored.
B. The only way to meaningfully troubleshoot code execution times in development notebooks is to use production-sized data and production-sized clusters with Run All execution.
C. Production code development should only be done using an IDE; executing code against a local build of open source Spark and Delta Lake will provide the most accurate benchmarks for how code will perform in production.
D. Calling `display()` forces a job to trigger, while many transformations will only add to the logical query plan; because of caching, repeated execution of the same logic does not provide meaningful results.
E. The Jobs UI should be leveraged to occasionally run the notebook as a job and track execution time during incremental code development because Photon can only be enabled on clusters launched for scheduled jobs.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 50**
A production cluster has 3 executor nodes and uses the same virtual machine type for the driver and executor.

When evaluating the Ganglia Metrics for this cluster, which indicator would signal a bottleneck caused by code executing on the driver?

A. The five Minute Load Average remains consistent/flat
B. Bytes Received never exceeds 80 million bytes per second
C. Total Disk Space remains constant
D. Network I/O never spikes
E. Overall cluster CPU utilization is around 25%

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 51**
Where in the Spark UI can one diagnose a performance problem induced by not leveraging predicate push-down?

A. In the Executor's log file, by grepping for "predicate push-down"
B. In the Stage's Detail screen, in the Completed Stages table, by noting the size of data read from the Input column
C. In the Storage Detail screen, by noting which RDDs are not stored on disk
D. In the Delta Lake transaction log. by noting the column statistics
E. In the Query Detail screen, by interpreting the Physical Plan

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 52**
Review the following error traceback:

```
---------------------------------------------------------------------------
AnalysisException                         Traceback (most recent call last)
<command-3293767849433948> in <module>
----> 1 display(df.select(3*"heartrate"))

/databricks/spark/python/pyspark/sql/dataframe.py in select(self, *cols)
   1690         [Row(name='Alice', age=12), Row(name='Bob', age=15)]
   1691         """
-> 1692         jdf = self._jdf.select(self._jcols(*cols))
   1693         return DataFrame(jdf, self.sql_ctx)
   1694

/databricks/spark/python/lib/py4j-0.10.9-src.zip/py4j/java_gateway.py in __call__(self, *args)
   1302
   1303         answer = self.gateway_client.send_command(command)
-> 1304         return_value = get_return_value(
   1305             answer, self.gateway_client, self.target_id, self.name)
   1306

/databricks/spark/python/pyspark/sql/utils.py in deco(*a, **kw)
    121                 # Hide where the exception came from that shows a non-Pythonic
    122                 # JVM exception message.
--> 123                 raise converted from None
    124             else:
    125                 raise

AnalysisException: cannot resolve '`heartrateheartrateheartrate`' given input columns:
[spark_catalog.database.table.device_id, spark_catalog.database.table.heartrate,
spark_catalog.database.table.mrn, spark_catalog.database.table.time];
'Project ['heartrateheartrateheartrate]
+- SubqueryAlias spark_catalog.database.table
   +- Relation[device_id#75L,heartrate#76,mrn#77L,time#78] parquet
```

Which statement describes the error being raised?

A. The code executed was PySpark but was executed in a Scala notebook.
B. There is no column in the table named **heartrateheartrateheartrate**
C. There is a type error because a column object cannot be multiplied.
D. There is a type error because a DataFrame object cannot be multiplied.
E. There is a syntax error because the **`heartrate`** column is not correctly identified as a column.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 53**
Which distribution does Databricks support for installing custom Python code packages?

A. sbt
B. CRAN
C. npm
D. Wheels
E. jars

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 54**
Which Python variable contains a list of directories to be searched when trying to locate required modules?

A. importlib.resource_path
B. sys.path
C. os.path
D. pypi.path
E. pylib.source

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 55**
Incorporating unit tests into a PySpark application requires upfront attention to the design of your jobs, or a potentially significant refactoring of existing code.

Which statement describes a main benefit that offset this additional effort?

A. Improves the quality of your data
B. Validates a complete use case of your application
C. Troubleshooting is easier since all steps are isolated and tested individually
D. Yields faster deployment and execution times
E. Ensures that all steps interact correctly to achieve the desired end result

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 56**
Which statement describes integration testing?

A. Validates interactions between subsystems of your application
B. Requires an automated testing framework
C. Requires manual intervention
D. Validates an application use case
E. Validates behavior of individual elements of your application

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 57**
Which REST API call can be used to review the notebooks configured to run as tasks in a multi-task job?

A. `/jobs/runs/list`
B. `/jobs/runs/get-output`
C. `/jobs/runs/get`

D. `/jobs/get`
E. `/jobs/list`

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 58**
A Databricks job has been configured with 3 tasks, each of which is a Databricks notebook. Task A does not depend on other tasks. Tasks B and C run in parallel, with each having a serial dependency on task A.

If tasks A and B complete successfully but task C fails during a scheduled run, which statement describes the resulting state?

A. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; some operations in task C may have completed successfully.
B. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; any changes made in task C will be rolled back due to task failure.
C. All logic expressed in the notebook associated with task A will have been successfully completed; tasks B and C will not commit any changes because of stage failure.
D. Because all tasks are managed as a dependency graph, no changes will be committed to the Lakehouse until all tasks have successfully been completed.
E. Unless all tasks complete successfully, no changes will be committed to the Lakehouse; because task C failed, all commits will be rolled back automatically.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 59**
A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_stor
USING DELTA
LOCATION "/mnt/prod/sales_by_store"
```

Realizing that the original query had a typographical error, the below code was executed:

```
ALTER TABLE prod.sales_by_stor RENAME TO prod.sales_by_store
```

Which result will occur after running the second command?

A. The table reference in the metastore is updated and no data is changed.
B. The table name change is recorded in the Delta transaction log.
C. All related files and metadata are dropped and recreated in a single ACID transaction.
D. The table reference in the metastore is updated and all data files are moved.
E. A new Delta transaction log is created for the renamed table.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 60**
The data engineering team maintains a table of aggregate statistics through batch nightly updates. This includes total sales for the previous day alongside totals and averages for a variety of time periods including the 7 previous days, year-to-date, and quarter-to-date. This table is named **store_saies_summary** and the schema is as follows:

```
store_id INT, total_sales_qtd FLOAT, avg_daily_sales_qtd FLOAT,  total_sales_ytd FLOAT,
avg_daily_sales_ytd FLOAT, previous_day_sales FLOAT, total_sales_7d FLOAT, avg_daily_sales_7d
FLOAT, updated TIMESTAMP
```

The table **daily_store_sales** contains all the information needed to update **store_sales_summary**. The schema for this table is:

```
store_id INT, sales_date DATE, total_sales FLOAT
```

If **daily_store_sales** is implemented as a Type 1 table and the **total_sales** column might be adjusted after manual data auditing, which approach is the safest to generate accurate reports in the **store_sales_summary** table?

A. Implement the appropriate aggregate logic as a batch read against the **daily_store_sales** table and overwrite the **store_sales_summary** table with each Update.
B. Implement the appropriate aggregate logic as a batch read against the **daily_store_sales** table and append new rows nightly to the **store_sales_summary** table.
C. Implement the appropriate aggregate logic as a batch read against the **daily_store_sales** table and use upsert logic to update results in the **store_sales_summary** table.
D. Implement the appropriate aggregate logic as a Structured Streaming read against the **daily_store_sales** table and use upsert logic to update results in the **store_sales_summary** table.
E. Use Structured Streaming to subscribe to the change data feed for **daily_store_sales** and apply changes to the aggregates in the **store_sales_summary** table with each update.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 61**
A member of the data engineering team has submitted a short notebook that they wish to schedule as part of a larger data pipeline. Assume that the commands provided below produce the logically correct results when run as presented.

**Cmd 1**

```
rawDF = spark.table("raw_data")
```

**Cmd 2**

```
rawDF.printSchema()
```

**Cmd 3**

```
flattenedDF = rawDF.select("*", "values.*")
```

**Cmd 4**

```
finalDF = flattenedDF.drop("values")
```

**Cmd 5**

```
finalDF.explain()
```

**Cmd 6**

```
display(finalDF)
```

**Cmd 7**

```
finalDF.write.mode("append").saveAsTable("flat_data")
```



Which command should be removed from the notebook before scheduling it as a job?

A. Cmd 2
B. Cmd 3
C. Cmd 4
D. Cmd 5
E. Cmd 6

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 62**
The business reporting team requires that data for their dashboards be updated every hour. The total processing time for the pipeline that extracts, transforms, and loads the data for their pipeline runs in 10 minutes.

Assuming normal operating conditions, which configuration will meet their service-level agreement requirements with the lowest cost?

A. Manually trigger a job anytime the business reporting team refreshes their dashboards
B. Schedule a job to execute the pipeline once an hour on a new job cluster

C. Schedule a Structured Streaming job with a trigger interval of 60 minutes
D. Schedule a job to execute the pipeline once an hour on a dedicated interactive cluster
E. Configure a job that executes every time new data lands in a given directory

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 63**
A Databricks SQL dashboard has been configured to monitor the total number of records present in a collection of Delta Lake tables using the following query pattern:

```
SELECT COUNT (*) FROM table
```

Which of the following describes how results are generated each time the dashboard is updated?

A. The total count of rows is calculated by scanning all data files
B. The total count of rows will be returned from cached results unless REFRESH is run
C. The total count of records is calculated from the Delta transaction logs
D. The total count of records is calculated from the parquet file metadata
E. The total count of records is calculated from the Hive metastore

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**



**QUESTION 64**
A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_store
AS (
  SELECT *
  FROM prod.sales a
  INNER JOIN prod.store b
  ON a.store_id = b.store_id
)
```

Consider the following query:

```
DROP TABLE prod.sales_by_store
```

If this statement is executed by a workspace admin, which result will occur?

A. Nothing will occur until a COMMIT command is executed.
B. The table will be removed from the catalog but the data will remain in storage.
C. The table will be removed from the catalog and the data will be deleted.
D. An error will occur because Delta Lake prevents the deletion of production data.
E. Data will be marked as deleted but still recoverable with Time Travel.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 65**
Two of the most common data locations on Databricks are the DBFS root storage and external object storage mounted with **`dbutils.fs.mount()`**.

Which of the following statements is correct?

A. DBFS is a file system protocol that allows users to interact with files stored in object storage using syntax and guarantees similar to Unix file systems.
B. By default, both the DBFS root and mounted data sources are only accessible to workspace administrators.
C. The DBFS root is the most secure location to store data, because mounted storage volumes must have full public read and write permissions.
D. Neither the DBFS root nor mounted storage can be accessed when using **`%sh`** in a Databricks notebook.
E. The DBFS root stores files in ephemeral block volumes attached to the driver, while mounted directories will always persist saved data to external storage between sessions.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**



**QUESTION 66**
The following code has been migrated to a Databricks notebook from a legacy workload:

```
%sh
git clone https://github.com/foo/data_loader;
python ./data_loader/run.py;
mv ./output /dbfs/mnt/new_data
```

The code executes successfully and provides the logically correct results, however, it takes over 20 minutes to extract and load around 1 GB of data.

Which statement is a possible explanation for this behavior?

A. **%sh** triggers a cluster restart to collect and install Git. Most of the latency is related to cluster startup time.
B. Instead of cloning, the code should use **%sh pip install** so that the Python code can get executed in parallel across all nodes in a cluster.
C. **%sh** does not distribute file moving operations; the final line of code should be updated to use **%fs** instead.
D. Python will always execute slower than Scala on Databricks. The **run.py** script should be refactored to Scala.
E. **%sh** executes shell code on the driver node. The code does not take advantage of the worker nodes or Databricks optimized Spark.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 67**
The data science team has requested assistance in accelerating queries on free-form text from user reviews. The data is currently stored in Parquet with the below schema:

```
item_id INT, user_id INT, review_id INT, rating FLOAT, review STRING
```

The **`review`** column contains the full text of the review left by the user. Specifically, the data science team is looking to identify if any of 30 key words exist in this field.

A junior data engineer suggests converting this data to Delta Lake will improve query performance.

Which response to the junior data engineer's suggestion is correct?

A. Delta Lake statistics are not optimized for free text fields with high cardinality.
B. Text data cannot be stored with Delta Lake.
C. ZORDER ON review will need to be run to see performance gains.
D. The Delta log creates a term matrix for free text fields to support selective filtering.
E. Delta Lake statistics are only collected on the first 4 columns in a table.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 68**
Assuming that the Databricks CLI has been installed and configured correctly, which Databricks CLI command can be used to upload a custom Python Wheel to object storage mounted with the DBFS for use with a production job?

A. `configure`
B. `fs`
C. `jobs`
D. `libraries`
E. `workspace`

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 69**
The business intelligence team has a dashboard configured to track various summary metrics for retail stores. This includes total sales for the previous day alongside totals and averages for a variety of time periods. The fields required to populate this dashboard have the following schema:

```
store_id INT, total_sales_qtd FLOAT, avg_daily_sales_qtd FLOAT,  total_sales_ytd
FLOAT, avg_daily_sales_ytd FLOAT, previous_day_sales FLOAT, total_sales_7d FLOAT,
avg_daily_sales_7d FLOAT, updated TIMESTAMP
```

For demand forecasting, the Lakehouse contains a validated table of all itemized sales updated incrementally in near real-time. This table, named **`products_per_order`**, includes the following fields:

```
store_id INT, order_id INT, product_id INT, quantity INT, price FLOAT,
order_timestamp TIMESTAMP
```

Because reporting on long-term sales trends is less volatile, analysts using the new dashboard only require data to be refreshed once daily. Because the dashboard will be queried interactively by many users throughout a normal business day, it should return results quickly and reduce total compute associated with each materialization.

Which solution meets the expectations of the end users while controlling and limiting possible costs?

A. Populate the dashboard by configuring a nightly batch job to save the required values as a table overwritten with each update.
B. Use Structured Streaming to configure a live dashboard against the **`products_per_order`** table within a Databricks notebook.
C. Configure a webhook to execute an incremental read against **`products_per_order`** each time the dashboard is refreshed.
D. Use the Delta Cache to persist the **`products_per_order`** table in memory to quickly update the dashboard with each query.
E. Define a view against the **`products_per_order`** table and define the dashboard against this view.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 70**
A data ingestion task requires a one-TB JSON dataset to be written out to Parquet with a target part-file size of 512 MB. Because Parquet is being used instead of Delta Lake, built-in file-sizing features such as Auto-Optimize & Auto-Compaction cannot be used.

Which strategy will yield the best performance without shuffling data?

A. Set **`spark.sql.files.maxPartitionBytes`** to 512 MB, ingest the data, execute the narrow transformations, and then write to parquet.
B. Set **`spark.sql.shuffle.partitions`** to 2,048 partitions (1TB*1024*1024/512), ingest the data, execute the narrow transformations, optimize the data by sorting it (which automatically repartitions the data), and then write to parquet.
C. Set **`spark.sql.adaptive.advisoryPartitionSizeInBytes`** to 512 MB bytes, ingest the data, execute the narrow transformations, coalesce to 2,048 partitions (1TB*1024*1024/512), and then write to parquet.
D. Ingest the data, execute the narrow transformations, repartition to 2,048 partitions (1TB* 1024*1024/512), and then write to parquet.
E. Set **`spark.sql.shuffle.partitions`** to 512, ingest the data, execute the narrow transformations, and then write to parquet.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 71**
A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame **`df`**. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Incremental state information should be maintained for 10 minutes for late-arriving data.

Streaming DataFrame **`df`** has the following schema:

```
"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"
```

Code block:

```
df._____
    .groupBy(
        window("event_time", "5 minutes").alias("time"),
        "device_id"
    )
    .agg(
        avg("temp").alias("avg_temp"),
        avg("humidity").alias("avg_humidity")
    )
    .writeStream
    .format("delta")
    .saveAsTable("sensor_avg")
```

Choose the response that correctly fills in the blank within the code block to complete this task.

A. `withWatermark("event_time", "10 minutes")`
B. `awaitArrival("event_time", "10 minutes")`
C. `await("event_time + '10 minutes'")`
D. `slidingWindow("event_time", "10 minutes")`
E. `delayWrite("event_time", "10 minutes")`

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 72**
A data team's Structured Streaming job is configured to calculate running aggregates for item sales to update a downstream marketing dashboard. The marketing team has introduced a new promotion, and they would like to add a new field to track the number of times this promotion code is used for each item. A junior data engineer suggests updating the existing query as follows. Note that proposed changes are in **bold**.

Original query:

```
df.groupBy("item")
    .agg(count("item").alias("total_count"),
         mean("sale_price").alias("avg_price"))
    .writeStream
    .outputMode("complete")
    .option("checkpointLocation", "/item_agg/__checkpoint")
    .start("/item_agg")
```

Proposed query:

```
df.groupBy("item")
    .agg(count("item").alias("total_count"),
         mean("sale_price").alias("avg_price"),
         count("promo_code = 'NEW_MEMBER'").alias("new_member_promo"))
    .writeStream
    .outputMode("complete")
    .option('mergeSchema', 'true')
    .option("checkpointLocation", "/item_agg/__checkpoint")
    .start("/item_agg")
```

Which step must also be completed to put the proposed query into production?

A. Specify a new **`checkpointLocation`**
B. Increase the shuffle partitions to account for additional aggregates
C. Run **`REFRESH TABLE delta.'/item_agg'`**
D. Register the data in the **`"/item_agg"`** directory to the Hive metastore
E. Remove **`.option('mergeSchema', 'true')`** from the streaming write

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 73**
A Structured Streaming job deployed to production has been resulting in higher than expected cloud storage costs. At present, during normal execution, each microbatch of data is processed in less than 3s; at least 12 times per minute, a microbatch is processed that contains 0 records. The streaming write was configured using the default trigger settings. The production job is currently scheduled alongside many other Databricks jobs in a workspace with instance pools provisioned to reduce start-up time for jobs with batch execution.

Holding all other variables constant and assuming records need to be processed in less than 10 minutes, which adjustment will meet the requirement?

A. Set the trigger interval to 3 seconds; the default trigger interval is consuming too many records per batch, resulting in spill to disk that can increase volume costs.
B. Increase the number of shuffle partitions to maximize parallelism, since the trigger interval cannot be modified without modifying the checkpoint directory.
C. Set the trigger interval to 10 minutes; each batch calls APIs in the source storage account, so decreasing trigger frequency to maximum allowable threshold should minimize this cost.
D. Set the trigger interval to 500 milliseconds; setting a small but non-zero trigger interval ensures that the source is not queried too frequently.
E. Use the trigger once option and configure a Databricks job to execute the query every 10 minutes; this approach minimizes costs for both compute and storage.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 74**
Which statement describes the correct use of **`pyspark.sql.functions.broadcast`**?

A. It marks a column as having low enough cardinality to properly map distinct values to available

partitions, allowing a broadcast join.

B. It marks a column as small enough to store in memory on all executors, allowing a broadcast join.
C. It caches a copy of the indicated table on attached storage volumes for all active clusters within a Databricks workspace.
D. It marks a DataFrame as small enough to store in memory on all executors, allowing a broadcast join.
E. It caches a copy of the indicated table on all nodes in the cluster for use in all future queries during the cluster lifetime.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 75**
A data engineer is configuring a pipeline that will potentially see late-arriving, duplicate records.

In addition to de-duplicating records within the batch, which of the following approaches allows the data engineer to deduplicate data against previously processed records as it is inserted into a Delta table?

A. Set the configuration `delta.deduplicate = true`.
B. `VACUUM` the Delta table after each batch completes.
C. Perform an insert-only merge with a matching condition on a unique key.
D. Perform a full outer join on a unique key and overwrite existing data.
E. Rely on Delta Lake schema enforcement to prevent duplicate records.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 76**
A data pipeline uses Structured Streaming to ingest data from Apache Kafka to Delta Lake. Data is being stored in a bronze table, and includes the Kafka-generated timestamp, key, and value. Three months after the pipeline is deployed, the data engineering team has noticed some latency issues during certain times of the day.

A senior data engineer updates the Delta Table's schema and ingestion logic to include the current timestamp (as recorded by Apache Spark) as well as the Kafka topic and partition. The team plans to use these additional metadata fields to diagnose the transient processing delays.

Which limitation will the team face while diagnosing this problem?

A. New fields will not be computed for historic records.
B. Spark cannot capture the topic and partition fields from a Kafka source.
C. New fields cannot be added to a production Delta table.
D. Updating the table schema will invalidate the Delta transaction log metadata.
E. Updating the table schema requires a default value provided for each field added.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 77**
In order to facilitate near real-time workloads, a data engineer is creating a helper function to leverage the schema detection and evolution functionality of Databricks Auto Loader. The desired function will

automatically detect the schema of the source directly, incrementally process JSON files as they arrive in a source directory, and automatically evolve the schema of the table when new fields are detected.

The function is displayed below with a blank:

```
def auto_load_json(source_path: str,
                   checkpoint_path: str,
                   target_table_path: str):
    (spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.schemaLocation", checkpoint_path)
        .load(source_path)
        ______
    )
```

Which response correctly fills in the blank to meet the specified requirements?

A.
```
.writeStream
.option("mergeSchema", True)
.start(target_table_path)
```

B.
```
.writeStream
.option("checkpointLocation", checkpoint_path)
.option("mergeSchema", True)
.trigger(once=True)
.start(target_table_path)
```

C.
```
.write
.option("checkpointLocation", checkpoint_path)
.option("mergeSchema", True)
.outputMode("append")
.save(target_table_path)
```

D.
```
.write
.option("mergeSchema", True)
.mode("append")
.save(target_table_path)
```

E.
```
.writeStream
.option("checkpointLocation", checkpoint_path)
.option("mergeSchema", True)
.start(target_table_path)
```

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 78**
The data engineering team maintains the following code:

```
import pyspark.sql.functions as F

(spark.table("silver_customer_sales")
  .groupBy("customer_id")
  .agg(
    F.min("sale_date").alias("first_transaction_date"),
    F.max("sale_date").alias("last_transaction_date"),
    F.mean("sale_total").alias("average_sales"),
    F.countDistinct("order_id").alias("total_orders"),
    F.sum("sale_total").alias("lifetime_value")
  ).write
  .mode("overwrite")
  .table("gold_customer_lifetime_sales_summary")
)
```

Assuming that this code produces logically correct results and the data in the source table has been de-duplicated and validated, which statement describes what will occur when this code is executed?

A. The **`silver_customer_sales`** table will be overwritten by aggregated values calculated from all records in the **`gold_customer_lifetime_sales_summary`** table as a batch job.
B. A batch job will update the **`gold_customer_lifetime_sales_summary`** table, replacing only those rows that have different values than the current version of the table, using **`customer_id`** as the primary key.
C. The **`gold_customer_lifetime_sales_summary`** table will be overwritten by aggregated values calculated from all records in the **`silver_customer_sales`** table as a batch job.
D. An incremental job will leverage running information in the state store to update aggregate values in the **`gold_customer_lifetime_sales_summary`** table.
E. An incremental job will detect if new rows have been written to the **`silver_customer_sales`** table; if new rows are detected, all aggregates will be recalculated and used to overwrite the **`gold_customer_lifetime_sales_summary`** table.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 79**
The data architect has mandated that all tables in the Lakehouse should be configured as external (also known as "unmanaged") Delta Lake tables.

Which approach will ensure that this requirement is met?

A. When a database is being created, make sure that the `LOCATION` keyword is used.
B. When configuring an external data warehouse for all table storage, leverage Databricks for all ELT.
C. When data is saved to a table, make sure that a full file path is specified alongside the Delta format.
D. When tables are created, make sure that the `EXTERNAL` keyword is used in the `CREATE TABLE` statement.
E. When the workspace is being configured, make sure that external cloud object storage has been mounted.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 80**
The marketing team is looking to share data in an aggregate table with the sales organization, but the field names used by the teams do not match, and a number of marketing-specific fields have not been approved for the sales org.

Which of the following solutions addresses the situation while emphasizing simplicity?

A. Create a view on the marketing table selecting only those fields approved for the sales team; alias the names of any fields that should be standardized to the sales naming conventions.
B. Create a new table with the required schema and use Delta Lake's `DEEP CLONE` functionality to sync up changes committed to one table to the corresponding table.
C. Use a CTAS statement to create a derivative table from the marketing table; configure a production job to propagate changes.
D. Add a parallel table write to the current production pipeline, updating a new sales table that varies as required from the marketing table.
E. Instruct the marketing team to download results as a CSV and email them to the sales organization.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 81**
A `CHECK` constraint has been successfully added to the Delta table named `activity_details` using the following logic:

```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
  latitude >= -90 AND
  latitude <= 90 AND
  longitude >= -180 AND
  longitude <= 180);
```

A batch job is attempting to insert new records to the table, including a record where `latitude = 45.50` and `longitude = 212.67`.

Which statement describes the outcome of this batch insert?

A. The write will fail when the violating record is reached; any records previously processed will be recorded to the target table.
B. The write will fail completely because of the constraint violation and no records will be inserted into the target table.
C. The write will insert all records except those that violate the table constraints; the violating records will be recorded to a quarantine table.
D. The write will include all records in the target table; any violations will be indicated in the boolean column named **`valid_coordinates`**.
E. The write will insert all records except those that violate the table constraints; the violating records will be reported in a warning log.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 82**
A junior data engineer has manually configured a series of jobs using the Databricks Jobs UI. Upon reviewing their work, the engineer realizes that they are listed as the "Owner" for each job. They attempt to transfer "Owner" privileges to the "DevOps" group, but cannot successfully accomplish this task.

Which statement explains what is preventing this privilege transfer?

A. Databricks jobs must have exactly one owner; "Owner" privileges cannot be assigned to a group.
B. The creator of a Databricks job will always have "Owner" privileges; this configuration cannot be changed.
C. Other than the default "admins" group, only individual users can be granted privileges on jobs.
D. A user can only transfer job ownership to a group if they are also a member of that group.
E. Only workspace administrators can grant "Owner" privileges to a group.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 83**
All records from an Apache Kafka producer are being ingested into a single Delta Lake table with the following schema:

```
key BINARY, value BINARY, topic STRING, partition LONG, offset LONG, timestamp LONG
```

There are 5 unique topics being ingested. Only the "**`registration`**" topic contains Personal Identifiable Information (PII). The company wishes to restrict access to PII. The company also wishes to only retain records containing PII in this table for 14 days after initial ingestion. However, for non-PII information, it would like to retain these records indefinitely.

Which of the following solutions meets the requirements?

A. All data should be deleted biweekly; Delta Lake's time travel functionality should be leveraged to maintain a history of non-PII information.
B. Data should be partitioned by the **`registration`** field, allowing ACLs and delete statements to be set for the PII directory.
C. Because the value field is stored as binary data, this information is not considered PII and no special precautions should be taken.
D. Separate object storage containers should be specified based on the **`partition`** field, allowing isolation at the storage level.

E. Data should be partitioned by the `topic` field, allowing ACLs and delete statements to leverage partition boundaries.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 84**
The data architect has decided that once data has been ingested from external sources into the Databricks Lakehouse, table access controls will be leveraged to manage permissions for all production tables and views.

The following logic was executed to grant privileges for interactive queries on a production database to the core engineering group.

```
GRANT USAGE ON DATABASE prod TO eng;
GRANT SELECT ON DATABASE prod TO eng;
```

Assuming these are the only privileges that have been granted to the eng group along with permission on the catalog, and that these users are not workspace administrators, which statement describes their privileges?

A. Group members have full permissions on the `prod` database and can also assign permissions to other users or groups.
B. Group members are able to list all tables in the `prod` database but are not able to see the results of any queries on those tables.
C. Group members are able to query and modify all tables and views in the `prod` database, but cannot create new tables or views.
D. Group members are able to query all tables and views in the `prod` database, but cannot create or edit anything in the database.
E. Group members are able to create, query, and modify all tables and views in the `prod` database, but cannot define custom functions.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 85**
A distributed team of data analysts share computing resources on an interactive cluster with autoscaling configured. In order to better manage costs and query throughput, the workspace administrator is hoping to evaluate whether cluster upscaling is caused by many concurrent users or resource-intensive queries.

In which location can one review the timeline for cluster resizing events?

A. Workspace audit logs
B. Driver's log file
C. Ganglia
D. Cluster Event Log
E. Executor's log file

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 86**
When evaluating the Ganglia Metrics for a given cluster with 3 executor nodes, which indicator would signal proper utilization of the VM's resources?

A. The five Minute Load Average remains consistent/flat
B. Bytes Received never exceeds 80 million bytes per second
C. Network I/O never spikes
D. Total Disk Space remains constant
E. CPU Utilization is around 75%

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 87**
Which of the following technologies can be used to identify key areas of text when parsing Spark Driver log4j output?

A. Regex
B. Julia
C. pyspsark.ml.feature
D. Scala Datasets
E. C++

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**



**QUESTION 88**
You are testing a collection of mathematical functions, one of which calculates the area under a curve as described by another function.

```
assert(myIntegrate(lambda x: x*x, 0, 3) [0] == 9)
```

Which kind of test does the above line exemplify?

A. Unit
B. Manual
C. Functional
D. Integration
E. End-to-end

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 89**
A Databricks job has been configured with 3 tasks, each of which is a Databricks notebook. Task A does not depend on other tasks. Tasks B and C run in parallel, with each having a serial dependency on Task A.

If task A fails during a scheduled run, which statement describes the results of this run?

A. Because all tasks are managed as a dependency graph, no changes will be committed to the

Lakehouse until all tasks have successfully been completed.

B. Tasks B and C will attempt to run as configured; any changes made in task A will be rolled back due to task failure.
C. Unless all tasks complete successfully, no changes will be committed to the Lakehouse; because task A failed, all commits will be rolled back automatically.
D. Tasks B and C will be skipped; some logic expressed in task A may have been committed before task failure.
E. Tasks B and C will be skipped; task A will not commit any changes because of stage failure.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 90**
Which statement regarding Spark configuration on the Databricks platform is true?

A. The Databricks REST API can be used to modify the Spark configuration properties for an interactive cluster without interrupting jobs currently running on the cluster.
B. Spark configurations set within a notebook will affect all SparkSessions attached to the same interactive cluster.
C. Spark configuration properties can only be set for an interactive cluster by creating a global init script.
D. Spark configuration properties set for an interactive cluster with the Clusters UI will impact all notebooks attached to that cluster.
E. When the same Spark configuration property is set for an interactive cluster and a notebook attached to that cluster, the notebook setting will always be ignored.



**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 91**
A developer has successfully configured their credentials for Databricks Repos and cloned a remote Git repository. They do not have privileges to make changes to the `main` branch, which is the only branch currently visible in their workspace.

Which approach allows this user to share their code updates without the risk of overwriting the work of their teammates?

A. Use Repos to checkout all changes and send the `git diff` log to the team.
B. Use Repos to create a fork of the remote repository, commit all changes, and make a pull request on the source repository.
C. Use Repos to pull changes from the remote Git repository; commit and push changes to a branch that appeared as changes were pulled.
D. Use Repos to merge all differences and make a pull request back to the remote repository.
E. Use Repos to create a new branch, commit all changes, and push changes to the remote Git repository.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 92**
In order to prevent accidental commits to production data, a senior data engineer has instituted a policy

that all development work will reference clones of Delta Lake tables. After testing both **DEEP** and **SHALLOW CLONE**, development tables are created using **SHALLOW CLONE**.

A few weeks after initial table creation, the cloned versions of several tables implemented as Type 1 Slowly Changing Dimension (SCD) stop working. The transaction logs for the source tables show that **VACUUM** was run the day before.

Which statement describes why the cloned tables are no longer working?

A. Because Type 1 changes overwrite existing records, Delta Lake cannot guarantee data consistency for cloned tables.
B. Running **VACUUM** automatically invalidates any shallow clones of a table; **DEEP CLONE** should always be used when a cloned table will be repeatedly queried.
C. Tables created with **SHALLOW CLONE** are automatically deleted after their default retention threshold of 7 days.
D. The metadata created by the **CLONE** operation is referencing data files that were purged as invalid by the **VACUUM** command.
E. The data files compacted by **VACUUM** are not tracked by the cloned metadata; running **REFRESH** on the cloned table will pull in recent changes.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 93**
You are performing a join operation to combine values from a static **userLookup** table with a streaming DataFrame **streamingDF**.

Which code block attempts to perform an invalid stream-static join?

A. `userLookup.join(streamingDF, ["userid"], how="inner")`
B. `streamingDF.join(userLookup, ["user_id"], how="outer")`
C. `streamingDF.join(userLookup, ["user_id”], how="left")`
D. `streamingDF.join(userLookup, ["userid"], how="inner")`
E. `userLookup.join(streamingDF, ["user_id"], how="right")`

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 94**
Spill occurs as a result of executing various wide transformations. However, diagnosing spill requires one to proactively look for key indicators.

Where in the Spark UI are two of the primary indicators that a partition is spilling to disk?

A. Query’s detail screen and Job’s detail screen
B. Stage’s detail screen and Executor’s log files
C. Driver’s and Executor’s log files
D. Executor’s detail screen and Executor’s log files
E. Stage’s detail screen and Query’s detail screen

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 95**
A task orchestrator has been configured to run two hourly tasks. First, an outside system writes Parquet data to a directory mounted at `/mnt/raw_orders/`. After this data is written, a Databricks job containing the following code is executed:

```
(spark.readStream
  .format("parquet")
  .load("/mnt/raw_orders/")
  .withWatermark("time", "2 hours")
  .dropDuplicates(["customer_id", "order_id"])
  .writeStream
  .trigger(once=True)
  .table("orders")
)
```

Assume that the fields `customer_id` and `order_id` serve as a composite key to uniquely identify each order, and that the `time` field indicates when the record was queued in the source system.

If the upstream system is known to occasionally enqueue duplicate entries for a single order hours apart, which statement is correct?

A. Duplicate records enqueued more than 2 hours apart may be retained and the `orders` table may contain duplicate records with the same `customer_id` and `order_id`.
B. All records will be held in the state store for 2 hours before being deduplicated and committed to the `orders` table.
C. The `orders` table will contain only the most recent 2 hours of records and no duplicates will be present.
D. Duplicate records arriving more than 2 hours apart will be dropped, but duplicates that arrive in the same batch may both be written to the `orders` table.
E. The `orders` table will not contain duplicates, but records arriving more than 2 hours late will be ignored and missing from the table.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 96**
A junior data engineer is migrating a workload from a relational database system to the Databricks Lakehouse. The source system uses a star schema, leveraging foreign key constraints and multi-table inserts to validate records on write.

Which consideration will impact the decisions made by the engineer while migrating this workload?

A. Databricks only allows foreign key constraints on hashed identifiers, which avoid collisions in highly-parallel writes.
B. Databricks supports Spark SQL and JDBC; all logic can be directly migrated from the source system

without refactoring.

C. Committing to multiple tables simultaneously requires taking out multiple table locks and can lead to a state of deadlock.
D. All Delta Lake transactions are ACID compliant against a single table, and Databricks does not enforce foreign key constraints.
E. Foreign keys must reference a primary key field; multi-table inserts must leverage Delta Lake's upsert functionality.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 97**
A data architect has heard about Delta Lake's built-in versioning and time travel capabilities. For auditing purposes, they have a requirement to maintain a full record of all valid street addresses as they appear in the **customers** table.

The architect is interested in implementing a Type 1 table, overwriting existing records with new values and relying on Delta Lake time travel to support long-term auditing. A data engineer on the project feels that a Type 2 table will provide better performance and scalability.

Which piece of information is critical to this decision?

A. Data corruption can occur if a query fails in a partially completed state because Type 2 tables require setting multiple fields in a single update.
B. Shallow clones can be combined with Type 1 tables to accelerate historic queries for long-term versioning.
C. Delta Lake time travel cannot be used to query previous versions of these tables because Type 1 changes modify data files in place.
D. Delta Lake time travel does not scale well in cost or latency to provide a long-term versioning solution.
E. Delta Lake only supports Type 0 tables; once records are inserted to a Delta Lake table, they cannot be modified.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 98**
A table named **`user_ltv`** is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.

The **`user_ltv`** table has the following schema:

```
email STRING, age INT, ltv INT
```

The following view definition is executed:

```
CREATE VIEW user_ltv_no_minors AS
SELECT email, age, ltv
FROM user_ltv
WHERE
  CASE
    WHEN is_member("auditing") THEN TRUE
    ELSE age >= 18
  END
```

An analyst who is not a member of the auditing group executes the following query:

```
SELECT * FROM user_ltv_no_minors
```

Which statement describes the results returned by this query?

A. All columns will be displayed normally for those records that have an **`age`** greater than 17; records not meeting this condition will be omitted.
B. All age values less than 18 will be returned as null values, all other columns will be returned with the values in **`user_ltv`**.
C. All values for the age column will be returned as null values, all other columns will be returned with the values in **`user_ltv`**.
D. All records from all columns will be displayed with the values in **`user_ltv`**.
E. All columns will be displayed normally for those records that have an **`age`** greater than 18; records not meeting this condition will be omitted.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 99**
The data governance team is reviewing code used for deleting records for compliance with GDPR. The following logic has been implemented to propagate delete requests from the **`user_lookup`** table to the **`user_aggregates`** table.

```
(spark.read
  .format("delta")
  .option("readChangeData", True)
  .option("startingTimestamp", '2021-08-22 00:00:00')
  .option("endingTimestamp", '2021-08-29 00:00:00')
  .table("user_lookup")
  .createOrReplaceTempView("changes"))

spark.sql("""
  DELETE FROM user_aggregates
  WHERE user_id IN (
    SELECT user_id
    FROM changes
    WHERE _change_type='delete'
    )
""")
```

Assuming that **`user_id`** is a unique identifying key and that all users that have requested deletion have been removed from the **`user_lookup`** table, which statement describes whether successfully executing the above logic guarantees that the records to be deleted from the **`user_aggregates`** table are no longer accessible and why?

A. No; the Delta Lake **`DELETE`** command only provides ACID guarantees when combined with the **`MERGE INTO`** command.
B. No; files containing deleted records may still be accessible with time travel until a **`VACUUM`** command is used to remove invalidated data files.
C. Yes; the change data feed uses foreign keys to ensure delete consistency throughout the Lakehouse.
D. Yes; Delta Lake ACID guarantees provide assurance that the **`DELETE`** command succeeded fully and permanently purged these records.
E. No; the change data feed only tracks inserts and updates, not deleted records.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 100**
The data engineering team has been tasked with configuring connections to an external database that does not have a supported native connector with Databricks. The external database already has data security configured by group membership. These groups map directly to user groups already created in Databricks that represent various teams within the company.

A new login credential has been created for each group in the external database. The Databricks Utilities Secrets module will be used to make these credentials available to Databricks users.

Assuming that all the credentials are configured correctly on the external database and group membership is properly configured on Databricks, which statement describes how teams can be granted the minimum necessary access to using these credentials?

A. "Manage" permissions should be set on a secret key mapped to those credentials that will be used by a given team.
B. "Read" permissions should be set on a secret key mapped to those credentials that will be used by a

given team.

C. "Read" permissions should be set on a secret scope containing only those credentials that will be used by a given team.

D. "Manage" permissions should be set on a secret scope containing only those credentials that will be used by a given team.

E. No additional configuration is necessary as long as all users are configured as administrators in the workspace where secrets have been added.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 101**
Which indicators would you look for in the Spark UI's Storage tab to signal that a cached table is not performing optimally? Assume you are using Spark's **`MEMORY_ONLY`** storage level.

A. Size on Disk is < Size in Memory

B. The RDD Block Name includes the "*" annotation signaling a failure to cache

C. Size on Disk is > 0

D. The number of Cached Partitions > the number of Spark Partitions

E. On Heap Memory Usage is within 75% of Off Heap Memory Usage

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**



**QUESTION 102**
What is the first line of a Databricks Python notebook when viewed in a text editor?

A. `%python`

B. `// Databricks notebook source`

C. `# Databricks notebook source`

D. `-- Databricks notebook source`

E. `# MAGIC %python`

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 103**
Which statement describes a key benefit of an end-to-end test?

A. Makes it easier to automate your test suite

B. Pinpoints errors in the building blocks of your application

C. Provides testing coverage for all code paths and branches

D. Closely simulates real world usage of your application

E. Ensures code is optimized for a real-life workflow

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 104**
The Databricks CLI is used to trigger a run of an existing job by passing the **`job_id`** parameter. The response that the job run request has been submitted successfully includes a field **`run_id`**.

Which statement describes what the number alongside this field represents?

A. The **`job_id`** and number of times the job has been run are concatenated and returned.
B. The total number of jobs that have been run in the workspace.
C. The number of times the job definition has been run in this workspace.
D. The **`job_id`** is returned in this field.
E. The globally unique ID of the newly triggered run.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 105**
The data science team has created and logged a production model using MLflow. The model accepts a list of column names and returns a new column of type **`DOUBLE`**.

The following code correctly imports the production model, loads the **`customers`** table containing the **`customer_id`** key column into a DataFrame, and defines the feature columns needed for the model.

```
model = mlflow.pyfunc.spark_udf(spark, model_uri="models:/churn/prod")
df = spark.table("customers")
columns = ["account_age", "time_since_last_seen", "app_rating"]
```

Which code block will output a DataFrame with the schema "**`customer_id LONG, predictions DOUBLE`**"?

A. `df.map(lambda x:model(x[columns])).select("customer_id, predictions")`
B. `df.select("customer_id", model(*columns).alias("predictions"))`
C. `model.predict(df, columns)`
D. `df.select("customer_id", pandas_udf(model, columns).alias("predictions"))`
E. `df.apply(model, columns).select("customer_id, predictions")`

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 106**
A nightly batch job is configured to ingest all data files from a cloud object storage container where records are stored in a nested directory structure **`YYYY/MM/DD`**. The data for each date represents all records that were processed by the source system on that date, noting that some records may be delayed as they await moderator approval. Each entry represents a user review of a product and has the following schema:

```
user_id STRING, review_id BIGINT, product_id BIGINT, review_timestamp
TIMESTAMP, review_text STRING
```

The ingestion job is configured to append all data for the previous date to a target table **`reviews_raw`** with an identical schema to the source system. The next step in the pipeline is a batch write to propagate all new records inserted into **`reviews_raw`** to a table where data is fully deduplicated, validated, and

enriched.

Which solution minimizes the compute costs to propagate this batch of data?

A. Perform a batch read on the **`reviews_raw`** table and perform an insert-only merge using the natural composite key **`user_id`**, **`review_id`**, **`product_id`**, **`review_timestamp`**.
B. Configure a Structured Streaming read against the **`reviews_raw`** table using the trigger once execution mode to process new records as a batch job.
C. Use Delta Lake version history to get the difference between the latest version of **`reviews_raw`** and one version prior, then write these records to the next table.
D. Filter all records in the **`reviews_raw`** table based on the **`review_timestamp`**; batch append those records produced in the last 48 hours.
E. Reprocess all records in **`reviews_raw`** and overwrite the next table in the pipeline.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 107**
Which statement describes Delta Lake optimized writes?

A. Before a Jobs cluster terminates, **`OPTIMIZE`** is executed on all tables modified during the most recent job.
B. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an **`OPTIMIZE`** job is executed toward a default of 1 GB.
C. Data is queued in a messaging bus instead of committing data directly to memory; all data is committed from the messaging bus in one batch once the job is complete.
D. Optimized writes use logical partitions instead of directory partitions; because partition boundaries are only represented in metadata, fewer small files are written.
E. A shuffle occurs prior to writing to try to group similar data together resulting in fewer files instead of each executor writing multiple files based on directory partitions.

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 108**
Which statement describes the default execution mode for Databricks Auto Loader?

A. Cloud vendor-specific queue storage and notification services are configured to track newly arriving files; the target table is materialized by directly querying all valid files in the source directory.
B. New files are identified by listing the input directory; the target table is materialized by directly querying all valid files in the source directory.
C. Webhooks trigger a Databricks job to run anytime new data arrives in a source directory; new data are automatically merged into target tables using rules inferred from the data.
D. New files are identified by listing the input directory; new files are incrementally and idempotently loaded into the target Delta Lake table.
E. Cloud vendor-specific queue storage and notification services are configured to track newly arriving files; new files are incrementally and idempotently loaded into the target Delta Lake table.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 109**
A Delta Lake table representing metadata about content posts from users has the following schema:

```
user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude
FLOAT, post_time TIMESTAMP, date DATE
```

Based on the above schema, which column is a good candidate for partitioning the Delta Table?

A. `post_time`
B. `latitude`
C. `post_id`
D. `user_id`
E. `date`

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 110**
A large company seeks to implement a near real-time solution involving hundreds of pipelines with parallel updates of many tables with extremely high volume and high velocity data.

Which of the following solutions would you implement to achieve this requirement?

A. Use Databricks High Concurrency clusters, which leverage optimized cloud storage connections to maximize data throughput.
B. Partition ingestion tables by a small time duration to allow for many data files to be written in parallel.
C. Configure Databricks to save all data to attached SSD volumes instead of object storage, increasing file I/O significantly.
D. Isolate Delta Lake tables in their own storage containers to avoid API limits imposed by cloud vendors.
E. Store all tables in a single database to ensure that the Databricks Catalyst Metastore can load balance overall throughput.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 111**
Which describes a method of installing a Python package scoped at the notebook level to all nodes in the currently active cluster?

A. Run source **`env/bin/activate`** in a notebook setup script
B. Use **`b`** in a notebook cell
C. Use **`%pip`** install in a notebook cell
D. Use **`%sh pip install`** in a notebook cell
E. Install libraries from PyPI using the cluster UI

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 112**

Each configuration below is identical to the extent that each cluster has 400 GB total of RAM, 160 total cores and only one Executor per VM.

Given an extremely long-running job for which completion must be guaranteed, which cluster configuration will be able to guarantee completion of the job in light of one or more VM failures?

A. • Total VMs: 8
   • 50 GB per Executor
   • 20 Cores / Executor
B. • Total VMs: 16
   • 25 GB per Executor
   • 10 Cores / Executor
C. • Total VMs: 1
   • 400 GB per Executor
   • 160 Cores/Executor
D. • Total VMs: 4
   • 100 GB per Executor
   • 40 Cores / Executor
E. • Total VMs: 2
   • 200 GB per Executor
   • 80 Cores / Executor

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 113**
A Delta Lake table in the Lakehouse named **`customer_churn_params`** is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.

Immediately after each update succeeds, the data engineering team would like to determine the difference between the new version and the previous version of the table.

Given the current implementation, which method can be used?

A. Execute a query to calculate the difference between the new version and the previous version using Delta Lake's built-in versioning and lime travel functionality.
B. Parse the Delta Lake transaction log to identify all newly written data files.
C. Parse the Spark event logs to identify those rows that were updated, inserted, or deleted.
D. Execute **`DESCRIBE HISTORY customer_churn_params`** to obtain the full operation metrics for the update, including a log of all records that have been added or modified.
E. Use Delta Lake's change data feed to identify those records that have been updated, inserted, or deleted.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 114**
A data team's Structured Streaming job is configured to calculate running aggregates for item sales to update a downstream marketing dashboard. The marketing team has introduced a new promotion, and they would like to add a new field to track the number of times this promotion code is used for each item. A junior data engineer suggests updating the existing query as follows. Note that proposed changes are in **bold**.

Original query:

```
df.groupBy("item")
    .agg(count("item").alias("total_count"),
         mean("sale_price").alias("avg_price"))
    .writeStream
    .outputMode("complete")
    .option("checkpointLocation", "/item_agg/__checkpoint")
    .start("/item_agg")
```

Proposed query:

```
df.groupBy("item")
    .agg(count("item").alias("total_count"),
         mean("sale_price").alias("avg_price"),
         count("promo_code = 'NEW_MEMBER'").alias("new_member_promo"))
    .writeStream
    .outputMode("complete")
    .option('mergeSchema', 'true')
    .option("checkpointLocation", "/item_agg/__checkpoint")
    .start("/item_agg")
```



Which step must also be completed to put the proposed query into production?

A. Specify a new **`CheckpointLocation`**
B. Remove **`.option('mergeSchema', 'true')`** from the streaming write
C. Increase the shuffle partitions to account for additional aggregates
D. Run **`REFRESH TABLE delta.'/item_agg'`**

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 115**
When using CLI or REST API to get results from jobs with multiple tasks, which statement correctly describes the response structure?

A. Each run of a job will have a unique **`job_id`**; all tasks within this job will have a unique **`job_id`**
B. Each run of a job will have a unique **`job_id`**; all tasks within this job will have a unique **`task_id`**
C. Each run of a job will have a unique **`orchestration_id`**; all tasks within this job will have a unique **`run_id`**
D. Each run of a job will have a unique **`run_id`**; all tasks within this job will have a unique **`task_id`**
E. Each run of a job will have a unique **`run_id`**; all tasks within this job will also have a unique **`run_id`**

**Correct Answer:** E
**Section:** (none)

**Explanation/Reference:**

**QUESTION 116**
The data engineering team is configuring environments for development, testing, and production before beginning migration on a new data pipeline. The team requires extensive testing on both the code and data resulting from code execution, and the team wants to develop and test against data as similar to production data as possible.

A junior data engineer suggests that production data can be mounted to the development and testing environments, allowing pre-production code to execute against production data. Because all users have admin privileges in the development environment, the junior data engineer has offered to configure permissions and mount this data for the team.

Which statement captures best practices for this situation?

A. All development, testing, and production code and data should exist in a single, unified workspace; creating separate environments for testing and development complicates administrative overhead.
B. In environments where interactive code will be executed, production data should only be accessible with read permissions; creating isolated databases for each environment further reduces risks.
C. As long as code in the development environment declares **USE dev_db** at the top of each notebook, there is no possibility of inadvertently committing changes back to production data sources.
D. Because Delta Lake versions all data and supports time travel, it is not possible for user error or malicious actors to permanently delete production data; as such, it is generally safe to mount production data anywhere.
E. Because access to production data will always be verified using passthrough credentials, it is safe to mount data to any Databricks development environment.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**



**QUESTION 117**
A data engineer, User A, has promoted a pipeline to production by using the REST API to programmatically create several jobs. A DevOps engineer, User B, has configured an external orchestration tool to trigger job runs through the REST API. Both users authorized the REST API calls using their personal access tokens.

A workspace admin, User C, inherits responsibility for managing this pipeline. User C uses the Databricks Jobs UI to take "Owner" privileges of each job. Jobs continue to be triggered using the credentials and tooling configured by User B.

An application has been configured to collect and parse run information returned by the REST API. Which statement describes the value returned in the **creator_user_name** field?

A. Once User C takes "Owner" privileges, their email address will appear in this field; prior to this, User A's email address will appear in this field.
B. User B's email address will always appear in this field, as their credentials are always used to trigger the run.
C. User A's email address will always appear in this field, as they still own the underlying notebooks.
D. Once User C takes "Owner" privileges, their email address will appear in this field; prior to this, User B's email address will appear in this field.
E. User C will only ever appear in this field if they manually trigger the job, otherwise it will indicate User B.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 118**

A member of the data engineering team has submitted a short notebook that they wish to schedule as part of a larger data pipeline. Assume that the commands provided below produce the logically correct results when run as presented.

```
Cmd 1

rawDF = spark.table("raw_data")

Cmd 2

rawDF.printSchema()

Cmd 3

flattenedDF = rawDF.select("*", "values.*")

Cmd 4

finalDF = flattenedDF.drop("values")

Cmd 5

display(finalDF)

Cmd 6

finalDF.write.mode("append").saveAsTable("flat_data")
```



Which command should be removed from the notebook before scheduling it as a job?

A. Cmd 2
B. Cmd 3
C. Cmd 4
D. Cmd 5

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 119**
Which statement regarding Spark configuration on the Databricks platform is true?

A. The Databricks REST API can be used to modify the Spark configuration properties for an interactive cluster without interrupting jobs currently running on the cluster.
B. Spark configurations set within a notebook will affect all SparkSessions attached to the same interactive cluster.
C. When the same Spark configuration property is set for an interactive cluster and a notebook attached to that cluster, the notebook setting will always be ignored.

D. Spark configuration properties set for an interactive cluster with the Clusters UI will impact all notebooks attached to that cluster.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 120**
The business reporting team requires that data for their dashboards be updated every hour. The total processing time for the pipeline that extracts, transforms, and loads the data for their pipeline runs in 10 minutes.

Assuming normal operating conditions, which configuration will meet their service-level agreement requirements with the lowest cost?

A. Configure a job that executes every time new data lands in a given directory
B. Schedule a job to execute the pipeline once an hour on a new job cluster
C. Schedule a Structured Streaming job with a trigger interval of 60 minutes
D. Schedule a job to execute the pipeline once an hour on a dedicated interactive cluster

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 121**
A Databricks SQL dashboard has been configured to monitor the total number of records present in a collection of Delta Lake tables using the following query pattern:

```
SELECT COUNT (*) FROM table
```

Which of the following describes how results are generated each time the dashboard is updated?

A. The total count of rows is calculated by scanning all data files
B. The total count of rows will be returned from cached results unless REFRESH is run
C. The total count of records is calculated from the Delta transaction logs
D. The total count of records is calculated from the parquet file metadata

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 122**
A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_store
AS (
  SELECT *
  FROM prod.sales a
  INNER JOIN prod.store b
  ON a.store_id = b.store_id
)
```

Consider the following query:

```
DROP TABLE prod.sales_by_store
```

If this statement is executed by a workspace admin, which result will occur?

A. Data will be marked as deleted but still recoverable with Time Travel.
B. The table will be removed from the catalog but the data will remain in storage.
C. The table will be removed from the catalog and the data will be deleted.
D. An error will occur because Delta Lake prevents the deletion of production data.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 123**
A developer has successfully configured their credentials for Databricks Repos and cloned a remote Git repository. They do not have privileges to make changes to the **main** branch, which is the only branch currently visible in their workspace.

Which approach allows this user to share their code updates without the risk of overwriting the work of their teammates?

A. Use Repos to create a new branch, commit all changes, and push changes to the remote Git repository.
B. Use Repos to create a fork of the remote repository, commit all changes, and make a pull request on the source repository.
C. Use Repos to pull changes from the remote Git repository; commit and push changes to a branch that appeared as changes were pulled.
D. Use Repos to merge all differences and make a pull request back to the remote repository.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 124**
The security team is exploring whether or not the Databricks secrets module can be leveraged for connecting to an external database.

After testing the code with all Python variables being defined with strings, they upload the password to the secrets module and configure the correct permissions for the currently active user. They then modify their code to the following (leaving all other variables unchanged).

```
password = dbutils.secrets.get(scope="db_creds", key="jdbc_password")

print(password)

df = (spark
  .read
  .format("jdbc")
  .option("url", connection)
  .option("dbtable", tablename)
  .option("user", username)
  .option("password", password)
  )
```

Which statement describes what will happen when the above code is executed?

A. The connection to the external table will succeed; the string "**REDACTED**" will be printed.
B. An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the encoded password will be saved to DBFS.
C. An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the password will be printed in plain text.
D. The connection to the external table will succeed; the string value of **password** will be printed in plain text.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 125**
The data science team has created and logged a production model using MLflow. The model accepts a list of column names and returns a new column of type **`DOUBLE`**.

The following code correctly imports the production model, loads the **`customers`** table containing the **`customer_id`** key column into a DataFrame, and defines the feature columns needed for the model.

```
model = mlflow.pyfunc.spark_udf (spark,
model_uri="models:/churn/prod")

df = spark.table("customers")

columns = ["account_age", "time_since_last_seen", "app_rating"]
```

Which code block will output a DataFrame with the schema "**`customer_id LONG, predictions DOUBLE`**"?

A. `df.map(lambda x:model(x[columns])).select("customer_id, predictions")`
B. `df.select("customer_id", model(*columns).alias("predictions"))`
C. `model.predict(df, columns)`
D. `df.apply(model, columns).select("customer_id, predictions")`

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 126**
A junior member of the data engineering team is exploring the language interoperability of Databricks notebooks. The intended outcome of the below code is to register a view of all sales that occurred in countries on the continent of Africa that appear in the **geo_lookup** table.

Before executing the code, running **SHOW TABLES** on the current database indicates the database contains only two tables: **geo_lookup** and **sales**.

Cmd 1

```
%python
countries_af = [x[0] for x in
spark.table("geo_lookup").filter("continent='AF'").select("country").collect()]
```

Cmd 2

```
%sql
CREATE VIEW sales_af AS
  SELECT *
  FROM sales
  WHERE city IN countries_af
  AND CONTINENT = "AF"
```

What will be the outcome of executing these command cells in order in an interactive notebook?

A. Both commands will succeed. Executing **SHOW TABLES** will show that **countries_af** and **sales_af** have been registered as views.
B. Cmd 1 will succeed. Cmd 2 will search all accessible databases for a table or view named **countries_af**: if this entity exists, Cmd 2 will succeed.
C. Cmd 1 will succeed and Cmd 2 will fail. **countries_af** will be a Python variable representing a PySpark DataFrame.
D. Cmd 1 will succeed and Cmd 2 will fail. **countries_af** will be a Python variable containing a list of strings.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 127**
The data science team has requested assistance in accelerating queries on free-form text from user reviews. The data is currently stored in Parquet with the below schema:

```
item_id INT, user_id INT, review_id INT, rating FLOAT, review STRING
```

The **review** column contains the full text of the review left by the user. Specifically, the data science team is looking to identify if any of 30 key words exist in this field.

A junior data engineer suggests converting this data to Delta Lake will improve query performance.

Which response to the junior data engineer's suggestion is correct?

A. Delta Lake statistics are not optimized for free text fields with high cardinality.
B. Delta Lake statistics are only collected on the first 4 columns in a table.
C. ZORDER ON review will need to be run to see performance gains.
D. The Delta log creates a term matrix for free text fields to support selective filtering.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 128**
The data engineering team has configured a job to process customer requests to be forgotten (have their data deleted). All user data that needs to be deleted is stored in Delta Lake tables using default table settings.

The team has decided to process all deletions from the previous week as a batch job at 1am each Sunday. The total duration of this job is less than one hour. Every Monday at 3am, a batch job executes a series of **VACUUM** commands on all Delta Lake tables throughout the organization.

The compliance officer has recently learned about Delta Lake's time travel functionality. They are concerned that this might allow continued access to deleted data.

Assuming all delete logic is correctly implemented, which statement correctly addresses this concern?

A. Because the **VACUUM** command permanently deletes all files containing deleted records, deleted records may be accessible with time travel for around 24 hours.
B. Because the default data retention threshold is 24 hours, data files containing deleted records will be retained until the **VACUUM** job is run the following day.
C. Because the default data retention threshold is 7 days, data files containing deleted records will be retained until the **VACUUM** job is run 8 days later.
D. Because Delta Lake's delete statements have ACID guarantees, deleted records will be permanently purged from all storage systems as soon as a delete job completes.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 129**
Assuming that the Databricks CLI has been installed and configured correctly, which Databricks CLI command can be used to upload a custom Python Wheel to object storage mounted with the DBFS for use with a production job?

A. `configure`
B. `fs`
C. `workspace`
D. `libraries`

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 130**
The following table consists of items found in user carts within an e-commerce website.

```
Carts (id LONG, items ARRAY<STRUCT<id: LONG, count: INT>>)
id  items                                                email
1001[{id: "DESK65", count: 1}]                           "u1@domain.com"
1002[{id: "KYBD45", count: 1}, {id: "M27", count: 2}]"u2@domain.com"
1003[{id: "M27", count: 1}]                              "u3@domain.com"
```

The following MERGE statement is used to update this table using an updates view, with schema evolution enabled on this table.

```
MERGE INTO carts c
USING updates u
ON c.id = u.id
WHEN MATCHED
  THEN UPDATE SET *
```

How would the following update be handled?

```
(new nested field, missing existing column)
id  items
1001[{id: "DESK65", count: 2, coupon: "BOGO50"}]
```

A. The update throws an error because changes to existing columns in the target schema are not supported.
B. The new nested Field is added to the target schema, and dynamically read as NULL for existing unmatched records.
C. The update is moved to a separate "rescued" column because it is missing a column expected in the target schema.
D. The new nested field is added to the target schema, and files underlying existing records are updated to include NULL values for the new field.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 131**
An upstream system is emitting change data capture (CDC) logs that are being written to a cloud object storage directory. Each record in the log indicates the change type (insert, update, or delete) and the values for each field after the change. The source table has a primary key identified by the field **pk_id**.

For analytical purposes, only the most recent value for each record needs to be recorded in the target Delta Lake table in the Lakehouse. The Databricks job to ingest these records occurs once per hour, but each individual record may have changed multiple times over the course of an hour.

Which solution meets these requirements?

A. Iterate through an ordered set of changes to the table, applying each in turn to create the current state of the table, (insert, update, delete), timestamp of change, and the values.
B. Use `MERGE INTO` to insert, update, or delete the most recent entry for each `pk_id` into a table, then

propagate all changes throughout the system.

C. Deduplicate records in each batch by `pk_id` and overwrite the target table.

D. Use Delta Lake's change data feed to automatically process CDC data from an external system, propagating all changes to all dependent tables in the Lakehouse.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 132**
An hourly batch job is configured to ingest data files from a cloud object storage container where each batch represent all records produced by the source system in a given hour. The batch job to process these records into the Lakehouse is sufficiently delayed to ensure no late-arriving data is missed. The **`user_id`** field represents a unique key for the data, which has the following schema:

```
user_id BIGINT, username STRING, user_utc STRING, user_region STRING,
last_login BIGINT, auto_pay BOOLEAN, last_updated BIGINT
```

New records are all ingested into a table named **`account_history`** which maintains a full record of all data in the same schema as the source. The next table in the system is named **`account_current`** and is implemented as a Type 1 table representing the most recent value for each unique **`user_id`**.

Which implementation can be used to efficiently update the described **`account_current`** table as part of each hourly batch job assuming there are millions of user accounts and tens of thousands of records processed hourly?

A. Filter records in **`account_history`** using the **`last_updated`** field and the most recent hour processed, making sure to deduplicate on username; write a merge statement to update or insert the most recent value for each **`username`**.

B. Use Auto Loader to subscribe to new files in the **`account_history`** directory; configure a Structured Streaming trigger available job to batch update newly detected files into the **`account_current`** table.

C. Overwrite the **`account_current`** table with each batch using the results of a query against the **`account_history`** table grouping by **`user_id`** and filtering for the max value of **`last_updated`**.

D. Filter records in **`account_history`** using the **`last_updated`** field and the most recent hour processed, as well as the max **`last_login`** by **`user_id`** write a merge statement to update or insert the most recent value for each **`user_id`**.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 133**
The business intelligence team has a dashboard configured to track various summary metrics for retail stores. This includes total sales for the previous day alongside totals and averages for a variety of time periods. The fields required to populate this dashboard have the following schema:

```
store_id INT, total_sales_qtd FLOAT, avg_daily_sales_qtd FLOAT,  total_sales_ytd
FLOAT, avg_daily_sales_ytd FLOAT, previous_day_sales FLOAT, total_sales_7d FLOAT,
avg_daily_sales_7d FLOAT, updated TIMESTAMP
```

For demand forecasting, the Lakehouse contains a validated table of all itemized sales updated incrementally in near real-time. This table, named **products_per_order**, includes the following fields:

```
store_id INT, order_id INT, product_id INT, quantity INT, price FLOAT,
order_timestamp TIMESTAMP
```

Because reporting on long-term sales trends is less volatile, analysts using the new dashboard only require data to be refreshed once daily. Because the dashboard will be queried interactively by many users throughout a normal business day, it should return results quickly and reduce total compute associated with each materialization.

Which solution meets the expectations of the end users while controlling and limiting possible costs?

A. Populate the dashboard by configuring a nightly batch job to save the required values as a table overwritten with each update.
B. Use Structured Streaming to configure a live dashboard against the **`products_per_order`** table within a Databricks notebook.
C. Define a view against the **`products_per_order`** table and define the dashboard against this view.
D. Use the Delta Cache to persist the **`products_per_order`** table in memory to quickly update the dashboard with each query.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 134**
A Delta lake table with CDF enabled table in the Lakehouse named **`customer_churn_params`** is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.

The churn prediction model used by the ML team is fairly stable in production. The team is only interested in making predictions on records that have changed in the past 24 hours.

Which approach would simplify the identification of these changed records?

A. Apply the churn model to all rows in the **`customer_churn_params`** table, but implement logic to perform an upsert into the predictions table that ignores rows where predictions have not changed.
B. nvert the batch job to a Structured Streaming job using the **`complete`** output mode; configure a Structured Streaming job to read from the **`customer_churn_params`** table and incrementally predict against the churn model.
C. Replace the current overwrite logic with a merge statement to modify only those records that have changed; write logic to make predictions on the changed records identified by the change data feed.
D. Modify the overwrite logic to include a field populated by calling **`spark.sql.functions.current_timestamp()`** as data are being written; use this field to identify records written on a particular date.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 135**
A view is registered with the following code:

```
CREATE VIEW recent_orders AS (
  SELECT a.user_id, a.email, b.order_id, b.order_date
  FROM
    (SELECT user_id, email
    FROM users) a
    INNER JOIN
    (SELECT user_id, order_id, order_date
    FROM orders
    WHERE order_date >= (current_date() - 7)) b
    ON a.user_id = b.user_id
)
```

Both users and orders are Delta Lake tables.

Which statement describes the results of querying **recent_orders**?

A. All logic will execute when the view is defined and store the result of joining tables to the DBFS; this stored data will be returned when the view is queried.
B. Results will be computed and cached when the view is defined; these cached results will incrementally update as new records are inserted into source tables.
C. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query finishes.
D. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.



**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 136**
A data ingestion task requires a one-TB JSON dataset to be written out to Parquet with a target part-file size of 512 MB. Because Parquet is being used instead of Delta Lake, built-in file-sizing features such as Auto-Optimize & Auto-Compaction cannot be used.

Which strategy will yield the best performance without shuffling data?

A. Set `spark.sql.files.maxPartitionBytes` to 512 MB, ingest the data, execute the narrow transformations, and then write to parquet.
B. Set `spark.sql.shuffle.partitions` to 2,048 partitions (1TB*1024*1024/512), ingest the data, execute the narrow transformations, optimize the data by sorting it (which automatically repartitions the data), and then write to parquet.
C. Set `spark.sql.adaptive.advisoryPartitionSizeInBytes` to 512 MB bytes, ingest the data, execute the narrow transformations, coalesce to 2,048 partitions (1TB*1024*1024/512), and then write to parquet.
D. Ingest the data, execute the narrow transformations, repartition to 2,048 partitions (1TB* 1024*1024/512), and then write to parquet.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 137**
Which statement regarding stream-static joins and static Delta tables is correct?

A. The checkpoint directory will be used to track updates to the static Delta table.
B. Each microbatch of a stream-static join will use the most recent version of the static Delta table as of the job's initialization.
C. The checkpoint directory will be used to track state information for the unique keys present in the join.
D. Stream-static joins cannot use static Delta tables because of consistency issues.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 138**
A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame **df**. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Events are recorded once per minute per device.

Streaming DataFrame **df** has the following schema:

```
"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"
```

Code block:

```
df.withWatermark("event_time", "10 minutes")
    .groupBy(
        _____,
        "device_id"
    )
    .agg(
        avg("temp").alias("avg_temp"),
        avg("humidity").alias("avg_humidity")
    )
    .writeStream
    .format("delta")
    .saveAsTable("sensor_avg")
```

Which line of code correctly fills in the blank within the code block to complete this task?

A. `to_interval("event_time", "5 minutes").alias("time")`
B. `window("event_time", "5 minutes").alias("time")`
C. `"event_time"`
D. `lag("event_time", "10 minutes").alias("time")`

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 139**
A Structured Streaming job deployed to production has been resulting in higher than expected cloud storage costs. At present, during normal execution, each microbatch of data is processed in less than 3s; at least 12 times per minute, a microbatch is processed that contains 0 records. The streaming write was

configured using the default trigger settings. The production job is currently scheduled alongside many other Databricks jobs in a workspace with instance pools provisioned to reduce start-up time for jobs with batch execution.

Holding all other variables constant and assuming records need to be processed in less than 10 minutes, which adjustment will meet the requirement?

A. Set the trigger interval to 3 seconds; the default trigger interval is consuming too many records per batch, resulting in spill to disk that can increase volume costs.
B. Use the trigger once option and configure a Databricks job to execute the query every 10 minutes; this approach minimizes costs for both compute and storage.
C. Set the trigger interval to 10 minutes; each batch calls APIs in the source storage account, so decreasing trigger frequency to maximum allowable threshold should minimize this cost.
D. Set the trigger interval to 500 milliseconds; setting a small but non-zero trigger interval ensures that the source is not queried too frequently.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 140**
Which statement describes Delta Lake optimized writes?

A. Before a Jobs cluster terminates, **OPTIMIZE** is executed on all tables modified during the most recent job.
B. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an **OPTIMIZE** job is executed toward a default of 1 GB.
C. A shuffle occurs prior to writing to try to group similar data together resulting in fewer files instead of each executor writing multiple files based on directory partitions.
D. Optimized writes use logical partitions instead of directory partitions; because partition boundaries are only represented in metadata, fewer small files are written.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 141**
Which statement characterizes the general programming model used by Spark Structured Streaming?

A. Structured Streaming leverages the parallel processing of GPUs to achieve highly parallel data throughput.
B. Structured Streaming is implemented as a messaging bus and is derived from Apache Kafka.
C. Structured Streaming relies on a distributed network of nodes that hold incremental state values for cached stages.
D. Structured Streaming models new data arriving in a data stream as new rows appended to an unbounded table.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 142**
Which configuration parameter directly affects the size of a spark-partition upon ingestion of data into Spark?

A. `spark.sql.files.maxPartitionBytes`
B. `spark.sql.autoBroadcastJoinThreshold`
C. `spark.sql.adaptive.advisoryPartitionSizeInBytes`
D. `spark.sql.adaptive.coalescePartitions.minPartitionNum`

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 143**
A Spark job is taking longer than expected. Using the Spark UI, a data engineer notes that the Min, Median, and Max Durations for tasks in a particular stage show the minimum and median time to complete a task as roughly the same, but the max duration for a task to be roughly 100 times as long as the minimum.

Which situation is causing increased duration of the overall job?

A. Task queueing resulting from improper thread pool assignment.
B. Spill resulting from attached volume storage being too small.
C. Network latency due to some cluster nodes being in different regions from the source data
D. Skew caused by more data being assigned to a subset of spark-partitions.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 144**
Each configuration below is identical to the extent that each cluster has 400 GB total of RAM, 160 total cores and only one Executor per VM.

Given an extremely long-running job for which completion must be guaranteed, which cluster configuration will be able to guarantee completion of the job in light of one or more VM failures?

A. • Total VMs: 8
   • 50 GB per Executor
   • 20 Cores / Executor
B. • Total VMs: 16
   • 25 GB per Executor
   • 10 Cores / Executor
C. • Total VMs: 1
   • 400 GB per Executor
   • 160 Cores/Executor
D. • Total VMs: 4
   • 100 GB per Executor
   • 40 Cores / Executor

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 145**
A task orchestrator has been configured to run two hourly tasks. First, an outside system writes Parquet data to a directory mounted at **/mnt/raw_orders/**. After this data is written, a Databricks job containing the

following code is executed:

```
(spark.readStream
  .format("parquet")
  .load("/mnt/raw_orders/")
  .withWatermark("time", "2 hours")
  .dropDuplicates(["customer_id", "order_id"])
  .writeStream
  .trigger(once=True)
  .table("orders")
)
```

Assume that the fields **customer_id** and **order_id** serve as a composite key to uniquely identify each order, and that the **time** field indicates when the record was queued in the source system.

If the upstream system is known to occasionally enqueue duplicate entries for a single order hours apart, which statement is correct?

A. Duplicate records enqueued more than 2 hours apart may be retained and the **orders** table may contain duplicate records with the same **customer_id** and **order_id**.
B. All records will be held in the state store for 2 hours before being deduplicated and committed to the **orders** table.
C. The **orders** table will contain only the most recent 2 hours of records and no duplicates will be present.
D. The **orders** table will not contain duplicates, but records arriving more than 2 hours late will be ignored and missing from the table.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 146**
A data engineer is configuring a pipeline that will potentially see late-arriving, duplicate records.

In addition to de-duplicating records within the batch, which of the following approaches allows the data engineer to deduplicate data against previously processed records as it is inserted into a Delta table?

A. Rely on Delta Lake schema enforcement to prevent duplicate records.
B. **VACUUM** the Delta table after each batch completes.
C. Perform an insert-only merge with a matching condition on a unique key.
D. Perform a full outer join on a unique key and overwrite existing data.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 147**
A junior data engineer seeks to leverage Delta Lake's Change Data Feed functionality to create a Type 1

table representing all of the values that have ever been valid for all rows in a **`bronze`** table created with the property **`delta.enableChangeDataFeed = true`**. They plan to execute the following code as a daily job:

```
from pyspark.sql.functions import col

(spark.read.format("delta")
  .option("readChangeFeed", "true")
  .option("startingVersion", 0)
  .table("bronze")
  .filter(col("_change_type").isin(["update_postimage", "insert"]))
  .write
  .mode("append")
  .table("bronze_history_type1")
 )
```

Which statement describes the execution and results of running the above query multiple times?

A. Each time the job is executed, newly updated records will be merged into the target table, overwriting previous values with the same primary keys.
B. Each time the job is executed, the entire available history of inserted or updated records will be appended to the target table, resulting in many duplicate entries.
C. Each time the job is executed, only those records that have been inserted or updated since the last execution will be appended to the target table, giving the desired result.
D. Each time the job is executed, the differences between the original and current versions are calculated; this may result in duplicate entries for some records.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 148**
A **DLT** pipeline includes the following streaming tables:

- **raw_iot** ingests raw device measurement data from a heart rate tracking device.
- **bpm_stats** incrementally computes user statistics based on BPM measurements from **raw_iot**.

How can the data engineer configure this pipeline to be able to retain manually deleted or updated records in the **raw_iot** table, while recomputing the downstream table **bpm_stats** table when a pipeline update is run?

A. Set the **`pipelines.reset.allowed`** property to false on **`raw_iot`**
B. Set the **`skipChangeCommits`** flag to true on **`raw_iot`**
C. Set the **`pipelines.reset.allowed`** property to false on **`bpm_stats`**
D. Set the **`skipChangeCommits`** flag to true on **`bpm_stats`**

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 149**
A data pipeline uses Structured Streaming to ingest data from Apache Kafka to Delta Lake. Data is being

stored in a bronze table, and includes the Kafka-generated timestamp, key, and value. Three months after the pipeline is deployed, the data engineering team has noticed some latency issues during certain times of the day.

A senior data engineer updates the Delta Table's schema and ingestion logic to include the current timestamp (as recorded by Apache Spark) as well as the Kafka topic and partition. The team plans to use these additional metadata fields to diagnose the transient processing delays.

Which limitation will the team face while diagnosing this problem?

A. New fields will not be computed for historic records.
B. Spark cannot capture the topic and partition fields from a Kafka source.
C. Updating the table schema requires a default value provided for each field added.
D. Updating the table schema will invalidate the Delta transaction log metadata.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 150**
A nightly job ingests data into a Delta Lake table using the following code:

```
from pyspark.sql.functions import current_timestamp, input_file_name, col
from pyspark.sql.column import Column

def ingest_daily_batch(time_col: Column, year:int, month:int, day:int):
    (spark.read
        .format("parquet")
        .load(f"/mnt/daily_batch/{year}/{month}/{day}")
        .select("*",
                time_col.alias("ingest_time"),
                inpute_file_name().alias("source_file")
               )
        .write
        .mode("append")
        .saveAsTable("bronze")
    )
```

The next step in the pipeline requires a function that returns an object that can be used to manipulate new records that have not yet been processed to the next table in the pipeline.

Which code snippet completes this function definition?

```
def new_records():
```

A. `return spark.readStream.table("bronze")`
B. `return spark.read.option("readChangeFeed", "true").table ("bronze")`
C.
```
return (spark.read
    .table("bronze")
    .filter(col("ingest_time") == current_timestamp())
)
```

D.
```
return (spark.read
      .table("bronze")
      .filter(col("source_file") == f"/mnt/daily_batch/{year}/{mo
  )
```

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 151**
A junior data engineer is working to implement logic for a Lakehouse table named **silver_device_recordings**. The source data contains 100 unique fields in a highly nested JSON structure.

The **silver_device_recordings** table will be used downstream to power several production monitoring dashboards and a production model. At present, 45 of the 100 fields are being used in at least one of these applications.

The data engineer is trying to determine the best approach for dealing with schema declaration given the highly-nested structure of the data and the numerous fields.

Which of the following accurately presents information about Delta Lake and Databricks that may impact their decision-making process?

A. The Tungsten encoding used by Databricks is optimized for storing string data; newly-added native support for querying JSON strings means that string types are always most efficient.
B. Because Delta Lake uses Parquet for data storage, data types can be easily evolved by just modifying file footer information in place.
C. Schema inference and evolution on Databricks ensure that inferred types will always accurately match the data types used by downstream systems.
D. Because Databricks will infer schema using types that allow all observed data to be processed, setting types manually provides greater assurance of data quality enforcement.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 152**
The data engineering team maintains the following code:

```
accountDF = spark.table("accounts")
orderDF = spark.table("orders")
itemDF = spark.table("items")

orderWithItemDF = (orderDF.join(
    itemDF,
    orderDF.itemID == itemDF.itemID)
  .select(
    orderDF.accountID,
    orderDF.itemID,

    itemDF.itemName))

finalDF = (accountDF.join(
    orderWithItemDF,
    accountDF.accountID == orderWithItemDF.accountID)
  .select(
    orderWithItemDF["*"],

    accountDF.city))

(finalDF.write
  .mode("overwrite")
  .table("enriched_itemized_orders_by_account"))
```



Assuming that this code produces logically correct results and the data in the source tables has been de-duplicated and validated, which statement describes what will occur when this code is executed?

A. A batch job will update the **enriched_itemized_orders_by_account** table, replacing only those rows that have different values than the current version of the table, using **accountID** as the primary key.
B. The **enriched_itemized_orders_by_account** table will be overwritten using the current valid version of data in each of the three tables referenced in the join logic.
C. No computation will occur until **enriched_itemized_orders_by_account** is queried; upon query materialization, results will be calculated using the current valid version of data in each of the three tables referenced in the join logic.
D. An incremental job will detect if new rows have been written to any of the source tables; if new rows are detected, all results will be recalculated and used to overwrite the **enriched_itemized_orders_by_account** table.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 153**
The data engineering team is configuring environments for development, testing, and production before

beginning migration on a new data pipeline. The team requires extensive testing on both the code and data resulting from code execution, and the team wants to develop and test against data as similar to production data as possible.

A junior data engineer suggests that production data can be mounted to the development and testing environments, allowing pre-production code to execute against production data. Because all users have admin privileges in the development environment, the junior data engineer has offered to configure permissions and mount this data for the team.

Which statement captures best practices for this situation?

A. All development, testing, and production code and data should exist in a single, unified workspace; creating separate environments for testing and development complicates administrative overhead.
B. In environments where interactive code will be executed, production data should only be accessible with read permissions; creating isolated databases for each environment further reduces risks.
C. Because access to production data will always be verified using passthrough credentials, it is safe to mount data to any Databricks development environment.
D. Because Delta Lake versions all data and supports time travel, it is not possible for user error or malicious actors to permanently delete production data; as such, it is generally safe to mount production data anywhere.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 154**
The data architect has mandated that all tables in the Lakehouse should be configured as external Delta Lake tables.

Which approach will ensure that this requirement is met?

A. Whenever a database is being created, make sure that the **LOCATION** keyword is used.
B. When the workspace is being configured, make sure that external cloud object storage has been mounted.
C. Whenever a table is being created, make sure that the **LOCATION** keyword is used.
D. When tables are created, make sure that the **UNMANAGED** keyword is used in the **CREATE TABLE** statement.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 155**
The marketing team is looking to share data in an aggregate table with the sales organization, but the field names used by the teams do not match, and a number of marketing-specific fields have not been approved for the sales org.

Which of the following solutions addresses the situation while emphasizing simplicity?

A. Create a view on the marketing table selecting only those fields approved for the sales team; alias the names of any fields that should be standardized to the sales naming conventions.
B. Create a new table with the required schema and use Delta Lake's **DEEP CLONE** functionality to sync up changes committed to one table to the corresponding table.
C. Use a CTAS statement to create a derivative table from the marketing table; configure a production job to propagate changes.
D. Add a parallel table write to the current production pipeline, updating a new sales table that varies as

required from the marketing table.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 156**
A Delta Lake table representing metadata about content posts from users has the following schema:

```
user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude
FLOAT, post_time TIMESTAMP, date DATE
```

This table is partitioned by the date column. A query is run with the following filter:

```
longitude < 20 & longitude > -20
```

Which statement describes how data will be filtered?

A. Statistics in the Delta Log will be used to identify partitions that might Include files in the filtered range.
B. No file skipping will occur because the optimizer does not know the relationship between the partition column and the longitude.
C. The Delta Engine will scan the parquet file footers to identify each row that meets the filter criteria.
D. Statistics in the Delta Log will be used to identify data files that might include records in the filtered range.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**



**QUESTION 157**
A small company based in the United States has recently contracted a consulting firm in India to implement several new data engineering pipelines to power artificial intelligence applications. All the company's data is stored in regional cloud storage in the United States.

The workspace administrator at the company is uncertain about where the Databricks workspace used by the contractors should be deployed.

Assuming that all data governance considerations are accounted for, which statement accurately informs this decision?

A. Databricks runs HDFS on cloud volume storage; as such, cloud virtual machines must be deployed in the region where the data is stored.
B. Databricks workspaces do not rely on any regional infrastructure; as such, the decision should be made based upon what is most convenient for the workspace administrator.
C. Cross-region reads and writes can incur significant costs and latency; whenever possible, compute should be deployed in the same region the data is stored.
D. Databricks notebooks send all executable code from the user's browser to virtual machines over the open internet; whenever possible, choosing a workspace region near the end users is the most secure.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 158**
A **`CHECK`** constraint has been successfully added to the Delta table named **`activity_details`** using the

following logic:

```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
  latitude >= -90 AND
  latitude <= 90 AND
  longitude >= -180 AND
  longitude <= 180);
```

A batch job is attempting to insert new records to the table, including a record where **`latitude = 45.50`** and **`longitude = 212.67`**.

Which statement describes the outcome of this batch insert?

A. The write will insert all records except those that violate the table constraints; the violating records will be reported in a warning log.
B. The write will fail completely because of the constraint violation and no records will be inserted into the target table.
C. The write will insert all records except those that violate the table constraints; the violating records will be recorded to a quarantine table.
D. The write will include all records in the target table; any violations will be indicated in the boolean column named **`valid_coordinates`**.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 159**
A junior data engineer is migrating a workload from a relational database system to the Databricks Lakehouse. The source system uses a star schema, leveraging foreign key constraints and multi-table inserts to validate records on write.

Which consideration will impact the decisions made by the engineer while migrating this workload?

A. Databricks only allows foreign key constraints on hashed identifiers, which avoid collisions in highly-parallel writes.
B. Foreign keys must reference a primary key field; multi-table inserts must leverage Delta Lake's upsert functionality.
C. Committing to multiple tables simultaneously requires taking out multiple table locks and can lead to a state of deadlock.
D. All Delta Lake transactions are ACID compliant against a single table, and Databricks does not enforce foreign key constraints.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 160**
A data architect has heard about Delta Lake's built-in versioning and time travel capabilities. For auditing purposes, they have a requirement to maintain a full record of all valid street addresses as they appear in

the **customers** table.

The architect is interested in implementing a Type 1 table, overwriting existing records with new values and relying on Delta Lake time travel to support long-term auditing. A data engineer on the project feels that a Type 2 table will provide better performance and scalability.

Which piece of information is critical to this decision?

A. Data corruption can occur if a query fails in a partially completed state because Type 2 tables require setting multiple fields in a single update.
B. Shallow clones can be combined with Type 1 tables to accelerate historic queries for long-term versioning.
C. Delta Lake time travel cannot be used to query previous versions of these tables because Type 1 changes modify data files in place.
D. Delta Lake time travel does not scale well in cost or latency to provide a long-term versioning solution.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 161**
A data engineer wants to join a stream of advertisement impressions (when an ad was shown) with another stream of user clicks on advertisements to correlate when impressions led to monetizable clicks.

In the code below, Impressions is a streaming DataFrame with a watermark ("event_time", "10 minutes")

```
.groupBy(
window("event_time", "5 minutes"),
"id")
.count()
).     withWatermark("event_time", 2 hours)
impressions.join(clicks, expr("clickAdId = impressionAdId"), "inner")
```

The data engineer notices the query slowing down significantly.

Which solution would improve the performance?

A. Joining on event time constraint: clickTime >= impressionTime AND clickTime <= impressionTime interval 1 hour
B. Joining on event time constraint: clickTime + 3 hours < impressionTime - 2 hours
C. Joining on event time constraint: clickTime == impressionTime using a leftOuter join
D. Joining on event time constraint: clickTime >= impressionTime - interval 3 hours and removing watermarks

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 162**
A junior data engineer has manually configured a series of jobs using the Databricks Jobs UI. Upon reviewing their work, the engineer realizes that they are listed as the "Owner" for each job. They attempt to transfer "Owner" privileges to the "DevOps" group, but cannot successfully accomplish this task.

Which statement explains what is preventing this privilege transfer?

A. Databricks jobs must have exactly one owner; "Owner" privileges cannot be assigned to a group.
B. The creator of a Databricks job will always have "Owner" privileges; this configuration cannot be changed.
C. Only workspace administrators can grant "Owner" privileges to a group.
D. A user can only transfer job ownership to a group if they are also a member of that group.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 163**
A table named **`user_ltv`** is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.

The **`user_ltv`** table has the following schema:

```
email STRING, age INT, ltv INT
```

The following view definition is executed:

```
CREATE VIEW user_ltv_no_minors AS
SELECT email, age, ltv
FROM user_ltv
WHERE
  CASE
    WHEN is_member("auditing") THEN TRUE
    ELSE age >= 18
  END
```



An analyst who is not a member of the auditing group executes the following query:

```
SELECT * FROM user_ltv_no_minors
```

Which statement describes the results returned by this query?

A. All columns will be displayed normally for those records that have an **age** greater than 17; records not meeting this condition will be omitted.
B. All age values less than 18 will be returned as null values, all other columns will be returned with the values in **user_ltv**.
C. All values for the age column will be returned as null values, all other columns will be returned with the values in **user_ltv**.
D. All columns will be displayed normally for those records that have an **age** greater than 18; records not meeting this condition will be omitted.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 164**
All records from an Apache Kafka producer are being ingested into a single Delta Lake table with the following schema:

```
key BINARY, value BINARY, topic STRING, partition LONG, offset LONG, timestamp LONG
```

There are 5 unique topics being ingested. Only the "**registration**" topic contains Personal Identifiable Information (PII). The company wishes to restrict access to PII. The company also wishes to only retain records containing PII in this table for 14 days after initial ingestion. However, for non-PII information, it would like to retain these records indefinitely.

Which solution meets the requirements?

A. All data should be deleted biweekly; Delta Lake's time travel functionality should be leveraged to maintain a history of non-PII information.
B. Data should be partitioned by the **registration** field, allowing ACLs and delete statements to be set for the PII directory.
C. Data should be partitioned by the **topic** field, allowing ACLs and delete statements to leverage partition boundaries.
D. Separate object storage containers should be specified based on the **partition** field, allowing isolation at the storage level.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 165**
The data governance team is reviewing code used for deleting records for compliance with GDPR. The following logic has been implemented to propagate delete requests from the **user_lookup** table to the **user_aggregates** table.

```
(spark.read
  .format("delta")
  .option("readChangeData", True)
  .option("startingTimestamp", '2021-08-22 00:00:00')
  .option("endingTimestamp", '2021-08-29 00:00:00')
  .table("user_lookup")
  .createOrReplaceTempView("changes"))

spark.sql("""
  DELETE FROM user_aggregates
  WHERE user_id IN (
    SELECT user_id
    FROM changes
    WHERE _change_type='delete'
    )
""")
```

Assuming that **user_id** is a unique identifying key and that all users that have requested deletion have

been removed from the **`user_lookup`** table, which statement describes whether successfully executing the above logic guarantees that the records to be deleted from the **`user_aggregates`** table are no longer accessible and why?

A. No; the Delta Lake **DELETE** command only provides ACID guarantees when combined with the **MERGE INTO** command.
B. No; files containing deleted records may still be accessible with time travel until a **VACUUM** command is used to remove invalidated data files.
C. No; the change data feed only tracks inserts and updates, not deleted records.
D. Yes; Delta Lake ACID guarantees provide assurance that the **DELETE** command succeeded fully and permanently purged these records.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 166**
An external object storage container has been mounted to the location `/mnt/finance_eda_bucket`.

The following logic was executed to create a database for the finance team:

```
CREATE DATABASE finance_eda_db
LOCATION '/mnt/finance_eda_bucket';
GRANT USAGE ON DATABASE finance_eda_db TO finance;
GRANT CREATE ON DATABASE finance_eda_db TO finance;
```

After the database was successfully created and permissions configured, a member of the finance team runs the following code:

```
CREATE TABLE finance_eda_db.tx_sales AS
SELECT *
FROM sales
WHERE state = "TX";
```

If all users on the finance team are members of the **finance** group, which statement describes how the **tx_sales** table will be created?

A. A logical table will persist the query plan to the Hive Metastore in the Databricks control plane.
B. An external table will be created in the storage container mounted to **/mnt/finance_eda_bucket**.
C. A managed table will be created in the DBFS root storage container.
D. An managed table will be created in the storage container mounted to **/mnt/finance_eda_bucket**.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 167**
The data engineering team has been tasked with configuring connections to an external database that

does not have a supported native connector with Databricks. The external database already has data security configured by group membership. These groups map directly to user groups already created in Databricks that represent various teams within the company.

A new login credential has been created for each group in the external database. The Databricks Utilities Secrets module will be used to make these credentials available to Databricks users.

Assuming that all the credentials are configured correctly on the external database and group membership is properly configured on Databricks, which statement describes how teams can be granted the minimum necessary access to using these credentials?

A. No additional configuration is necessary as long as all users are configured as administrators in the workspace where secrets have been added.
B. "Read" permissions should be set on a secret key mapped to those credentials that will be used by a given team.
C. "Read" permissions should be set on a secret scope containing only those credentials that will be used by a given team.
D. "Manage" permissions should be set on a secret scope containing only those credentials that will be used by a given team.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 168**
What is the retention of job run history?

A. It is retained until you export or delete job run logs
B. It is retained for 30 days, during which time you can deliver job run logs to DBFS or S3
C. It is retained for 60 days, during which you can export notebook run results to HTML
D. It is retained for 60 days, after which logs are archived

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 169**
A data engineer, User A, has promoted a new pipeline to production by using the REST API to programmatically create several jobs. A DevOps engineer, User B, has configured an external orchestration tool to trigger job runs through the REST API. Both users authorized the REST API calls using their personal access tokens.

Which statement describes the contents of the workspace audit logs concerning these events?

A. Because the REST API was used for job creation and triggering runs, a Service Principal will be automatically used to identify these events.
B. Because User A created the jobs, their identity will be associated with both the job creation events and the job run events.
C. Because these events are managed separately, User A will have their identity associated with the job creation events and User B will have their identity associated with the job run events.
D. Because the REST API was used for job creation and triggering runs, user identity will not be captured in the audit logs.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 170**
A distributed team of data analysts share computing resources on an interactive cluster with autoscaling configured. In order to better manage costs and query throughput, the workspace administrator is hoping to evaluate whether cluster upscaling is caused by many concurrent users or resource-intensive queries.

In which location can one review the timeline for cluster resizing events?

A. Workspace audit logs
B. Driver's log file
C. Ganglia
D. Cluster Event Log

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 171**
When evaluating the Ganglia Metrics for a given cluster with 3 executor nodes, which indicator would signal proper utilization of the VM's resources?

A. The five Minute Load Average remains consistent/flat
B. CPU Utilization is around 75%
C. Network I/O never spikes
D. Total Disk Space remains constant



**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 172**
The data engineer is using Spark's **MEMORY_ONLY** storage level.

Which indicators should the data engineer look for in the Spark UI's Storage tab to signal that a cached table is not performing optimally?

A. On Heap Memory Usage is within 75% of Off Heap Memory Usage
B. The RDD Block Name includes the “*” annotation signaling a failure to cache
C. Size on Disk is > 0
D. The number of Cached Partitions > the number of Spark Partitions

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 173**
Review the following error traceback:

```
---------------------------------------------------------------------------
AnalysisException                         Traceback (most recent call last)
<command-3293767849433948> in <module>
----> 1 display(df.select(3*"heartrate"))

/databricks/spark/python/pyspark/sql/dataframe.py in select(self, *cols)
   1690         [Row(name='Alice', age=12), Row(name='Bob', age=15)]
   1691         """
-> 1692         jdf = self._jdf.select(self._jcols(*cols))
   1693         return DataFrame(jdf, self.sql_ctx)
   1694

/databricks/spark/python/lib/py4j-0.10.9-src.zip/py4j/java_gateway.py in __call__(self, *args)
   1302
   1303         answer = self.gateway_client.send_command(command)
-> 1304         return_value = get_return_value(
   1305             answer, self.gateway_client, self.target_id, self.name)
   1306

/databricks/spark/python/pyspark/sql/utils.py in deco(*a, **kw)
    121                 # Hide where the exception came from that shows a non-Pythonic
    122                 # JVM exception message.
--> 123                 raise converted from None
    124             else:
    125                 raise

AnalysisException: cannot resolve '`heartrateheartrateheartrate`' given input columns:
[spark_catalog.database.table.device_id, spark_catalog.database.table.heartrate,
spark_catalog.database.table.mrn, spark_catalog.database.table.time];
'Project ['heartrateheartrateheartrate]
+- SubqueryAlias spark_catalog.database.table
   +- Relation[device_id#75L,heartrate#76,mrn#77L,time#78] parquet
```

Which statement describes the error being raised?

A. There is a syntax error because the **heartrate** column is not correctly identified as a column.
B. There is no column in the table named **heartrateheartrateheartrate**
C. There is a type error because a column object cannot be multiplied.
D. There is a type error because a DataFrame object cannot be multiplied.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 174**
What is a method of installing a Python package scoped at the notebook level to all nodes in the currently active cluster?

A. Run source `env/bin/activate` in a notebook setup script
B. `Install libraries` from PyPI using the cluster UI
C. Use `%pip` install in a notebook cell
D. Use `%sh pip install` in a notebook cell

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

Reference:

**QUESTION 175**
What is the first line of a Databricks Python notebook when viewed in a text editor?

A. `%python`
B. `// Databricks notebook source`
C. `# Databricks notebook source`
D. `-- Databricks notebook source`

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 176**
Incorporating unit tests into a PySpark application requires upfront attention to the design of your jobs, or a potentially significant refactoring of existing code.

Which benefit offsets this additional effort?

A. Improves the quality of your data
B. Validates a complete use case of your application
C. Troubleshooting is easier since all steps are isolated and tested individually
D. Ensures that all steps interact correctly to achieve the desired end result

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**



**QUESTION 177**
What describes integration testing?

A. It validates an application use case.
B. It validates behavior of individual elements of an application,
C. It requires an automated testing framework.
D. It validates interactions between subsystems of your application.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 178**
The Databricks CLI is used to trigger a run of an existing job by passing the **job_id** parameter. The response that the job run request has been submitted successfully includes a field **run_id**.

Which statement describes what the number alongside this field represents?

A. The **job_id** and number of times the job has been run are concatenated and returned.
B. The globally unique ID of the newly triggered run.
C. The number of times the job definition has been run in this workspace.
D. The **job_id** is returned in this field.

**Correct Answer:** B

**Section:** (none)

**Explanation/Reference:**
Reference:

**QUESTION 179**
A Databricks job has been configured with three tasks, each of which is a Databricks notebook. Task A does not depend on other tasks. Tasks B and C run in parallel, with each having a serial dependency on task A.

What will be the resulting state if tasks A and B complete successfully but task C fails during a scheduled run?

A. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; some operations in task C may have completed successfully.
B. Unless all tasks complete successfully, no changes will be committed to the Lakehouse; because task C failed, all commits will be rolled back automatically.
C. Because all tasks are managed as a dependency graph, no changes will be committed to the Lakehouse until all tasks have successfully been completed.
D. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; any changes made in task C will be rolled back due to task failure.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 180**
When scheduling Structured Streaming jobs for production, which configuration automatically recovers from query failures and keeps costs low?

A. Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1
B. Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: Unlimited
C. Cluster: Existing All-Purpose Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1
D. Cluster: New Job Cluster;
Retries: None;
Maximum Concurrent Runs: 1

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
**New Job Cluster**: This configuration ensures a clean, isolated environment for each job run, reducing dependencies or conflicts that may arise from shared resources on an all-purpose cluster.
**Retries: Unlimited**: By setting retries to unlimited, the job will automatically attempt to recover from transient failures, a common requirement in production environments where job availability is critical.
**Maximum Concurrent Runs: 1**: This setting ensures that only one instance of the job runs at a time, preventing unnecessary parallel runs that could increase costs and interfere with each other.

**QUESTION 181**
A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_stor
USING DELTA
LOCATION "/mnt/prod/sales_by_store"
```

Realizing that the original query had a typographical error, the below code was executed:

```
ALTER TABLE prod.sales_by_stor RENAME TO prod.sales_by_store
```

Which result will occur after running the second command?

A. The table reference in the metastore is updated.
B. All related files and metadata are dropped and recreated in a single ACID transaction.
C. The table name change is recorded in the Delta transaction log.
D. A new Delta transaction log is created for the renamed table.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
When the `ALTER TABLE ... RENAME TO ...` command is used in Databricks, it only updates the table reference in the metastore. The operation changes the name of the table without affecting the underlying data files or creating a new Delta transaction log. This is a metadata-only operation, meaning it's lightweight and doesn't involve recreating or moving files.

**QUESTION 182**
The data engineering team has configured a Databricks SQL query and alert to monitor the values in a Delta Lake table. The **`recent_sensor_recordings`** table contains an identifying **`sensor_id`** alongside the **`timestamp`** and **`temperature`** for the most recent 5 minutes of recordings.

The below query is used to create the alert:

```
SELECT MEAN(temperature), MAX(temperature), MIN(temperature)
FROM recent_sensor_recordings
GROUP BY sensor_id
```

The query is set to refresh each minute and always completes in less than 10 seconds. The alert is set to trigger when **`mean (temperature) > 120`**. Notifications are triggered to be sent at most every 1 minute.

If this alert raises notifications for 3 consecutive minutes and then stops, which statement must be true?

A. The total average temperature across all sensors exceeded 120 on three consecutive executions of the query
B. The average temperature recordings for at least one sensor exceeded 120 on three consecutive executions of the query
C. The source query failed to update properly for three consecutive minutes and then restarted
D. The maximum temperature recording for at least one sensor exceeded 120 on three consecutive executions of the query

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**

**QUESTION 183**
A junior developer complains that the code in their notebook isn't producing the correct results in the development environment. A shared screenshot reveals that while they're using a notebook versioned with

Databricks Repos, they're using a personal branch that contains old logic. The desired branch named **`dev-2.3.9`** is not available from the branch selection dropdown.

Which approach will allow this developer to review the current logic for this notebook?

A. Use Repos to make a pull request use the Databricks REST API to update the current branch to **`dev-2.3.9`**
B. Use Repos to pull changes from the remote Git repository and select the **`dev-2.3.9`** branch.
C. Use Repos to checkout the **`dev-2.3.9`** branch and auto-resolve conflicts with the current branch
D. Use Repos to merge the current branch and the **`dev-2.3.9`** branch, then make a pull request to sync with the remote repository

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
In Databricks Repos, if the desired branch (in this case, `dev-2.3.9`) is not available, the developer likely needs to fetch the latest updates from the remote Git repository. By using Repos to pull changes, they can ensure that the most recent branches, including `dev-2.3.9`, are available. After pulling, the developer can select the `dev-2.3.9` branch to view the current logic.

**QUESTION 184**
Two of the most common data locations on Databricks are the DBFS root storage and external object storage mounted with **`dbutils.fs.mount()`**.

Which of the following statements is correct?

A. DBFS is a file system protocol that allows users to interact with files stored in object storage using syntax and guarantees similar to Unix file systems.
B. By default, both the DBFS root and mounted data sources are only accessible to workspace administrators.
C. The DBFS root is the most secure location to store data, because mounted storage volumes must have full public read and write permissions.
D. The DBFS root stores files in ephemeral block volumes attached to the driver, while mounted directories will always persist saved data to external storage between sessions.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Databricks File System (DBFS) is an abstraction layer over cloud storage (such as S3, Azure Blob Storage, etc.) that allows users to interact with files stored in object storage as if they were using a Unix-like file system. This simplifies file operations in Databricks and allows users to use familiar commands and syntax.

**QUESTION 185**
An upstream system has been configured to pass the date for a given batch of data to the Databricks Jobs API as a parameter. The notebook to be scheduled will use this parameter to load data with the following code:

```
df = spark.read.format("parquet").load(f"/mnt/source/(date)")
```

Which code block should be used to create the **date** Python variable used in the above code block?

A.
```
date = spark.conf.get("date")
```
B.
```
import sys
date = sys.argv[1]
```
C.
```
date = dbutils.notebooks.getParam("date")
```

D.
```
dbutils.widgets.text("date", "null")
date = dbutils.widgets.get("date")
```

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
In Databricks, the `dbutils.widgets` API is used to define and retrieve parameters passed to a notebook. By creating a widget with `dbutils.widgets.text("date", "null")`, you define a parameter named "date" with a default value of `"null"`. You can then retrieve the actual value passed through the Databricks Jobs API using `dbutils.widgets.get("date")`.

**QUESTION 186**
The Databricks workspace administrator has configured interactive clusters for each of the data engineering groups. To control costs, clusters are set to terminate after 30 minutes of inactivity. Each user should be able to execute workloads against their assigned clusters at any time of the day.

Assuming users have been added to a workspace but not granted any permissions, which of the following describes the minimal permissions a user would need to start and attach to an already configured cluster.

A. "Can Manage" privileges on the required cluster
B. Cluster creation allowed, "Can Restart" privileges on the required cluster
C. Cluster creation allowed, "Can Attach To" privileges on the required cluster
D. "Can Restart" privileges on the required cluster

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**



**QUESTION 187**
The data science team has created and logged a production model using MLflow. The following code correctly imports and applies the production model to output the predictions as a new DataFrame named **`preds`** with the schema "**`customer_id LONG, predictions DOUBLE, date DATE`**".

```
from pyspark.sql.functions import current_date

model = mlflow.pyfunc.spark_udf(spark, model_uri="models:/churn/prod")
df = spark.table("customers")
columns = ["account_age", "time_since_last_seen", "app_rating"]
preds = (df.select(
  "customer_id",
  model(*columns).alias("predictions"),
  current_date().alias("date")
  )
```

The data science team would like predictions saved to a Delta Lake table with the ability to compare all predictions across time. Churn predictions will be made at most once per day.

Which code block accomplishes this task while minimizing potential compute costs?

A. `preds.write.mode("append").saveAsTable("churn_preds")`
B. `preds.write.format("delta").save("/preds/churn_preds")`

C.

```
(preds.writeStream
  .outputMode("append")
  .option("checkpointPath", "/_checkpoints/churn_preds")
  .table("churn_preds")
)
```

D.

```
(preds.write
  .format("delta")
  .mode("overwrite")
  .saveAsTable("churn_preds")
)
```

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

**QUESTION 188**
The following code has been migrated to a Databricks notebook from a legacy workload:

```
%sh
git clone https://github.com/foo/data_loader;
python ./data_loader/run.py;
mv ./output /dbfs/mnt/new_data
```

The code executes successfully and provides the logically correct results, however, it takes over 20 minutes to extract and load around 1 GB of data.

Which statement is a possible explanation for this behavior?

A. **%sh** triggers a cluster restart to collect and install Git. Most of the latency is related to cluster startup time.
B. Instead of cloning, the code should use **%sh pip install** so that the Python code can get executed in parallel across all nodes in a cluster.
C. **%sh** does not distribute file moving operations; the final line of code should be updated to use **%fs** instead.
D. **%sh** executes shell code on the driver node. The code does not take advantage of the worker nodes or Databricks optimized Spark.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
The %sh magic command in Databricks allows you to run shell commands directly, but these commands are executed on the **driver node** only. This means that the provided code is running in a single-threaded environment, which can be very inefficient for operations like cloning a repository, running Python scripts,

and moving large amounts of data (like 1 GB). The driver node's limited resources compared to a distributed cluster setup are the reason for the slowness, as the worker nodes and Databricks' Spark optimizations are not being utilized.

**QUESTION 189**
A Delta table of weather records is partitioned by date and has the below schema:

```
date DATE, device_id INT, temp FLOAT, latitude FLOAT, longitude FLOAT
```

To find all the records from within the Arctic Circle, you execute a query with the below filter:

```
latitude > 66.3
```

Which statement describes how the Delta engine identifies which files to load?

A. All records are cached to an operational database and then the filter is applied
B. The Parquet file footers are scanned for min and max statistics for the **`latitude`** column
C. The Hive metastore is scanned for min and max statistics for the **`latitude`** column
D. The Delta log is scanned for min and max statistics for the **`latitude`** column

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 190**
In order to prevent accidental commits to production data, a senior data engineer has instituted a policy that all development work will reference clones of Delta Lake tables. After testing both **`DEEP`** and **`SHALLOW CLONE`**, development tables are created using **`SHALLOW CLONE`**.

A few weeks after initial table creation, the cloned versions of several tables implemented as Type 1 Slowly Changing Dimension (SCD) stop working. The transaction logs for the source tables show that **`VACUUM`** was run the day before.

Which statement describes why the cloned tables are no longer working?

A. Because Type 1 changes overwrite existing records, Delta Lake cannot guarantee data consistency for cloned tables.
B. Running **`VACUUM`** automatically invalidates any shallow clones of a table; **`DEEP CLONE`** should always be used when a cloned table will be repeatedly queried.
C. The data files compacted by **`VACUUM`** are not tracked by the cloned metadata; running **`REFRESH`** on the cloned table will pull in recent changes.
D. The metadata created by the **`CLONE`** operation is referencing data files that were purged as invalid by the **`VACUUM`** command.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
A **SHALLOW CLONE** in Delta Lake only replicates the metadata of the source table, meaning that the data files themselves are not duplicated; they are shared between the original table and the clone. When a **VACUUM** operation is executed, it deletes any data files that are no longer needed by the source table, which can include files still referenced by the shallow clone. As a result, the cloned tables may lose access to the data they need, causing the cloned versions to fail.

**QUESTION 191**
A junior data engineer has configured a workload that posts the following JSON to the Databricks REST API endpoint **`2.0/jobs/create`**.

```
{
  "name": "Ingest new data",
  "existing_cluster_id": "6015-954420-peace720",
  "notebook_task": {
    "notebook_path": "/Prod/ingest.py"
  }
}
```

Which statement describes the result of executing this workload three times assuming that all configurations and referenced resources are available?

A. The logic defined in the referenced notebook will be executed three times on the referenced existing all purpose cluster.
B. The logic defined in the referenced notebook will be executed three times on new clusters with the configurations of the provided cluster ID.
C. Three new jobs named "Ingest new data" will be defined in the workspace, but no jobs will be executed.
D. One new job named "Ingest new data" will be defined in the workspace, but it will not be executed.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

**QUESTION 192**
A Delta Lake table in the Lakehouse named **`customer_churn_params`** is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.

Immediately after each update succeeds, the data engineering team would like to determine the difference between the new version and the previous version of the table.

Given the current implementation, which method can be used?

A. Execute a query to calculate the difference between the new version and the previous version using Delta Lake's built-in versioning and lime travel functionality.
B. Parse the Delta Lake transaction log to identify all newly written data files.
C. Parse the Spark event logs to identify those rows that were updated, inserted, or deleted.
D. Execute **`DESCRIBE HISTORY customer_churn_params`** to obtain the full operation metrics for the update, including a log of all records that have been added or modified.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Delta Lake provides **versioning and time travel** capabilities, which allow you to access previous versions of a table. By using these features, you can execute a query that compares the current version of the table with a previous version to identify changes. This is the most efficient method to determine the difference between the updated version and the prior state of the table.

**QUESTION 193**
A view is registered with the following code:

```
CREATE VIEW recent_orders AS (
  SELECT a.user_id, a.email, b.order_id, b.order_date
  FROM
    (SELECT user_id, email
    FROM users) a
    INNER JOIN
    (SELECT user_id, order_id, order_date
    FROM orders
    WHERE order_date >= (current_date() - 7)) b
    ON a.user_id = b.user_id
)
```

Both users and orders are Delta Lake tables.

Which statement describes the results of querying **`recent_orders`**?

A. The versions of each source table will be stored in the table transaction log; query results will be saved to DBFS with each query.
B. All logic will execute when the table is defined and store the result of joining tables to the DBFS; this stored data will be returned when the table is queried.
C. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query finishes.
D. All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.



**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 194**
A data engineer is performing a join operation to combine values from a static **`userLookup`** table with a streaming DataFrame **`streamingDF`**.

Which code block attempts to perform an invalid stream-static join?

A. `userLookup.join(streamingDF, ["user_id"], how="right")`
B. `streamingDF.join(userLookup, ["user_id"], how="inner")`
C. `userLookup.join(streamingDF, ["user_id"], how="inner")`
D. `userLookup.join(streamingDF, ["user_id"], how="left")`

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
In Delta Lake and Spark, **stream-static joins** are allowed under certain conditions. When performing a join between a streaming DataFrame and a static DataFrame (or table), the streaming DataFrame must be the left side of the join, and only **inner** and **right** joins are allowed.

The static table (`userLookup`) is on the left side of the join, and the `how="left"` join is specified, which is **invalid** for a stream-static join because the static DataFrame cannot drive the join and return unmatched records from the left side.

**QUESTION 195**

A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame **df**. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Incremental state information should be maintained for 10 minutes for late-arriving data.

Streaming DataFrame **df** has the following schema:

```
"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"
```

Code block:

```
df._____
    .groupBy(
        window("event_time", "5 minutes").alias("time"),
        "device_id"
    )
    .agg(
        avg("temp").alias("avg_temp"),
        avg("humidity").alias("avg_humidity")
    )
    .writeStream
    .format("delta")
    .saveAsTable("sensor_avg")
```

Choose the response that correctly fills in the blank within the code block to complete this task.

A. `withWatermark("event_time", "10 minutes")`
B. `awaitArrival("event_time", "10 minutes")`
C. `await("event_time + ‘10 minutes'")`
D. `slidingWindow("event_time", "10 minutes")`

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
In structured streaming, when working with windowed aggregations, you can use the **`withWatermark()`** method to handle late-arriving data. The watermark specifies how long the system should wait for late data before considering the data for a given window complete. In this case, the watermark is set to **10 minutes**, allowing the aggregation to incorporate any data that arrives within 10 minutes of its event time.

**QUESTION 196**
A data architect has designed a system in which two Structured Streaming jobs will concurrently write to a single bronze Delta table. Each job is subscribing to a different topic from an Apache Kafka source, but they will write data with the same schema. To keep the directory structure simple, a data engineer has decided to nest a checkpoint directory to be shared by both streams.

The proposed directory structure is displayed below:


Which statement describes whether this checkpoint directory structure is valid for the given scenario and why?

A. No; Delta Lake manages streaming checkpoints in the transaction log.
B. Yes; both of the streams can share a single checkpoint directory.
C. No; only one stream can write to a Delta Lake table.
D. No; each of the streams needs to have its own checkpoint directory.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
When writing with **Structured Streaming** in Delta Lake, each streaming job must have its own **checkpoint directory**. The checkpoint directory keeps track of streaming state and progress, allowing the system to recover in case of failure or interruption. If two streams share a single checkpoint directory, it will lead to inconsistent metadata and potentially data corruption because the two streams will be overwriting each other's progress and metadata.

**QUESTION 197**
A Structured Streaming job deployed to production has been experiencing delays during peak hours of the day. At present, during normal execution, each microbatch of data is processed in less than 3 seconds. During peak hours of the day, execution time for each microbatch becomes very inconsistent, sometimes exceeding 30 seconds. The streaming write is currently configured with a trigger interval of 10 seconds.

Holding all other variables constant and assuming records need to be processed in less than 10 seconds, which adjustment will meet the requirement?

A. Decrease the trigger interval to 5 seconds; triggering batches more frequently allows idle executors to begin processing the next batch while longer running tasks from previous batches finish.
B. Decrease the trigger interval to 5 seconds; triggering batches more frequently may prevent records from backing up and large batches from causing spill.
C. The trigger interval cannot be modified without modifying the checkpoint directory; to maintain the current stream state, increase the number of shuffle partitions to maximize parallelism.
D. Use the trigger once option and configure a Databricks job to execute the query every 10 seconds; this ensures all backlogged records are processed with each batch.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
When the trigger interval is set to 10 seconds, during peak hours the delay may cause more records to accumulate, leading to larger and potentially slower batches, especially as resources become overburdened. **Reducing the trigger interval** to 5 seconds allows for smaller batches to be processed more frequently, which can help prevent **record backlog** and reduce the risk of large batches causing **spill** and other resource-intensive operations. This adjustment keeps processing times manageable and more consistent, ensuring records are processed within the required window.

**QUESTION 198**
Which statement describes the default execution mode for Databricks Auto Loader?

A. Cloud vendor-specific queue storage and notification services are configured to track newly arriving files; new files are incrementally and idempotently loaded into the target Delta Lake table.
B. New files are identified by listing the input directory; the target table is materialized by directly querying all valid files in the source directory.
C. Webhooks trigger a Databricks job to run anytime new data arrives in a source directory; new data are automatically merged into target tables using rules inferred from the data.
D. New files are identified by listing the input directory; new files are incrementally and idempotently loaded into the target Delta Lake table.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 199**
Which statement describes the correct use of `pyspark.sql.functions.broadcast`?

A. It marks a column as having low enough cardinality to properly map distinct values to available partitions, allowing a broadcast join.
B. It marks a column as small enough to store in memory on all executors, allowing a broadcast join.
C. It caches a copy of the indicated table on all nodes in the cluster for use in all future queries during the cluster lifetime.
D. It marks a DataFrame as small enough to store in memory on all executors, allowing a broadcast join.



**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
In PySpark, `pyspark.sql.functions.broadcast()` is used to indicate that a **DataFrame** is small enough to be **broadcasted** to all executors' memory. This allows for a **broadcast join**, where the smaller DataFrame is copied to all executor nodes, enabling each executor to perform the join locally without needing to shuffle the larger DataFrame. This significantly improves performance for joining a large DataFrame with a small one.

**QUESTION 200**
Spill occurs as a result of executing various wide transformations. However, diagnosing spill requires one to proactively look for key indicators.

Where in the Spark UI are two of the primary indicators that a partition is spilling to disk?

A. Stage’s detail screen and Query’s detail screen
B. Stage’s detail screen and Executor’s log files
C. Driver’s and Executor’s log files
D. Executor’s detail screen and Executor’s log files

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
In Spark, **spill** occurs when the data does not fit in memory and must be written to disk. To diagnose spill effectively:
**Stage’s Detail Screen**: In the **Spark UI**, you can look at the **stage’s detail screen** to see metrics like the amount of memory spilled to disk during execution, often indicated in metrics like "Spilled Bytes" or "Disk

Spilled."
**Executor's Log Files**: The **executor's log files** contain detailed information about spill events, including warnings or errors when partitions exceed memory limits and spill to disk. These logs are helpful for diagnosing and understanding the circumstances leading to spilling.

**QUESTION 201**
An upstream source writes Parquet data as hourly batches to directories named with the current date. A nightly batch job runs the following code to ingest all data from the previous day as indicated by the **`date`** variable:

```
(spark.read
  .format("parquet")
  .load(f"/mnt/raw_orders/{date}")
  .dropDuplicates(["customer_id", "order_id"])
  .write
  .mode("append")
  .saveAsTable("orders")
)
```

Assume that the fields **`customer_id`** and **`order_id`** serve as a composite key to uniquely identify each order.

If the upstream system is known to occasionally produce duplicate entries for a single order hours apart, which statement is correct?

A. Each write to the **`orders`** table will only contain unique records, and only those records without duplicates in the target table will be written.
B. Each write to the **`orders`** table will only contain unique records, but newly written records may have duplicates already present in the target table.
C. Each write to the **`orders`** table will only contain unique records; if existing records with the same key are present in the target table, these records will be overwritten.
D. Each write to the **`orders`** table will run deduplication over the union of new and existing records, ensuring no duplicate records are present.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
The code reads Parquet files, drops duplicates **within the batch being processed**, and then appends the records to the existing `orders` Delta table. The `dropDuplicates(["customer_id", "order_id"])` operation ensures that each hourly batch of data being processed only contains unique records based on the composite key (`customer_id`, `order_id`). However, **appending** to the `orders` table using `.mode("append")` does **not perform deduplication across the entire table**, meaning that if duplicates already exist in the target table, those will remain.

**QUESTION 202**
A junior data engineer has implemented the following code block.

```
MERGE INTO events
USING new_events
ON events.event_id = new_events.event_id
WHEN NOT MATCHED
  INSERT *
```

The view **`new_events`** contains a batch of records with the same schema as the **`events`** Delta table. The **`event_id`** field serves as a unique key for this table.

When this query is executed, what will happen with new records that have the same **`event_id`** as an existing record?

A. They are merged.
B. They are ignored.
C. They are updated.
D. They are inserted.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
The provided `MERGE` statement only has a `WHEN NOT MATCHED` clause with an `INSERT` operation. This means that if the `event_id` in the `new_events` batch does **not** match any existing `event_id` in the `events` table, the new record will be **inserted**.

However, if there **is** a matching `event_id`, no action is specified for the matched records, meaning they will simply be **ignored** and **not updated**. This is because there is no `WHEN MATCHED` clause to define an update or other action.

**QUESTION 203**
A new data engineer notices that a critical field was omitted from an application that writes its Kafka source to Delta Lake. This happened even though the critical field was in the Kafka source. That field was further missing from data written to dependent, long-term storage. The retention threshold on the Kafka service is seven days. The pipeline has been in production for three months.

Which describes how Delta Lake can help to avoid data loss of this nature in the future?

A. The Delta log and Structured Streaming checkpoints record the full history of the Kafka producer.
B. Delta Lake schema evolution can retroactively calculate the correct value for newly added fields, as long as the data was in the original source.
C. Delta Lake automatically checks that all fields present in the source data are included in the ingestion layer.
D. Ingesting all raw data and metadata from Kafka to a bronze Delta table creates a permanent, replayable history of the data state.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
In Delta Lake, a common practice is to create a **bronze table** that ingests **all raw data** from the source, such as Kafka. This approach allows you to store the complete set of incoming data without any transformation or filtering, creating a **permanent, replayable history**. If there is ever a need to add a previously omitted field, you can go back to the bronze table, which holds the original data, and replay it, ensuring that no data is lost.

**QUESTION 204**
The data engineering team maintains the following code:

```
import pyspark.sql.functions as F

(spark.table("silver_customer_sales")
  .groupBy("customer_id")
  .agg(
    F.min("sale_date").alias("first_transaction_date"),
    F.max("sale_date").alias("last_transaction_date"),
    F.mean("sale_total").alias("average_sales"),
    F.countDistinct("order_id").alias("total_orders"),
    F.sum("sale_total").alias("lifetime_value")
  ).write
  .mode("overwrite")
  .table("gold_customer_lifetime_sales_summary")
)
```

Assuming that this code produces logically correct results and the data in the source table has been de-duplicated and validated, which statement describes what will occur when this code is executed?

A. The **`silver_customer_sales`** table will be overwritten by aggregated values calculated from all records in the **`gold_customer_lifetime_sales_summary`** table as a batch job.
B. A batch job will update the **`gold_customer_lifetime_sales_summary`** table, replacing only those rows that have different values than the current version of the table, using **`customer_id`** as the primary key.
C. The **`gold_customer_lifetime_sales_summary`** table will be overwritten by aggregated values calculated from all records in the **`silver_customer_sales`** table as a batch job.
D. An incremental job will detect if new rows have been written to the **`silver_customer_sales`** table; if new rows are detected, all aggregates will be recalculated and used to overwrite the **`gold_customer_lifetime_sales_summary`** table.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
The attached code reads from the `silver_customer_sales` table, performs **aggregations** on it (using `groupBy` and `agg`), and then **writes the result** to the `gold_customer_lifetime_sales_summary` table with `.write.mode("overwrite")`. The `mode("overwrite")` means that the target table (`gold_customer_lifetime_sales_summary`) will be completely overwritten by the results of this batch aggregation, which includes recalculating all the aggregated metrics for every `customer_id`.

**QUESTION 205**
The data engineering team is migrating an enterprise system with thousands of tables and views into the Lakehouse. They plan to implement the target architecture using a series of bronze, silver, and gold tables. Bronze tables will almost exclusively be used by production data engineering workloads, while silver tables will be used to support both data engineering and machine learning workloads. Gold tables will largely serve business intelligence and reporting purposes. While personal identifying information (PII) exists in all tiers of data, pseudonymization and anonymization rules are in place for all data at the silver and gold

levels.

The organization is interested in reducing security concerns while maximizing the ability to collaborate across diverse teams.

Which statement exemplifies best practices for implementing this system?

A. Isolating tables in separate databases based on data quality tiers allows for easy permissions management through database ACLs and allows physical separation of default storage locations for managed tables.
B. Because databases on Databricks are merely a logical construct, choices around database organization do not impact security or discoverability in the Lakehouse.
C. Storing all production tables in a single database provides a unified view of all data assets available throughout the Lakehouse, simplifying discoverability by granting all users view privileges on this database.
D. Working in the default Databricks database provides the greatest security when working with managed tables, as these will be created in the DBFS root.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Isolating tables based on **data quality tiers** (bronze, silver, gold) into **separate databases** allows for easier **permission management** through **database ACLs** (Access Control Lists). This enables you to apply different permissions for different teams or users based on their roles and responsibilities. Moreover, it provides a **physical separation** of the default storage locations for managed tables, which enhances data organization and security. This approach aligns well with the goal of reducing security concerns while allowing collaboration across diverse teams, as it helps control access to sensitive data (like PII) more effectively.

**QUESTION 206**
The data architect has mandated that all tables in the Lakehouse should be configured as external (also known as "unmanaged") Delta Lake tables.

Which approach will ensure that this requirement is met?

A. When a database is being created, make sure that the **`LOCATION`** keyword is used.
B. When the workspace is being configured, make sure that external cloud object storage has been mounted.
C. When data is saved to a table, make sure that a full file path is specified alongside the **`USING DELTA`** clause.
D. When tables are created, make sure that the **`UNMANAGED`** keyword is used in the **`CREATE TABLE`** statement.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
In **Delta Lake**, to create **external (unmanaged) tables**, you must specify the **full file path** where the data should be stored. This is typically done by including the `LOCATION` keyword in the `CREATE TABLE` statement or when using `USING DELTA`. By explicitly providing the **path**, you ensure that the data resides outside of Databricks-managed storage, thus making the table **unmanaged**.

**QUESTION 207**
To reduce storage and compute costs, the data engineering team has been tasked with curating a series of aggregate tables leveraged by business intelligence dashboards, customer-facing applications, production machine learning models, and ad hoc analytical queries.

The data engineering team has been made aware of new requirements from a customer-facing

application, which is the only downstream workload they manage entirely. As a result, an aggregate table used by numerous teams across the organization will need to have a number of fields renamed, and additional fields will also be added.

Which of the solutions addresses the situation while minimally interrupting other teams in the organization without increasing the number of tables that need to be managed?

A. Send all users notice that the schema for the table will be changing; include in the communication the logic necessary to revert the new table schema to match historic queries.
B. Configure a new table with all the requisite fields and new names and use this as the source for the customer-facing application; create a view that maintains the original data schema and table name by aliasing select fields from the new table.
C. Create a new table with the required schema and new fields and use Delta Lake's deep clone functionality to sync up changes committed to one table to the corresponding table.
D. Replace the current table definition with a logical view defined with the query logic currently writing the aggregate table; create a new table to power the customer-facing application.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
This solution allows the **new schema** and **additional fields** to be used by the customer-facing application, without disrupting the **existing consumers** of the original table. By creating a **view** that aliases fields to match the original schema, downstream users can continue using their current queries without any changes. This approach minimizes the interruption to other teams while ensuring that the new requirements are met.

**QUESTION 208**
A Delta Lake table representing metadata about content posts from users has the following schema:

```
user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude
FLOAT, post_time TIMESTAMP, date DATE
```

Based on the above schema, which column is a good candidate for partitioning the Delta Table?

A. `post_time`
B. `date`
C. `post_id`
D. `user_id`

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
**Partitioning** a Delta Lake table is most effective when you choose a column that has a **moderate cardinality** (not too many distinct values) and is frequently used in filters during queries. The **`date`** column is a good candidate because:
**Moderate Cardinality**: The `date` column will likely have fewer unique values (e.g., one value per day) compared to other columns like `post_id` or `user_id`, which can have a very large number of distinct values. This makes it ideal for partitioning.
**Efficient Filtering**: Partitioning by `date` can make queries that filter by specific dates or date ranges much more efficient, reducing the amount of data read during query execution.

**QUESTION 209**
The downstream consumers of a Delta Lake table have been complaining about data quality issues impacting performance in their applications. Specifically, they have complained that invalid **`latitude`** and **`longitude`** values in the **`activity_details`** table have been breaking their ability to use other geolocation processes.

A junior engineer has written the following code to add **CHECK** constraints to the Delta Lake table:

```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
  latitude >= -90 AND
  latitude <= 90 AND
  longitude >= -180 AND
  longitude <= 180);
```

A senior engineer has confirmed the above logic is correct and the valid ranges for latitude and longitude are provided, but the code fails when executed.

Which statement explains the cause of this failure?

A. The current table schema does not contain the field **valid_coordinates**; schema evolution will need to be enabled before altering the table to add a constraint.
B. The **activity_details** table already exists; **CHECK** constraints can only be added during initial table creation.
C. The **activity_details** table already contains records that violate the constraints; all existing data must pass **CHECK** constraints in order to add them to an existing table.
D. The **activity_details** table already contains records; **CHECK** constraints can only be added prior to inserting values into a table.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
When adding a **CHECK constraint** to a Delta Lake table, the constraint ensures that **all existing records** meet the specified condition. If there are any records that **violate** the new constraint (in this case, having invalid latitude or longitude values), the ALTER TABLE statement will **fail**. Therefore, you must ensure that all existing data satisfies the constraint before you can add it.

**QUESTION 210**
What is true for Delta Lake?

A. Views in the Lakehouse maintain a valid cache of the most recent versions of source tables at all times.
B. Primary and foreign key constraints can be leveraged to ensure duplicate values are never entered into a dimension table.
C. Delta Lake automatically collects statistics on the first 32 columns of each table which are leveraged in data skipping based on query filters.
D. Z-order can only be applied to numeric values stored in Delta Lake tables.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Delta Lake collects **statistics** for the first 32 columns of a table, which are used for **data skipping**. These statistics include min and max values for each partition, allowing Delta Lake to skip reading files that do not match the query filters, thereby improving query performance.

**QUESTION 211**
The view **updates** represents an incremental batch of all newly ingested data to be inserted or updated in the **customers** table.

The following logic is used to process these records.

```
MERGE INTO customers
USING (
  SELECT updates.customer_id as merge_ey, updates.*
  FROM updates

  UNION ALL

  SELECT NULL as merge_key, updates.*
  FROM updates JOIN customers
  ON updates.customer_id = customers.customer_id
  WHERE customers.current = true AND updates.address <> customers.address
) staged_updates
ON customers.customer_id = mergeKey
WHEN MATCHED AND customers.current = true AND customers.address <> staged_updates.address THEN
  UPDATE SET current = false, end_date = staged_updates.effective_date
WHEN NOT MATCHED THEN
  INSERT(customer_id, address, current, effective_date, end_date)
  VALUES(staged_updates.customer_id, staged_updates.address, true, staged_updates.effective_date,
null)
```

Which statement describes this implementation?

A. The **`customers`** table is implemented as a Type 2 table; old values are overwritten and new customers are appended.
B. The **`customers`** table is implemented as a Type 2 table; old values are maintained but marked as no longer current and new values are inserted.
C. The **`customers`** table is implemented as a Type 0 table; all writes are append only with no changes to existing values.
D. The **`customers`** table is implemented as a Type 1 table; old values are overwritten by new values and no history is maintained.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
The provided `MERGE` statement is implementing a **Type 2 Slowly Changing Dimension (SCD)** table, where:
**Old values are updated to mark them as no longer current**: This is evident from the `WHEN MATCHED` clause, which updates records by setting `current = false` and setting an `end_date`, indicating that the existing record is no longer active.
**New values are inserted**: In the `WHEN NOT MATCHED` clause, the new record is inserted with `current = true` and no `end_date` (which is set to `null`), maintaining historical data by inserting a new row whenever there is a change.

**QUESTION 212**
A team of data engineers are adding tables to a DLT pipeline that contain repetitive expectations for many of the same data quality checks. One member of the team suggests reusing these data quality rules across all tables defined for this pipeline.

What approach would allow them to do this?

A. Add data quality constraints to tables in this pipeline using an external job with access to pipeline configuration files.
B. Use global Python variables to make expectations visible across DLT notebooks included in the same pipeline.
C. Maintain data quality rules in a separate Databricks notebook that each DLT notebook or file can import as a library.
D. Maintain data quality rules in a Delta table outside of this pipeline's target schema, providing the

schema name as a pipeline parameter.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**

**QUESTION 213**
The DevOps team has configured a production workload as a collection of notebooks scheduled to run daily using the Jobs UI. A new data engineering hire is onboarding to the team and has requested access to one of these notebooks to review the production logic.

What are the maximum notebook permissions that can be granted to the user without allowing accidental changes to production code or data?

A. Can manage
B. Can edit
C. Can run
D. Can read

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Granting **"Can read"** permission allows the user to **view** the notebook content without the ability to **edit** or **run** the notebook. This ensures that the new data engineering hire can **review** the production logic without the risk of **accidentally changing** production code or data. It is the safest level of access for onboarding purposes.

**QUESTION 214**
A table named **`user_ltv`** is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.

The **`user_ltv`** table has the following schema:

```
email STRING, age INT, ltv INT
```

The following view definition is executed:

```
CREATE VIEW email_ltv AS
SELECT
CASE WHEN
  is_member('marketing') THEN email
  ELSE 'REDACTED'
END AS email,
ltv
FROM user_ltv
```

An analyst who is not a member of the marketing group executes the following query:

```
SELECT * FROM email_ltv
```

Which statement describes the results returned by this query?

A. Three columns will be returned, but one column will be named "**REDACTED**" and contain only null values.
B. Only the **email** and **ltv** columns will be returned; the **email** column will contain all null values.
C. The **email** and **ltv** columns will be returned with the values in **user_ltv**.
D. Only the **email** and **ltv** columns will be returned; the **email** column will contain the string "**REDACTED**" in each row.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
The view `email_ltv` is defined with a CASE statement that checks if the user is a member of the **'marketing'** group using the `is_member()` function:
If the user **is** a member of the **'marketing'** group, the **email** value from the `user_ltv` table is returned.
Otherwise, the **email** value is set to `'REDACTED'`.

In the given scenario, the analyst **is not** a member of the **'marketing'** group. Therefore, when they query the `email_ltv` view, the **email** column will contain the value `'REDACTED'` for all rows, while the **ltv** column will be returned with its original values.

Only the columns **email** (with `'REDACTED'`) and **ltv** will be returned, so the correct result is that the **email** column will contain `'REDACTED'` in each row.

**QUESTION 215**
The data governance team has instituted a requirement that all tables containing Personal Identifiable Information (PII) must be clearly annotated. This includes adding column comments, table comments, and setting the custom table property **"contains_pii" = true**.

The following SQL DDL statement is executed to create a new table:

```
CREATE TABLE dev.pii_test
(id INT, name STRING COMMENT "PII")
COMMENT "Contains PII"
TBLPROPERTIES ('contains_pii' = True)
```

Which command allows manual confirmation that these three requirements have been met?

A. **DESCRIBE EXTENDED dev.pii_test**
B. **DESCRIBE DETAIL dev.pii_test**
C. **SHOW TBLPROPERTIES dev.pii_test**
D. **DESCRIBE HISTORY dev.pii_test**

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
The command **DESCRIBE EXTENDED** provides detailed metadata about the table, including:
**Column comments** for each column.
**Table comments** (e.g., the "Contains PII" comment).
**Table properties**, including custom properties like `'contains_pii' = True`.

Using `DESCRIBE EXTENDED` will allow you to manually confirm all of these annotations.

**QUESTION 216**
The data governance team is reviewing code used for deleting records for compliance with GDPR. They note the following logic is used to delete records from the Delta Lake table named **`users`**.

```
DELETE FROM users
WHERE user_id IN
   (SELECT user_id FROM delete_requests)
```

Assuming that **`user_id`** is a unique identifying key and that **`delete_requests`** contains all users that have requested deletion, which statement describes whether successfully executing the above logic guarantees that the records to be deleted are no longer accessible and why?

A. Yes; Delta Lake ACID guarantees provide assurance that the **`DELETE`** command succeeded fully and permanently purged these records.
B. No; files containing deleted records may still be accessible with time travel until a **`VACUUM`** command is used to remove invalidated data files.
C. Yes; the Delta cache **`immediately`** updates to reflect the latest data files recorded to disk.
D. No; the Delta Lake **`DELETE`** command only provides ACID guarantees when combined with the **`MERGE INTO`** command.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
In **Delta Lake**, even after successfully executing a **DELETE** command, the deleted records are not **physically removed** from storage immediately. Instead, Delta Lake keeps the previous versions of data files, allowing you to access them using **time travel**. To **permanently remove** the deleted data from disk and make it inaccessible, you need to run the **`VACUUM`** command, which removes files that have been invalidated by recent transactions. The VACUUM operation deletes these old versions, thus ensuring that the data is no longer accessible.

**QUESTION 217**
The data architect has decided that once data has been ingested from external sources into the Databricks Lakehouse, table access controls will be leveraged to manage permissions for all production tables and views.

The following logic was executed to grant privileges for interactive queries on a production database to the core engineering group.

```
GRANT USAGE ON DATABASE prod TO eng;
GRANT SELECT ON DATABASE prod TO eng;
```

Assuming these are the only privileges that have been granted to the eng group along with permission on the catalog, and that these users are not workspace administrators, which statement describes their privileges?

A. Group members are able to create, query, and modify all tables and views in the **`prod`** database, but cannot define custom functions.
B. Group members are able to list all tables in the **`prod`** database but are not able to see the results of any queries on those tables.
C. Group members are able to query and modify all tables and views in the **`prod`** database, but cannot create new tables or views.
D. Group members are able to query all tables and views in the **`prod`** database, but cannot create or edit anything in the database.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
**`GRANT USAGE ON DATABASE prod TO eng;`** - This grants **USAGE** on the database, which allows users to see the database and list its tables.
**`GRANT SELECT ON DATABASE prod TO eng;`** - This grants **SELECT** privileges on all tables and views within the `prod` database, allowing users to run queries against them.

These privileges allow users to **query** tables and views but do **not** grant permissions to **create, modify, or edit** tables, views, or any other objects in the database.

**QUESTION 218**
A user wants to use DLT expectations to validate that a derived table report contains all records from the source, included in the table validation_copy.

The user attempts and fails to accomplish this by adding an expectation to the report table definition.

```
CREATE LIVE TABLE report (
  CONSTRAINT no_missing_records EXPECT (key IN validation_copy)
)
AS SELECT <...>
```

Which approach would allow using DLT expectations to validate all expected records are present in this table?

A. Define a temporary table that performs a left outer join on validation_copy and report, and define an expectation that no report key values are null
B. Define a SQL UDF that performs a left outer join on two tables, and check if this returns null values for report key values in a DLT expectation for the report table
C. Define a view that performs a left outer join on validation_copy and report, and reference this view in DLT expectations for the report table
D. Define a function that performs a left outer join on validation_copy and report, and check against the result in a DLT expectation for the report table

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
To ensure that **all records** from the `validation_copy` source are present in the `report` table, you can create a **temporary table** that performs a **left outer join** between `validation_copy` and `report`. This join will help you identify if there are any records in `validation_copy` that are **missing** from `report`. You can then use a **Delta Live Tables (DLT) expectation** on this temporary table to ensure that there are **no null key values** from the `report`.

You could perform a left outer join and create a derived temporary table that contains any records from `validation_copy` that are missing in `report`.
Define a DLT expectation that ensures that no `key` values are null, which would mean all records in `validation_copy` exist in `report`.

**QUESTION 219**
A user new to Databricks is trying to troubleshoot long execution times for some pipeline logic they are working on. Presently, the user is executing code cell-by-cell, using **`display()`** calls to confirm code is producing the logically correct results as new transformations are added to an operation. To get a measure of average time to execute, the user is running each cell multiple times interactively.

Which adjustment will get a more accurate measure of how code is likely to perform in production and what is the limitation of this approach?

A. The Jobs UI should be leveraged to occasionally run the notebook as a job and track execution time during incremental code development because Photon can only be enabled on clusters launched for scheduled jobs.
B. The only way to meaningfully troubleshoot code execution times in development notebooks is to use production-sized data and production-sized clusters with Run All execution.
C. Production code development should only be done using an IDE; executing code against a local build of open source Spark and Delta Lake will provide the most accurate benchmarks for how code will perform in production.
D. Calling `display()` forces a job to trigger, while many transformations will only add to the logical query plan; because of caching, repeated execution of the same logic does not provide meaningful results.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
In Spark (including Databricks), many transformations like `select()`, `filter()`, and `groupBy()` are **lazy**, meaning they are added to the **logical query plan** but not immediately executed. Only **actions** (such as `display()`, `count()`, or `collect()`) actually trigger execution of the transformations. When the user repeatedly executes code with `display()` or other actions, **caching** can impact performance because data may already be in memory, which leads to faster execution times that do not represent the typical performance of the first run.

Thus, running the same cells multiple times to measure execution time does **not provide an accurate measure** of real-world performance. To get an accurate assessment, the user should consider **disabling caching**, running the entire pipeline (using **Run All**), or using a notebook in **Jobs** with cold data (i.e., uncached).

**QUESTION 220**
Where in the Spark UI can one diagnose a performance problem induced by not leveraging predicate push-down?

A. In the Executor's log file, by grepping for "predicate push-down"
B. In the Stage's Detail screen, in the Completed Stages table, by noting the size of data read from the Input column
C. In the Query Detail screen, by interpreting the Physical Plan
D. In the Delta Lake transaction log. by noting the column statistics

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
To diagnose a performance problem caused by **not leveraging predicate push-down**, you should look at the **Physical Plan** in the **Query Detail screen** of the **Spark UI**. The Physical Plan shows how Spark executes the query, and it will include information about whether **predicate push-down** was applied. Predicate push-down refers to the optimization where filtering operations are pushed down to the data source, reducing the amount of data read and improving performance. By examining the Physical Plan, you can see if predicates (filters) are being applied at the data source level or if a larger amount of data is being read and filtered in memory.

**QUESTION 221**
A data engineer needs to capture pipeline settings from an existing setting in the workspace, and use them to create and version a JSON file to create a new pipeline.

Which command should the data engineer enter in a web terminal configured with the Databricks CLI?

A. Use list pipelines to get the specs for all pipelines; get the pipeline spec from the returned results;

parse and use this to create a pipeline

B. Stop the existing pipeline; use the returned settings in a **`reset`** command

C. Use the **`get`** command to capture the settings for the existing pipeline; remove the **`pipeline_id`** and rename the pipeline; use this in a **`create`** command

D. Use the clone command to create a **`copy`** of an existing pipeline; use the get **`JSON`** command to get the pipeline definition; save this to **`git`**

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
To create a new pipeline using the settings of an existing one, the data engineer can:
Use the **`databricks pipelines get`** command to **capture** the settings of the existing pipeline.
**Remove** the `pipeline_id` (since it will be unique for each pipeline).
**Rename** the pipeline to differentiate it from the original one.
Use these settings with a **`databricks pipelines create`** command to create a new pipeline.

This process allows the engineer to **reuse** the existing pipeline configuration and make the necessary changes to avoid conflicts, effectively versioning the pipeline with a new identity.

**QUESTION 222**
Which Python variable contains a list of directories to be searched when trying to locate required modules?

A. importlib.resource_path

B. sys.path

C. os.path

D. pypi.path



**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
In Python, **`sys.path`** is a list of **directories** that the interpreter searches to locate the required modules. When you attempt to **import** a module, Python goes through the paths listed in `sys.path` to find the module. You can add directories to `sys.path` if you want Python to search additional locations.

**QUESTION 223**
You are testing a collection of mathematical functions, one of which calculates the area under a curve as described by another function.

```
assert(myIntegrate(lambda x: x*x, 0, 3) [0] == 9)
```

Which kind of test does the above line exemplify?

A. Unit

B. Manual

C. Functional

D. Integration

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
The provided line is an example of a **unit test**. A **unit test** is used to verify that a small, isolated part of your code (often a single function or method) behaves as expected. In this case, the `assert` statement checks if the output of the function `myIntegrate(lambda x: x*x, 0, 3)` is equal to `9`. This test is

designed to validate the correctness of the function for a specific input, which is characteristic of unit testing.

**QUESTION 224**
What is a key benefit of an end-to-end test?

A. It makes it easier to automate your test suite
B. It pinpoints errors in the building blocks of your application
C. It provides testing coverage for all code paths and branches
D. It closely simulates real world usage of your application

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
**End-to-end (E2E) testing** is designed to **simulate real-world scenarios** and verify that the entire application functions correctly from start to finish. It tests the application flow as a whole, ensuring that all integrated components work together as expected, which helps to identify issues that could occur in an actual user environment. This type of testing provides confidence that the application will behave correctly for users.

**QUESTION 225**
Which REST API call can be used to review the notebooks configured to run as tasks in a multi-task job?

A. `/jobs/runs/list`
B. `/jobs/list`
C. `/jobs/runs/get`
D. `/jobs/get`



**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
The **`/jobs/get`** REST API call is used to **retrieve metadata** about a specific job, including the **notebooks** configured as tasks in a **multi-task job**. This API call provides detailed information about the configuration of the job, including task settings, which would include any notebooks that are set up to run as part of that job.

**QUESTION 226**
A Data Engineer wants to run unit tests using common Python testing frameworks on Python functions defined across several Databricks notebooks currently used in production.

How can the data engineer run unit tests against functions that work with data in production?

A. Define and import unit test functions from a separate Databricks notebook
B. Define and unit test functions using **`Files`** in Repos
C. Run unit tests against non-production data that closely mirrors production
D. Define unit tests and functions within the same notebook

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
To run **unit tests** on Python functions across several Databricks notebooks, the **best practice** is to use **Files in Repos**. By storing the functions in reusable Python files (`.py`) within Databricks **Repos**, the Data Engineer can import these functions into different notebooks and easily manage **unit testing** using Python testing frameworks like `unittest` or `pytest`. This approach supports **modular code**, **version control**,

and makes it easier to maintain and test the functions.

With Files in Repos:
The functions can be organized and **imported** where needed.
The testing code can be separated from the production code.
Common **Python testing frameworks** can be easily used to write and execute tests.

**QUESTION 227**
A data engineer wants to refactor the following **DLT** code, which includes multiple table definitions with very similar code.

```
@dlt.table(name=f"t1_dataset")
def t1_dataset():
    return spark.read.table(t1)

@dlt.table(name=f"t2_dataset")
def t2_dataset():
    return spark.read.table(t2)

@dlt.table(name=f"t3_dataset")
def t3_dataset():
    return spark.read.table(t3)

...
```

In an attempt to programmatically create these tables using a parameterized table definition, the data engineer writes the following code.

```
tables = ["t1", "t2", "t3"]

for t in tables:
    @dlt.table(name=f"{t}_dataset")
        def new_table():
            return spark.read.table(t)
```

The pipeline runs an update with this refactored code, but generates a different **DAG** showing incorrect configuration values for these tables.

How can the data engineer fix this?

A. Wrap the for loop inside another table definition, using generalized names and properties to replace with those from the inner table definition.
B. Convert the list of configuration values to a dictionary of table settings, using table names as keys.
C. Move the table definition into a separate function, and make calls to this function using different input parameters inside the for loop.
D. Load the configuration values for these tables from a separate file, located at a path provided by a pipeline parameter.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
In the provided refactored code, the for loop dynamically attempts to define multiple tables, but the use of a **loop** within the **DLT (@dlt.table)** decorator does not work properly because it results in a single function reference being overwritten for each iteration. This leads to an incorrect DAG because all the table definitions end up pointing to the **last iteration** of the loop.

**QUESTION 228**
A data engineer has created a 'transactions' Delta table on Databricks that should be used by the analytics team. The analytics team wants to use the table with another tool which requires Apache Iceberg format.

What should the data engineer do?

A. Require the analytics team to use a tool which supports Delta table.
B. Create an Iceberg copy of the 'transactions' Delta table which can be used by the analytics team.
C. Convert the 'transactions' Delta to Iceberg and enable uniform so that the table can be read as a Delta table.
D. Enable uniform on the transactions table to 'iceberg' so that the table can be read as an Iceberg table.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
The analytics team requires the Iceberg format, which is different from Delta. The data engineer should create a separate Iceberg copy of the transactions Delta table so it can be used with tools that support Iceberg without affecting the original Delta table. This approach preserves the Delta table for existing workflows while providing Iceberg access for the analytics team.

**QUESTION 229**
A junior data engineer is working to implement logic for a Lakehouse table named `silver_device_recordings`. The source data contains 100 unique fields in a highly nested JSON structure.

The `silver_device_recordings` table will be used downstream for highly selective joins on a number of fields, and will also be leveraged by the machine learning team to filter on a handful of relevant fields. In total, 15 fields have been identified that will often be used for filter and join logic.

The data engineer is trying to determine the best approach for dealing with these nested fields before declaring the table schema.

Which of the following accurately presents information about Delta Lake and Databricks that may impact their decision-making process?

A. Because Delta Lake uses Parquet for data storage, Dremel encoding information for nesting can be directly referenced by the Delta transaction log.
B. Schema inference and evolution on Databricks ensure that inferred types will always accurately match the data types used by downstream systems.
C. The Tungsten encoding used by Databricks is optimized for storing string data; newly-added native support for querying JSON strings means that string types are always most efficient.
D. By default, Delta Lake collects statistics on the first 32 columns in a table; these statistics are leveraged for data skipping when executing selective queries.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Delta Lake automatically collects column-level statistics (min, max, null count) for the first 32 columns by default. These statistics enable data skipping, which improves performance for selective queries and joins. This is highly relevant when deciding which fields to flatten or index in a Lakehouse table, as selective

fields benefit most from data skipping.

**QUESTION 230**
A platform engineer is creating catalogs and schemas for the development team to use.

The engineer has created an initial catalog, Catalog_A, and initial schema, Schema_A. The engineer has also granted `USE CATALOG`, `USE SCHEMA`, and `CREATE TABLE` to the development team so that the engineer can begin populating the schema with new tables.

Despite being owner of the catalog and schema, the engineer noticed that they do not have access to the underlying tables in `Schema_A`.

What explains the engineer's lack of access to the underlying tables?

A. The owner of the schema does not automatically have permission to tables within the schema, but can grant them to themselves at any point.
B. Users granted with `USE CATALOG` can modify the owner's permissions to downstream tables.
C. Permissions explicitly given by the table creator are the only way the Platform Engineer could access the underlying tables in their schema.
D. The platform engineer needs to execute a `REFRESH` statement as the table permissions did not automatically update for owners.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
In Databricks, owning a catalog or schema does not automatically grant access to the tables within it. Table-level permissions are separate, so even the schema or catalog owner must be explicitly granted privileges on individual tables or use the ability to grant themselves access.

**QUESTION 231**
A data engineer has created a new cluster using shared access mode with default configurations. The data engineer needs to allow the development team access to view the driver logs if needed.

What are the minimal cluster permissions that allow the development team to accomplish this?

A. CAN VIEW
B. CAN RESTART
C. CAN ATTACH TO
D. CAN MANAGE

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
The CAN VIEW permission on a cluster allows users to see cluster details, including driver and executor logs. This is the minimal permission required for the development team to access logs without granting them the ability to modify, restart, or attach notebooks to the cluster.

**QUESTION 232**
A data engineer wants to create a cluster using the Databricks CLI for a big ETL pipeline. The cluster should have five workers and one driver of type i3.xlarge and should use the `'14.3.x-scala2.12'` runtime.

Which command should the data engineer use?

A. databricks compute add `14.3.x-scala2.12 --num-workers 5 --node-type-id i3.xlarge --cluster-name Data Engineer_cluster`
B. databricks clusters create `14.3.x-scala2.12 --num-workers 5 --node-type-id`

```
i3.xlarge --cluster-name Data Engineer_cluster
```

C. databricks compute create `14.3.x-scala2.12 --num-workers 5 --node-type-id i3.xlarge --cluster-name Data Engineer_cluster`

D. databricks clusters add `14.3.x-scala2.12 --num-workers 5 --node-type-id i3.xlarge --cluster-name Data Engineer_cluster`

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
The correct Databricks CLI command to create a new cluster is databricks clusters create. You specify the runtime with --spark-version (here '14.3.x-scala2.12'), the number of workers with --num-workers, the node type with --node-type-id, and the cluster name with --cluster-name. This command properly initializes the cluster with the desired configuration.

**QUESTION 233**
A `'transactions'` table has been liquid clustered on the columns `'product_id', 'user_id'` and `'event_date'`.

Which operation lacks support for cluster on write?

A. `CTAS and RTAS statements`
B. `spark.writeStream.format('delta').mode('append')`
C. `spark.write.format('delta').mode('append')`
D. `INSERT INTO operations`

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
In Delta Lake, INSERT INTO operations do not support “clustered on write.” Clustered write is supported when creating or overwriting a table with CTAS/RTAS statements or when writing via Spark with .write or .writeStream in append mode. INSERT INTO only appends data without reorganizing it according to the clustering columns.

**QUESTION 234**
The data governance team has instituted a requirement that the "user" table containing Personal Identifiable Information (PII) must have the appropriate masking on the SSN column. This means that anyone outside of the `HRAdminGroup` should see masked social security numbers as ***-**-****.

The team created a masking function:

```
CREATE FUNCTION ssn_mask(ssn STRING)
  RETURN CASE WHEN is_member(HRAdminGroup) THEN ssn ELSE '***-**-****' END;
```

What does the data governance team need to do next to achieve this goal?

A.
```
CREATE TABLE users
(name STRING);
ALTER TABLE users CREATE COLUMN ssn CREATE MASK ssn_mask;
```

B.
```
CREATE TABLE users
(name STRING, int STRING);
ALTER TABLE users ALTER COLUMN ssn CREATE MASK if is_member('HRAdminGroup');
```

C.
```
CREATE TABLE users
(name STRING, ssn INT MASKED ssn_mask);
```

D.
```
CREATE TABLE users
(name STRING, ssn STRING);
ALTER TABLE users ALTER COLUMN ssn SET MASK ssn_mask;
```

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
In Databricks, after creating a masking function, you apply it to a column using ALTER TABLE <table> ALTER COLUMN <column> SET MASK <mask_function>. The table must already include the column (here, ssn as STRING). This ensures that only users in the HRAdminGroup see the unmasked SSN, while all others see the masked value.

**QUESTION 235**
A data engineer needs to create an application that will collect information about the latest job run including the repair history.

How should the data engineer format the request?

A. Call `/api/2.1/jobs/runs/list` with the `run_id` and `include_history` parameters
B. Call `/api/2.1/jobs/runs/get` with the `run_id` and `include_history` parameters
C. Call `/api/2.1/jobs/runs/get` with the `job_id` and `include_history` parameters
D. Call `/api/2.1/jobs/runs/list` with the `job_id` and `include_history` parameters

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
To retrieve information about the latest job runs along with their repair history, you use the jobs/runs/list endpoint with the job_id and include_history=true. This endpoint returns a list of runs for a specific job, including details about retries and repair attempts, which is not available via runs/get that retrieves a single run by run_id.

**QUESTION 236**
A data engineer is working in an interactive notebook with many transformations before outputting the result from `display(df.collect() )`. The notebook includes wide transformations and a cross join.

The data engineer is getting the following error: "The spark driver has stopped unexpectedly and is restarting. Your notebook will be automatically reattached."

Which action should the data engineer take?

A. Run the notebook on a single node cluster to keep driver from falling.
B. Rewrite their code to avoid putting memory pressure on the driver node.
C. Check into the Spark UI to see how many jobs are assigned to each stage as they are employing fewer executors.
D. Look at the compute metrics UI to see if the executors have higher than 90% memory utilization.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
Calling df.collect() on a large DataFrame forces all data to be loaded into the driver’s memory. With wide transformations and a cross join, this can easily exceed the driver’s capacity, causing it to crash. The data engineer should rewrite the code to avoid collecting large datasets on the driver, using operations like display(df) or writing to storage instead.

**QUESTION 237**
An analytics team wants run an experiment in the short term on the customer transaction Delta table (with 20 billions records) created by the data engineering team in Databricks SQL.

Which strategy should the data engineering team use to ensure minimal downtime and no impact on the ongoing ETL processes?

A. Deep clone the table for the analytics team.
B. Create a new table for the analytics team using a CTAS statement.
C. Shallow clone the table for the analytics team.
D. Give access to the table for the analytics team.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
A shallow clone creates a new Delta table that references the same underlying data files as the original table. This allows the analytics team to query and experiment on the data without copying 20 billion records, ensuring minimal downtime and no impact on ongoing ETL processes. Changes made to the clone do not affect the original table's metadata or operations.

**QUESTION 238**
A data team is working to optimize an existing large, fast-growing table 'orders' with high cardinality columns, which experiences significant data skew and requires frequent concurrent writes. The team notice that the columns `'user_id'`, `'event_timestamp'` and `'product_id'` are heavily used in analytical queries and filters, although those keys may be subject to change in the future due to different business requirements.

Which partitioning strategy should the team choose to optimize the table for immediate data skipping, incremental management over time, and flexibility?

A. Partition the table with: `ALTER TABLE` orders `PARTITION BY user_id, product_id, event_timestamp`
B. Use z-order after partitiing the table: `OPTIMIZE` orders `ZORDER BY (user_id, product_id) WHERE event_timestamp = current date () - 1 DAY`
C. Cluster the table with: `ALTER TABLE` orders `CLUSTER BY user_id, product_id, event_timestamp`
D. Z-order the table with `OPTIMIZE` orders `ZORDER BY (user_id, product_id, event_timestamp)`

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Z-ordering optimizes data skipping for selective queries on high-cardinality columns without physically repartitioning the table, making it flexible if query patterns change. Using OPTIMIZE ... ZORDER BY (user_id, product_id, event_timestamp) improves query performance for filters and joins while allowing incremental writes, avoiding the data skew and maintenance overhead that explicit partitioning or clustering could introduce.

**QUESTION 239**
A faulty IoT sensor in a factory reports a temperature of -500, causing the LDP pipeline to fail the expectation, which only allows values between -100 and 200 degrees Celsius. The data engineer would like to further analyze the faulty data to better understand the reason behind this.

How should the data engineer resolve the faulty data while ensuring data quality standards are maintained?

A. Remove all expectations form the pipeline to prevent any future failures, regardless of data quality.
B. Ignore the error and simply re-run the pipeline, as Databricks will automatically skip the problematic record on the next run.
C. Fix the pipeline code and implement a quarantine logic to isolate the faulty data before re-running the

pipeline.

D. Change the expectation action from fail to warn so that invalid records are included in the output and the pipeline does not fail.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Implementing quarantine logic allows invalid records to be isolated for further analysis while keeping them out of trusted datasets. Fixing the pipeline and re-running it ensures the expectation continues to enforce data quality standards, prevents future failures, and enables root-cause analysis of the faulty sensor data without compromising downstream consumers.

**QUESTION 240**
A data engineer is optimizing a managed table that suffers from data skew and frequently changing query filter columns. The engineer needs to avoid costly data rewrites when query patterns evolve. The table size is under 1TB.

How should data engineer meet this requirement?

A. Use Hive-style partitioning, as it provides efficient data skipping and is easy to change partition columns at any time.
B. Combine partitioning and Z-ordering to maximize flexibility and minimize maintenance as query patterns change.
C. Enable liquid clustering, as it efficiently handles data skew, allows clustering keys to be changed without rewriting existing data, and adapts to evolving query patterns.
D. Apply Z-ordering, since it allows flexible reorganization of data layout without rewriting existing and adapts easily to new filter columns.



**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Liquid clustering is designed for managed tables under 1TB with evolving query patterns. It efficiently addresses data skew, continuously optimizes data layout, and allows clustering keys to be changed without requiring full data rewrites, making it well suited for frequently changing filter columns while minimizing maintenance overhead.

**QUESTION 241**
A security team wants to enforce data protection for a customer table containing customer PII data. To comply with local policies, sales team members should only see customers from their region, while non-admin users should have email addresses masked.

Which implementation approach should be used when using Unity Catalog row filters and column masks?

A. Create SQL UDFs for row filtering based on user region and column masking based on group membership, then apply them using ALTER TABLE SET ROW FILTER and ALTER COLUMN SET MASK commands.
B. Use table ACLs to restrict access using tags with GRANT SELECT ON table_name WITH TAG command, and rely on application-level filtering for sensitive data based on user region.
C. Create a view with dynamic WHERE clauses for region filtering and use string replacement functions for email masking using ALTER COLUMN SET MASK command.
D. Implement row filters with SQL UDFs based on user region only since column masks cannot be combined with row filters on the same table, then apply them be recreating the table with DROP TABLE and CREATE TABLE SET ROW FILTER commands.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Unity Catalog enforces fine-grained access control by applying SQL UDF–based row filters and column masks directly at the table level. Row filter UDFs can restrict visible rows based on the user's region, while column mask UDFs can dynamically mask sensitive fields like email addresses for non-admin users. Applying them with ALTER TABLE SET ROW FILTER and ALTER COLUMN SET MASK ensures centralized, consistent enforcement of data protection policies across all access paths.

**QUESTION 242**
A data engineer is evaluating tools to build a production-grade data pipeline. The team must process change data from cloud object storage, filter out or isolate invalid records, and ensure the timely delivery of clean data to downstream consumers. The team is small, under tight deadlines, and wants to minimize operational overhead while keeping pipelines auditable and maintainable.

Which approach should the data engineer implement?

A. Ingest data directly into Delta tables via Spark jobs, apply data quality filters using UDFs, and use LDP for creating Materialized Views.
B. Use a hybrid approach: Ingest with Auto Loader into Bronze tables, then process using SQL queries in Databricks Workflows to generate cleaned Silver and Gold tables on a schedule.
C. Implement ingestion using Auto Loader with Structured Streaming, and manage invalid data handling and table updates using checkpointing and merge logic.
D. Use LDP to build declarative pipelines with Streaming Tables and Materialized Views, leveraging built-in support for data expectations and incremental processing.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
LDP provides a declarative framework for building production-grade pipelines with minimal operational overhead. Streaming Tables and Materialized Views handle incremental processing automatically, while built-in data expectations allow invalid records to be filtered or isolated in a consistent and auditable way. This approach is well suited for small teams under tight deadlines, as it simplifies maintenance, improves reliability, and ensures timely delivery of clean data to downstream consumers.

**QUESTION 243**
A data engineer is using Structured Streaming to read in transaction data from a bronze Delta table. It was discovered that the data has quality issues where sometimes the transaction value is negative, and when that occurs, the rows need to be routed to a separate quarantine table. They have low latency requirements for the good data since it is used by downstream systems, but the bad data will only be analyzed periodically and has no production dependencies. The quarantine job needs to be implemented so that it cannot affect the production processes that depend on the good data, and the cost of the job needs to be minimized.

How should the quarantine process be implemented in order to satisfy these requirements?

A. The streaming job for the good data needs to be modified to filter out records with a transaction value less than 0 before writing. The streaming job for the quarantine data needs to filter out records with a transaction value greater than or equal to 0 before writing. Both should run as separate streams on the same cluster to minimize cost.
B. The existing streaming job for the good data should be updated to incorporate the quarantining of the bad data. A new boolean column called "quarantine" should be added to the dataframe, and its value should be set to true if the transaction value is less than 0 and false if the transaction value is greater than or equal to 0. Processing and storing all the data together will save costs.
C. The existing streaming job for the good data should be updated to incorporate the quarantining of the bad data. Inside a foreachBatch function, the dataframe should be filtered so that records with a transaction value greater than or equal to 0 are written to the good data table and records with a transaction value less than 0 are written to a quarantine table. Try/Catch can be added around the writes in the foreachBatch function so that the stream can't fail.
D. The streaming job for the good data needs to be modified to filter out records with a transaction value

less than 0 before writing, and should not share compute with other processes. The streaming job for the quarantine data needs to filter out records with a transaction value greater than or equal to 0 before writing, and should be implemented on a separate small cluster and only run once a day to minimize cost.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Separating the quarantine processing from the low-latency production stream ensures the good-data pipeline cannot be slowed down or destabilized by quarantine writes or failures. Filtering out negative values in the production stream preserves latency and reliability for downstream systems. Running the quarantine stream on a small, independent cluster on a daily schedule minimizes cost while still capturing the bad records for periodic analysis, and it fully isolates quarantine operations from production dependencies.

**QUESTION 244**
When monitoring a complex workload, being able to see the query plan is critical to understanding what the workload is doing.

Where can the visualization of the query plan be found?

A. In the Spart UI, under the Jobs tab
B. In the Query Profiler, under Query Source
C. In the Spark UI, under the SQL/DataFrame tab
D. In the Query Profiler, under the Stages tab

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
The Spark UI provides detailed visibility into how queries are executed. The SQL/DataFrame tab displays the logical and physical query plans, allowing engineers to visualize execution details and understand how the workload is processed across stages and operators.

**QUESTION 245**
A data engineer is troubleshooting a slow-running Delta Lake query on Databricks SQL involves complex joins and large datasets. They need to identify whether the root cause is related to poor data skipping, inefficient join strategies, or excessive data shuffling.

Which approach should identify the specific bottlenecks using native Databricks tools?

A. Analyze the Top Operators panel in the Query Profile to identify high-cost operations like BroadcastNestedLoopJoin
B. Check the query's execution time in the Jobs UI and correlate it with cluster resource utilization metrics.
C. Enable the EXPLAIN command to review the parsed logical plan and manually estimate shuffle sizes.
D. Use the LIMIT clause to run a subset of the query and compare execution times with the full dataset.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
The Query Profile's Top Operators panel surfaces the most expensive operators in the query execution, making it possible to directly identify bottlenecks such as inefficient join strategies, poor data skipping, or excessive shuffling. This native visualization highlights where time and resources are spent, enabling precise root-cause analysis for slow-running queries.

**QUESTION 246**

A data engineer is designing a secure data sharing strategy for their organization. The company needs to share sensitive customer analytics data with two different partners. Partner A uses Databricks with Unity Catalog enabled, while Partner B uses Apache Spark on AWS without Databricks.

How should the company implement secure data sharing for these scenarios?

A. For Partner A, implement Databricks-to-Databricks sharing (D2D) with Unit Catalog integration and no-token exchange system. For Partner B, use open sharing protocol (D2O) with either bearer tokens or OIDC federation for authentication, ensuring both approaches maintain robust security and governance.
B. Both partners should use the same Delta Sharing approach since security requirements are identical. You should create bearer tokens for both partners and use the open sharing protocol (D2O) for maximum compatibility.
C. Open sharing protocol (D2O) should be used for both partners because it provides better security than D2D sharing. The bearer token approach is always more secure than Unity Catalog's native authentication.
D. Databricks-to-Databricks sharing (D2D) can only be used within the same cloud provider, so you must use open sharing (D2O) for any cross-cloud scenarios. Unit Catalog governance is not available when sharing with external platforms.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Databricks-to-Databricks sharing with Unity Catalog provides the most seamless and secure option for Partner A by enabling native governance, fine-grained access controls, and a no-token exchange model. For Partner B, which does not use Databricks, the open sharing protocol enables secure access from external Spark environments using standard authentication mechanisms such as bearer tokens or OIDC federation, while still enforcing sharing policies and protecting sensitive data.

**QUESTION 247**
Which approach demonstrates a modular and testable way to use DataFrame transform for ETL code in PySpark?

A.
```
class Pipeline:
   def transform(self, df):
     return df.withColumn("value_upper", upper(col("value")))

pipeline = Pipeline()
assertDataFrameEqual (pipeline.transform(test_input), expected)
```

B.
```
def transform_data(input_df):
   # transformation logic here
   return output_df

test_input = spark.createDataFrame ([(1, "a")], ["id", "value"])
assertDataFrameEqual (transform_data (test_input), expected)
```

C.
```
def upper_transform(df):
   return df.withColumn("value_upper", upper(col("value")))

actual = test_input.transform(upper_transform)
assertDataFrameEqual (actual, expected)
```

D.

```
def upper_value (df):
   return df.withColumn("value_upper", upper(col("value")))

def filter_positive(df):
   return df.filter (df["id"] > 0)

pipeline_df = df.transform(upper_value).transform
(filter_positive)
```

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Using DataFrame.transform with a pure transformation function promotes modular, reusable, and easily testable ETL logic. Each transformation is encapsulated as a standalone function, can be independently unit tested, and composed cleanly in a pipeline without coupling to orchestration or class state.

**QUESTION 248**
A company stores account transactions in a Delta Lake table. The company needs to apply frequent account-level correlations (e.g., UPDATE statements) but wants to avoid rewriting entire Parquet files for each change to reduce file churn and improve write performance.

Which Delta Lake feature should they enable?

A. Enable automatic file compaction on writes
B. Enable change data feed on the Delta table
C. Partition the Delta table by `account_id`
D. Enable deletion vectors on the Delta table



**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Deletion vectors allow Delta Lake to track row-level deletes and updates without rewriting entire Parquet files. By recording changes separately from the base files, this feature significantly reduces file churn and improves write performance for workloads with frequent row-level modifications such as account-level updates.

**QUESTION 249**
In a Databricks Asset Bundle project, in the file `resources/app.yml`, the data engineer would like to deploy a Databricks Apps `databricks_app_deployed` and Volume `volume_deployed` and grant the Service Principal behind Databricks Apps permissions to `READ` and `WRITE` to the Volume.

How should the data engineer achieve the deployment?

A.

```
resources:
  apps:
    databricks_app_deployed:
      name: "name-databricks_app"
      description: 'Databricks App'
      source_code_path: ../src/databricks_app_code

volumes:
  volume_deployed:
    name: name_volume
    catalog_name: name_catalog
    schema_name: name_schema
    grants:
      -principal: ${resources.app.databricks_app_deployed.id}
      privileges:
       -READ_VOLUME
       -WRITE_VOLUME
```

B.

```
resources:
  apps:
    databricks_app_deployed:
      name: "name-databricks_app"
      description: 'Databricks App'
      source_code_path: ../src/databricks_app_code

volumes:
  volume_deployed:
  name: name_volume
  catalog_name: name_catalog
  schema_name: name_schema
  grants:
    -principal: ${resources.app.name-databricks_app.id}
    privileges:
      -READ_VOLUME
      -WRITE_VOLUME
```

C.
```
resources:
  apps:
    databricks_app_deployed:
      name: "name-databricks_app"
      description: 'Databricks App'
      source_code_path: ../src/databricks_app_code

volumes:
  volume_deployed:
    name: name_volume
    catalog_name: name_catalog
    schema_name: name_scheama
    grants:
      -principal: ${resources.app.name-
databricks_app_deployed.id}
      privileges:
       -READ
       -WRITE
```

D.
```
resources:
  apps:
    databricks_app_deployed:
     name: "name-databricks_app"
     description: 'Databricks App'
     source_code_path: ../src/databricks_app_code

volumes:
  volume_deployed:
    name: name_volume
    catalog_name: name_catalog
    schema_name: name schema
  grants:
   -principal: ${resources.app.name-name-databricks_app.id}
   privileges:
    -READ_VOLUME
    -WRITE_VOLUME
```

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
This configuration correctly references the service principal created for the Databricks App using the deployed app resource identifier, and it grants the required READ and WRITE privileges at the Volume level. The privileges are specified using the correct Volume-specific permissions, ensuring the Databricks App can securely access the Volume after deployment.

**QUESTION 250**
A data engineering teams needs to implement a tagging system for their tables as part of an automated ETL process, and needs to apply tags programmatically to tables in Unity Catalog.

Which SQL command adds tags to a table programmatically?

A. `ALTER TABLE table_name SET TAGS ('key1' = 'value1', 'key2' = 'value2');`
B. `APPLY TAGS ON table_name VALUES ('key1' = 'value1', 'key2' = 'value2')`
C. `COMMENT ON TABLE table_name TAGS ('key1' = 'value1', 'key2' = 'value2')`
D. `SET TAGS FOR table_name AS ('key1' = 'value1', 'key2' = 'value2')`

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Unity Catalog supports programmatic tagging through the ALTER TABLE statement. Using SET TAGS allows tags to be added or updated directly on a table as part of automated ETL workflows, making it the correct and supported SQL command for managing table tags.

**QUESTION 251**
A platform engineer needs to report the resource consumption, categorized by SKU tier, across all workspaces. The engineer decides to use the `system.billing.usage` system table to create a query.

Which SQL query will accurately return the daily usage by product?

A.
```
SELECT
   DATE_TRUNC('day', usage_start_time) AS day,
   sku_name,
   SUM(dbus) AS daily_dbus
FROM
   system.billing.usage
WHERE
   sku_name IS NOT NULL
GROUP BY
   day, sku_name;
```

B.
```
SELECT
  CAST(usage_start_time AS DATE) AS usage_day,
  sku_name,
  COUNT (usage_quantity) AS dbu_count
FROM
  system, billing.usage
GROUP BY
  usage_day, sku_name
ORDER BY
  usage_day;
```

C.
```
SELECT
  date_trunc('day', usage_start_time) AS usage_day,
  sku_name,
  SUM(usage_quantity) AS total_daily_dbus
FROM
  system.billing.usage
WHERE
  usage_quantity IS NOT NULL
GROUP BY
  usage_day, sku_name
ORDER BY
  usage_day;
```

D.
```
SELECT
  TO_DATE(usage_start_time)AS usage_day,
  sku_name,
  SUM(usage_quantity) AS total_dbus
FROM
  system.billing.usage
GROUP BY
  usage_day
ORDER BY
  usage_day
```



**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
This query correctly aggregates usage at a daily granularity by truncating the usage start timestamp to the day and summing the usage quantity, which represents DBUs. Grouping by both the derived daily value and the SKU name ensures usage is accurately categorized by product tier across all workspaces.

**QUESTION 252**
A data engineer is running a groupBy aggregation on a massive user activity log grouped by `user_id`. A few users have millions of records, causing task skew and long runtimes.

Which technique will fix the skew in this aggregation?

A. Increase the Spark driver memory and retry.
B. Filter out the skewed users before the aggregation
C. Use salting by adding a random prefix to skewed keys before aggregation, then aggregate again after removing the prefix.
D. Use reduceByKey instead of groupBy to avoid shuffles.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Salting distributes records for heavily skewed keys across multiple partitions by adding a random prefix,

which balances task execution during the aggregation. A second aggregation after removing the prefix correctly recombines the partial results, eliminating skew-related bottlenecks without losing accuracy.

**QUESTION 253**
A job runs four independent tasks (X, Y, Z, W) in parallel to process regional sales data. The Data Engineering team recently updated its cluster policy to ban cost-prohibitive instance types. Task Y now fails due to the newly enforced cluster policy restricting the use of a specific instance type. A data engineer needs to resolve the failure quickly without disrupting the other tasks.

How should the data engineer resolve the failure of tasks?

A. Delete the failed run, disable the cluster policy, and re-execute all tasks.
B. Manually create a new cluster for Task Y, update the job configuration, and trigger a full re-run.
C. Use “Repair run”, override the cluster configuration for Task Y to use a permitted instance type, and let Databricks re-run only Task Y.
D. Edit the global cluster policy to allow the restricted instance type, then re-run the entire job.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Repair run allows re-running only the failed task without affecting successfully completed tasks. Overriding the cluster configuration for the specific task resolves the policy violation quickly while keeping the rest of the job intact and minimizing disruption.

**QUESTION 254**
A data engineer is analyzing a large, partitioned retail dataset in Databricks, where each row represents a sale made by a salesperson. The dataset contains millions of records with the following schema:

```
sales_df: [salesperson_id: string, region: string, sale_amount: double,
sale_date: date]
```

The data engineer needs to generate a DataFrame that ranks salespeople within each region based on their total cumulative sales, with the highest seller ranked as 1. If multiple salespeople have the same total sales, they should share the same rank.

The data engineer wants to implement this logic using a PySpark window function and the dense_rank () function.

Which code snippet will perform this ranking?

A.
```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

window_spec = Window.partitionBy(“region”).orderBy
(F.desc(“sale_amount”))

ranked_df= sales_df.withColumn(“rank”,
F.dense_rank().over(window_spec))
```

B.

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

agg_df= sales_df.groupBY("salesperson_id", "region") \
   .agg(F.sum("sale_amount").alias("total_sales"))

window_spec = Window.partitionBy("region").orderBy
(F.desc("total_sales"))

ranked_df= agg_df.withColumn("rank", F.rank().over(window_spec))
```

C.

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

agg_df= sales_df.groupBY("salesperson_id", "region") \
   .agg(F.sum("sale_amount").alias("total_sales"))

window_spec = Window.orderBy (F.desc("total_sales"))

ranked_df= agg_df.withColumn("rank",
F.dense_rank().over(window_spec))
```

D.

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

agg_df= sales_df.groupBY("salesperson_id", "region") \
   .agg(F.sum("sale_amount").alias("total_sales"))

window_spec =
Window.parititionBy("region").orderBy(F.desc("total_sales"))

ranked_df=agg_df.withColumn("rank",
F.dense_rank().over(window_spec))
```

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
This approach first aggregates sales by salesperson and region to compute total cumulative sales. It then applies a window function partitioned by region and ordered by total sales in descending order, using dense_rank to assign ranks so that salespeople with equal totals share the same rank and the highest total receives rank 1.

**QUESTION 255**
What describes a primary technical challenge in ensuring consistent PII masking across all nodes in large-scale, distributed Databricks batch and streaming pipelines?

A. PII masking is only required for direct identifiers.
B. Dynamic data masking is applied only at rest, so it does not affect query performance.

C. Masking functions must be standardized and managed through Unity Catalog, with enforcement applied across all relevant datasets to avoid any data inconsistency.
D. Native masking in Databricks automatically synchronizes with all downstream external Databricks systems.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Consistent PII masking in distributed batch and streaming pipelines requires centrally defined and governed masking logic. Standardizing masking functions and enforcing them through Unity Catalog ensures that the same rules are applied uniformly across all datasets and execution paths, preventing inconsistencies across nodes and workloads.

**QUESTION 256**
A data engineer is working on a Databricks notebook that requires several third-party Python libraries. Some of these are available on PyPI, while others are custom-developed and stored as local.wheel (.whl) and source (.tar.gz) files in an S3 bucket. The goal is to ensure all dependencies are installed and correctly available across multiple jobs running on any automated cluster in a Unity Catalog-enabled workspace. The engineer needs to install the required dependencies in a way that ensures a consistent environment setup across interactive notebooks and jobs and complies with workspace security policies (no internet access).

Which approach should the engineer use to install and manage these dependencies while also ensuring reproducibility and compliance?

A. Use an init script on the cluster to install all dependencies using pip, referencing the local file system.
B. Install all dependencies manually in the driver node of an interactive cluster, then export the environment and reimport on job clusters using %conda.
C. Create a Python wheel file for the entire project, upload it to the Databricks Workspace Files or Volumes, and install it using a Cluster Library or pip install in a requirements.txt declared within a Databricks Asset Bundle.
D. Use %pip install in every notebook and job to install packages directly from PyPI and custom S3 paths.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Packaging dependencies as wheel files and managing them through Databricks-supported artifacts ensures consistent, reproducible environments across interactive and job clusters. Storing the wheels in Workspace Files or Unity Catalog Volumes complies with no-internet security constraints, while installing them via cluster libraries or Asset Bundles standardizes dependency management across all jobs and pipelines.

**QUESTION 257**
A data engineer is using Auto Loader to read in JSON data as it arrives. They have configured Auto Loader to quarantine invalid JSON records. They are noticing that over time, some records are being quarantined even though they are well-formed JSON.

The snippet of code is:

```
df = (spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("badRecordsPath", "/tmp/somewhere/badRecordsPath")
  .schema("a int, b int")
  .load("/Volumes/catalog/schema/raw_data/"))
```

What is the cause of the missing data?

A. The source data is valid JSON, but doesn't conform to their defined schema in some way
B. The badRecordsPath location is accumulating many small files
C. The engineer forgot to set the option "cloudFiles.quarantineMode", "rescue".
D. At some point, the upstream data provider switched everything to multi-line JSON.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Auto Loader quarantines records that cannot be parsed according to the specified schema, even if the JSON itself is well formed. If incoming records contain additional fields, missing fields, or incompatible data types compared to the declared schema, they are treated as invalid and routed to the bad records path.

**QUESTION 258**
A data engineer wants to enforce the principle of least privilege when configuring ACLs for Databricks jobs in a collaborative workspace.

Which approach should the data engineer use?

A. Assign users only the minimum permission level (e.g., CAN RUN or CAN VIEW) required for their role on each job.
B. Use only folder-level permissions and avoid setting permissions on individual jobs.
C. Grant CAN RUN permission to everyone and CAN MANAGE to a single admin group.
D. Grant all users CAN MANAGE permission on all jobs to avoid access issues.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Assigning the minimum required permission level on each job ensures users can perform only the actions necessary for their role. This directly enforces the principle of least privilege while maintaining secure and controlled access in a collaborative workspace.

**QUESTION 259**
A data engineering workspace was automatically enabled for Unity Catalog, creating a workspace catalog. New team members report they can create tables in the default schema but cannot access table in other schemas within the same workspace catalog.

Why are the new team members unable to access tables in other schemas?

A. Workspace catalog permissions are not subject to inheritance rules.
B. Workspace users receive `USE CATALOG` and specific privileges on default schema only.
C. Tables in other schemas require additional `BROWSE` privileges that new users don't receive automatically
D. New users only receive `CREATE TABLE` privileges on the default schema.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
When a workspace catalog is automatically created, new users are granted USE CATALOG and limited privileges on the default schema only. Access to other schemas requires explicit grants, so users cannot see or query tables in those schemas without additional permissions.

**QUESTION 260**
A data engineer is implementing a job to download multiple PDF files from a third-party provided REST API endpoint by specifying different report types. The REST API is time-consuming and encounters intermittent errors, so the engineer wants to track each download activity to know when it fails and to retry partially, while providing scalable throughput. The engineer needs to download ten report types, and the list can be changed over time.

How should the data engineer achieve this?

A. Use a `foreach` task with a list of report types as its inputs.
B. Define ten `Notebook` tasks to clearly track which report download failed.
C. Use a Delta Lake table to track each report download status as 10 rows, and use it as a source table to execute the download function as a `Pandas UDF`.
D. Define a list variable within a Notebook to loop through the report types to download them, and print the download results. Execute it as a `Notebook` tasks.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
A foreach task allows the job to dynamically iterate over a configurable list of report types, execute downloads in parallel, and track the success or failure of each item independently. This enables scalable throughput, partial retries for failed downloads, and easy updates when the list of report types changes, without hardcoding tasks or introducing unnecessary complexity.

**QUESTION 261**
A data engineer and a platform engineer are working together to automate their system tasks. A script needs to be executed outside of Databricks only if a particular daily Databricks job finishes successfully for the day. Databricks CLI command was used to check the last execution of the job.

What are the required command options for that task?

A. `databricks jobs list-runs --job-id JOB_ID --start-time-to TODAY_MIDNIGHT_EPOCH_MS --completed-only`
B. `databricks jobs list-runs --job-id JOB_ID --start-time-from TODAY_MIDNIGHT_EPOCH_MS --active-only`
C. `databricks jobs list-runs --job-id JOB_ID --start-time-to TODAY_MIDNIGHT_EPOCH_MS --active-only`
D. `databricks jobs list-runs --job-id JOB_ID --start-time-from TODAY_MIDNIGHT_EPOCH_MS --completed-only`

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Using --start-time-from with today's midnight epoch ensures only runs that started during the current day are considered, and --completed-only filters out in-progress runs. This allows the script to reliably check whether the daily job has finished successfully before triggering downstream external actions.

**QUESTION 262**
A data engineering team needs to create a SQL Alert that monitors data quality across multiple columns in their customer table. They want to trigger an alert when both the percentage of customers with missing email addresses exceeds 15% AND the percentage of customers with invalid phone number formats exceeds 10%.

Which SQL query pattern is appropriate for implementing this multi-column alert condition?

A. `SELECT COUNT (*) FROM customers WHERE email IS NULL OR phone_format_invalid = true`
B. `SELECT email, phone FROM customers WHERE email IS NULL AND phone NOT RLIKE '^[0-9-+()\\s]+$'`
C. `SELECT email_null_pct, phone_invalid_pct FROM (SELECT (COUNT(CASE WHEN email IS NULL THEN 1 END) * 100.0/COUNT (*)) as email_null_pct, (COUNT(CASE WHEN phone NOT RLIKE '^[0-9-+()\\s]+$' THEN 1 END)* 100.0/COUNT (*)) as phone_invalid_pct FROM customers)`
D. `SELECT CASE WHEN email_null_pct >15 AND phone_invalid_pct> 10 THEN 1 ELSE 0 END FROM (SELECT (COUNT (CASE WHEN email IS NULL THEN 1 END) * 100.0 / COUNT (*)) as phone_invalid_pct FROM customers) metrics`

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
This pattern computes independent percentage metrics for each data quality condition in a single aggregated query. By calculating the percentage of missing emails and invalid phone formats as separate columns, it enables the SQL Alert to evaluate a compound condition where both thresholds must be exceeded before triggering.

**QUESTION 263**
A data engineer is brining an existing production Databricks job under asset bundle management and wants to ensure that:
- The job's current configuration is captured as YAML, and all referenced files are included in their bundle project.
- Future changes to the bundle's YAML will update the existing job in-place (not create a new job)

How should the data engineer successfully move the production job under asset bundle management?

A. Run Databricks bundle generate job --existing-job-id to generate the YAML and download referenced files. Then, run Databricks bundle deploy to deploy the bundle, which will always update the existing job automatically.
B. Export the job definition as JSON, convert it to YAML, and place it in your bundle. Then, run Databricks bundle deploy to update the existing job.
C. Manually create the YAML configuration for the job in your bundle project, ensuring all settings match the existing job. Then, run Databricks bundle deploy the bundle, which will update the existing job in your workspace.
D. Run databricks bundle generate job --existing-job-id to generate the YAML and download referenced files. Then, run Databricks bundle deployment, bind to link the bundle's job resource to the existing job in Databricks.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Generating the bundle from the existing job captures the full job configuration as YAML and pulls in all referenced files into the bundle project. Binding the generated job resource to the existing Databricks job establishes a persistent link, ensuring that future bundle deployments update the same production job in place rather than creating a new one.

**QUESTION 264**
A data architect is implementing Delta Sharing as part of their data governance strategy to enable secure data collaboration with external partners and internal business units. The architect must establish a permission framework that allows designated data stewards to create shares for their respective domains while maintaining security boundaries and audit compliance.

Which specific permissions and roles must be assigned to enable users to create, configure, and manage Delta Shares while maintaining proper security governance and access controls?

A. Only workspace admins can create and manage shares
B. Users need the MANAGE SHARES permission on the workspace
C. Users need to be metastore admins or have CREATE SHARE privilege for the metastore
D. Any user with USE_CATALOG privilege can create shares

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Creating and managing Delta Shares requires elevated governance privileges at the metastore level. Assigning users as metastore admins or granting them the CREATE SHARE privilege allows designated data stewards to create and configure shares within defined security boundaries, while preserving centralized auditability and access control.

**QUESTION 265**
A data engineer manages a production Lakeflow Spark Declarative Pipeline that processes customer transaction data. The pipeline includes several data quality expectations, such as: `transaction_amount > 0` and `customer_id IS NOT NULL.`

These expectations are defined using the EXPECT clause in SQL. The engineer aims to monitor the pipeline's data quality by analyzing the number of records that passed or failed each expectation during the latest pipeline update. The Lakeflow Spark Declarative Pipelines event logs are stored in a Delta table named `event_log_table.`

For the most recent pipeline update, determine a programmatically approach to extract information like the name of each expectation, associated dataset, count of records that passed the expectation, and count of records that failed the expectation.

Which method retrieves the desired data quality metrics from the Lakeflow Spark Declarative Pipelines event log?

A. Access the `event_log_table`, filter for events where `event_type = 'flow progress'`, and parse the `details.flow_progress.data_quality.expectations` field to extract the required metrics.
B. Use the Lakeflow Spark Declarative Pipelines UI to navigate to the specific pipeline, select the dataset, and view the Data Quality tab to manually retrieve the expectation metrics.
C. Query the `event_log_table` for events with `event_type = 'data_quality'` and directly select the `passed_records` and `failed_records fields.`
D. Access the `event_log_table`, filter for events where `event_type = 'expectatation_result',` and extract the expectation metrics from the details field.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Lakeflow Spark Declarative Pipelines emit data quality metrics as part of flow progress events. These events include structured details about expectations under the flow_progress.data_quality.expectations field, which contains the expectation name, dataset, and counts of records that passed or failed. Parsing this field from the most recent flow progress event provides a programmatic way to analyze data quality results.

**QUESTION 266**
Predictive Optimization is an automated Databricks service enabled by default for Unity Catalog Managed tables. It helps maintain Delta tables by continuously optimizing them to ensure optimal performance and costs.

Which two operations does Predictive Optimization run to maintain the Delta tables? (Choose two.)

A. PARTITION BY
B. COMPACT
C. ANALYZE
D. OPTIMIZE
E. BUCKETING

**Correct Answer:** CD
**Section:** (none)

**Explanation/Reference:**
Explanation:
Predictive Optimization automatically runs OPTIMIZE to compact small files and improve data layout, and ANALYZE to collect and refresh table statistics. Together, these operations ensure efficient query planning, better data skipping, and optimized performance and cost for Unity Catalog managed Delta tables.

**QUESTION 267**
A data engineer is building a customer data pipeline in Lakeflow Spark Declarative Pipelines. The source is a cloud-based event stream with limited retention containing inserts, updates, and deletes for customer records. These changes are being applied using the AUTO CDC INTO syntax to maintain an SCD Type 1 table as the target table, `customer_dim`.

How should the data engineer build a downstream job that streams from the `customer_dim` table to only act on updates and delete events, processing data incrementally?

A. Use ignoreChanges flag while streaming from `customer_dim` to avoid breaking the pipeline during updates and deletes.
B. Read change data feed from `customer_dim` table and apply filters to incrementally act on the change events.
C. Streaming from `customer_dim` table would only be possible in the case of SCD 2 retention.
D. When stored as SCD 1, the target of AUTO CDC INTO includes updates and deletes. Streaming from `customer_dim` can fail due to these operations. Instead, build another stream from the original source.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
Reading the change data feed from the customer_dim table enables downstream processing to react specifically to update and delete events while operating incrementally. Change data feed exposes row-level change types and versions, making it the correct mechanism for streaming only the relevant changes from an SCD Type 1 table maintained with AUTO CDC INTO.

**QUESTION 268**
A data engineer is designing a system leveraging Lakeflow Declarative Pipeline technology to process real-time truck telemetry data ingested from JSON files in S3 using Auto Loader. The data includes `truck_id`, timestamp, location, speed, and `fuel_level`. The system must support two use cases:

1. Near-real-time monitoring of the latest location, speed, and `fuel_level` per `truck_id` for the operations team.
2. Daily aggregated reports of total distance traveled and average fuel efficiency per `truck_id` for the management team.

Which approach should the data engineer use for streaming tables and materialized views in the Lakeflow Declarative Pipeline to meet these requirements?

A. Define a streaming table to ingest and store the raw telemetry data, and create a streaming table to compute the daily aggregated distance and fuel efficiency per `truck_id` reporting. Create a materialized view to compute the latest location, speed, and `fuel_level` per `truck_id` for real-time monitoring.
B. Define a streaming table to ingest and store the raw telemetry data, and create a materialized view to compute the latest location, speed, and `fuel_level` per `truck_id` for real-time monitoring. Create another materialized view to compute the daily aggregated distance and fuel efficiency per `truck_id` for reporting.
C. Define a streaming table to ingest and store the raw telemetry data, and create a streaming table to incrementally compute the latest location, speed, and `fuel_level` per `truck_id` for real-time monitoring. Create a materialized view to compute the daily aggregated distance and fuel efficiency per `truck_id` for reporting.
D. Define a materialized view to ingest and store the raw telemetry data, and create a streaming table to compute the latest location, speed, and `fuel_level` per `truck_id` for real-time monitoring. Create another materialized view to compute the daily aggregated distance and fuel efficiency per `truck_id` for reporting.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
A streaming table is the right construct to ingest continuously arriving telemetry from Auto Loader. Computing the latest per truck_id requires near-real-time incremental updates as new events arrive, which is best handled with a downstream streaming table. The daily aggregates are naturally suited to a materialized view, which maintains precomputed results for reporting and refreshes efficiently without requiring a continuously running streaming aggregation for a once-per-day consumption pattern.

**QUESTION 269**
A platform team lead is responsible for automating the individual teams attribution towards SQL Warehouse usage. The requirement is to identify the SQL warehouse usage at the individual user's level and generate a daily report to be shared with an executive team that includes leaders from all business units.

How should the platform lead generate an automated report that can be shared daily?

A. Use the system tables to capture the audit and billing usage data and share the queries with the executive team. This enables the executives to execute the query and see the latest results any time.
B. Use the system tables to capture the audit and billing usage data and create a dashboard with daily refresh schedules and shared with the executive team.
C. Restrict users from running any SQL query unless they provide all the query details so that the attribution can be calculated and shared with the executive team.
D. Let the users run the SQL query and then directly report the usage to the executives. The ownership of the SQL warehouse usage will be with the individual teams.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
System tables provide authoritative audit and billing data needed for per-user SQL Warehouse attribution. Creating a dashboard with a scheduled daily refresh automates report generation and ensures executives receive consistent, up-to-date insights without needing to run queries themselves.

**QUESTION 270**
A data engineering team is collaborating on a Databricks project where each team member needs to develop and test code independently before merging changes into the main branch.

They want to avoid accidental overwrites or branch switching issues while ensuring that all work is version-controlled and can be integrated into their CI/CD pipeline.

How should the data engineer achieve collaboration?

A. Each team member creates their own Databricks Git folder, mapped to the same remote Git repository, and works in their own development branch within their personal folder.
B. All team members work in the same Databricks Git folder and perform Git operations (pull, push, commit, branch switching) directly in that shared folder.
C. Team members edit notebooks directly in the workspace's shared folder and periodically copy changes into a Git folder for version control.
D. Team members use the Databricks CLI to clone the Git repository and perform Git operations from a cluster's web terminal.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Using separate Databricks Git folders per user mapped to the same remote repository allows each team member to work independently on their own branch without interfering with others. This prevents accidental overwrites or branch conflicts while ensuring all changes are version-controlled and easily integrated into CI/CD workflows.

**QUESTION 271**
A data engineer is using the AUTO CDC API in Lakeflow Spark Declarative Pipeline to propagate deletions from a source table (`orders_source`) to a target table (`orders_target`). The source has Change Data Feed (CDF) enabled, but some delete events arrive out of order due to upstream delays.

How does the AUTO CDC API internally ensure deletions are applied correctly despite out-of-order events?

A. It ignores deletions if they arrive after updates for the same key.
B. It manually sorts incoming events by timestamp before applying changes.
C. It runs VACUUM on the target table to purge conflicting records.
D. It uses `sequence_by` to order events and retains tombstones for deleted rows until older sequences are processed.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
AUTO CDC uses the sequence_by column to deterministically order change events for each key. Delete operations create tombstones that are retained until all earlier sequence values have been processed, ensuring that out-of-order delete events are still applied correctly and consistently in the target table.

**QUESTION 272**
A data governance team at a large enterprise is improving data discoverability across its organization. The team has hundreds of tables in their Databricks Lakehouse with thousands of columns that lack proper documentation. Many of these tables were created by different teams over several years, with missing context about column meanings and business logic. The data governance team needs to quickly generate comprehensive column descriptions for all existing tables to meet compliance requirements and improve data literacy across the organization. They want to leverage modern capabilities to automatically generate meaningful descriptions rather than manually documenting each column, which would take months to complete. The team is looking for a solution that can understand data patterns, column names, and sample values to create intelligent descriptions.

Which approach should the team use in Databricks to automatically generate column comments and descriptions for existing tables?

A. Write custom PySpark code using `df.describe()` and `df.schema` to programmatically generate basic statistical descriptions for each column.
B. Navigate to the table in Databricks Catalog Explorer, select the table schema view, and use the “AI Generate” option which leverages artificial intelligence to automatically create meaningful column descriptions based on column names, data types, sample values, and data patterns.
C. Use the `DESCRIBE TABLE` command to extract existing schema information and manually write descriptions based on column names and data types.
D. Use Delta Lake’s `DESCRIBE HISTORY` command to analyze table evolution and infer column purposes from historical changes.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
The Catalog Explorer provides an AI-powered “AI Generate” capability that automatically creates intelligent column descriptions by analyzing column names, data types, sample values, and observed data patterns. This approach enables rapid, scalable documentation of existing tables, significantly improving data discoverability and compliance without manual effort.

**QUESTION 273**
A company processes semi-structured JSON files from an external source using Auto Loader in a classic Databricks job. Occasionally, records arrive with null critical fields, invalid types, or unexpected nested schema variations. The engineer must ensure that malformed or non-conforming records are not dropped silently and are captured in a separate quarantine table. The pipeline should continue processing good records into the Bronze layer without failing the job, and the approach must support both batch and streaming ingestion.

The data engineer needs to build a robust ingestion pattern that automatically routes bad records to a quarantine Delta table, while still ingesting good records into the Bronze layer for further processing.

Which approach fulfills the quarantine mechanism in this ingestion architecture?

A. Create a notebook job with inferSchema= True, write a streaming query with `.foreachBatch()` and catch exceptions using try/except to redirect failed batches to quarantine.
B. Use Auto Loader with failFast mode to set to false, and enable schema evolution; invalid records will be silently ignored during ingestion.
C. Use Lakeflow Spark Declarative Pipelines with a SQL pipeline; configure it to drop rows with nulls using where `critical_fields` is not null, and rely on audit logs for malformed data.
D. Use Auto Loader with LDP and implement an EXPECT () constraint with a record audit logic to route bad records.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Lakeflow Spark Declarative Pipelines provide built-in support for data quality enforcement using EXPECT clauses. By defining expectations on critical fields and configuring the pipeline to audit or quarantine failed records, malformed or non-conforming JSON records are automatically routed to a quarantine Delta table while valid records continue flowing into the Bronze layer. This approach is robust, auditable, works for both batch and streaming ingestion, and avoids silently dropping bad data or failing the pipeline.

**QUESTION 274**
A data engineer is analyzing transactional data in a PySpark DataFrame df containing `customer_id`, `transaction_timestamp` (precise to milliseconds), and `amount_spent`. The objective is to compute a cumulative sum of `amount_spent` per customer, strictly ordered by `transaction_timestamp`. The cumulative sum must include all transactions from the earliest timestamp up to and including the current row, respecting temporal ordering within each customer partition.

Which PySpark code snippet most accurately constructs the appropriate window specification and applies

the aggregation to yield the correct cumulative expenditure per customer?

A.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col

window_spec = Window.partitionBy("customer_id")\
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("cumulative_spent", sum("amount_spent").over(windo
```

B.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col

window_spec = Window.orderBy("transaction_timestamp")\
    .rowsBetween(Window.unboundedPreceding, Window.currentRo
df.withColumn("cumulative_spent", sum("amount_spent").over(w
```

C.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col

window_spec = Window.partitionBy("customer_id").orderBy("trans
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("cumulative_spent", sum("amount_spent").over(win
```

D.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col

window_spec = Window.partitionBy("customer_id").orderBy("t
df.withColumn("cumulative_spent", sum("amount_spent").over
```

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
This window specification partitions the data by customer_id, orders transactions by transaction_timestamp, and defines the frame from the first transaction through the current one. This guarantees that the cumulative sum is computed independently per customer and strictly follows the temporal order, including all prior transactions up to the current row.

**QUESTION 275**
A data team is implementing an append-only Delta Lake pipeline that needs to handle both and streaming data. They want to ensure that schema changes in the source data can be automatically incorporated without breaking the pipeline.

Which configuration should the team use when writing data to the Delta table?

A. ignoreChanges = false
B. validateSchema= false
C. overwriteSchema=true
D. mergeSchema=true

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Setting mergeSchema to true allows Delta Lake to automatically evolve the table schema by incorporating new columns from the source data during writes. This enables append-only pipelines to handle schema changes seamlessly in both batch and streaming workloads without breaking the pipeline.

**QUESTION 276**
A senior data engineer is planning large-scale data workflows. The current task is to identify the considerations that form a foundation for creating scalable data models that are essential for effective management of large datasets. The data engineering team has identified the core capabilities as part of a scalable data model to build a modern data platform and provided their reasoning for considering Delta Lake for review. The senior data engineer is responsible for identifying the recommendations that are not valid.

Which key features can be ignored while evaluating Delta Lake?

A. Delta Lake works with various data formats (e.g., Parquet, JSON, CSV) and integrates well with Spark and Databricks tools:
B. Delta Lake optimizes metadata handling, efficiently managing billions of files and facilitating scalability to petabyte-scale datasets.
C. Delta Lake's capability to process data in both batch and streaming modes seamlessly, providing flexibility in data ingestion and processing.
D. Delta Lake provides limited support for monitoring and troubleshooting data pipelines, so relevant partner tools have to be identified and set up for enhanced operational efficiency.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Delta Lake includes built-in capabilities for monitoring, auditing, and troubleshooting through transaction logs, history, and tight integration with the Databricks platform. Therefore, limited support for monitoring and troubleshooting is not a valid concern when evaluating Delta Lake and can be ignored.

**QUESTION 277**
A data engineering team is implementing an append-only data pipeline using Delta Lake, and wants to ensure that data is never modified or deleted once written.

Which Delta Lake feature should the data engineer enable to prevent modifications to existing data?

A. Delta APPEND_ONLY
B. Delta VACUUM
C. Delta OPTIMIZE
D. Delta Time Travel

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Enabling the append-only table property enforces that data can only be inserted into the Delta table. Updates and deletes are blocked, ensuring that once data is written it is never modified or removed, which is essential for strict append-only pipeline guarantees.

**QUESTION 278**
A data engineering team is setting up a Git project to automate integration tests using Databricks Asset Bundles and the Git provider's CI/CD functionalities. When a pull containing changes to their pipleline is sent, they need to run a Job to test their data pipeline.

What is the correct `databricks bundle` command sequence to be executed from the Git provider's CI/CD automation for this task?

A. init, deploy, run, validate
B. validate, deploy, run
C. init, validate, deploy, run
D. deploy, run, validate



**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
The correct sequence is to first validate the bundle to ensure the configuration is correct, then deploy it to provision or update the defined resources, and finally run the job to execute the integration tests. This aligns with CI/CD best practices by catching configuration issues early and only running tests after a successful deployment.

**QUESTION 279**
A data engineer is attempting to execute the following PySpark code:

```
df=spark.read.table("sales")
result=df.groupBy("region").agg(sum("revenue"))
```

However, upon inspecting the execution plan and profiling the Spark job, they observe excessive data shuffling during the aggregation phase.

Which technique should be applied to reduce shuffling during the groupBy aggregation operation?

A. Use.coalesce(1) after the aggregation
B. Caching the DataFrame df
C. Use broadcast join
D. Repartition by region before aggregation

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Repartitioning the DataFrame by the grouping key ensures that records with the same region are

colocated in the same partitions before the aggregation runs. This significantly reduces the amount of data shuffled during the groupBy operation, leading to more efficient execution.

**QUESTION 280**
While reviewing a query's execution in the Databricks Query Profile, a data engineer observes that the "Top operators" panel shows a sort operator with high "Time spent" and "Memory peak" metrics, and the Spark UI reports frequent data spilling.

How should the data engineering address this issue?

A. Repartition the DataFrame to a single partition before sorting.
B. Convert the sort operation to a filter operation.
C. Switch to a broadcast join to reduce memory usage.
D. Increase the number of shuffle partitions to better distribute data.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Increasing the number of shuffle partitions distributes the data across more tasks, reducing per-task memory pressure during the sort operation. This helps mitigate spilling by lowering memory peak usage per task and improves overall sort performance in large-scale distributed queries.

**QUESTION 281**
A data engineer manages a Unity Catalog table `customer_data` in schema finance that includes sensitive fields like ssn and `credit_score`. Intern Group should only see masked values, while Analyst Group should only access rows for their assigned region. The data engineer needs to restrict access based on user role and region without duplicating data.

How should the data engineer enforce this security policy?

A. Use Unity Catalog's row filters based on the region and column masks based on user roles.
B. Create views using `current_user()` and `is_account_group_member()` functions, and apply masking logic inside the SQL SELECT clause for each sensitive column.
C. Create dynamic views for each user role and manage access with ACLs.
D. Use Unity Catalog's row filters based on the user roles and column masks based on the region.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Unity Catalog row filters can restrict which rows are visible based on attributes such as the user's assigned region, while column masks can dynamically obfuscate sensitive fields like ssn and credit_score based on user roles. This enforces fine-grained, role- and region-based access control directly at the table level without duplicating data or relying on custom views.

**QUESTION 282**
A data engineer us ingesting JSON files from cloud object storage using Databricks Auto Loader. The source folder may occasionally receive large files of data, which risks overwhelming the stream. To ensure predictable micro-batch sizes, the team wants to throttle ingestion based on the volume of data scanned at 1 GB, regardless of the number of files.

Which Auto Loader configuration should the data engineer used to achieve this?

A. Configure `cloudFiles.maxBytesPerTrigger` with 1 GB to place a limit.
B. Configure `cloudFiles.maxSizePerTrigger` with 1 GB to place a limit.
C. Configure `cloudFiles.maxFilesPerTrigger` and estimate the average file size to approximate a size-based throttle of 1 GB.

D. Configure `cloudFiles.maxPartitionBytes` with 1GB to limit data in each partition.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
cloudFiles.maxBytesPerTrigger limits the total volume of data scanned in each micro-batch based on size rather than file count. Setting it to 1 GB ensures predictable ingestion throughput even when large files arrive, preventing any single trigger from overwhelming the streaming job.

**QUESTION 283**
A data company uses Databricks Unity Catalog and has multiple enterprise data sources, including PostgreSQL, Snowflake, and SQL Server. The central data platform team wants to configure Lakehouse Federation so analysts can query external tables directly in Databricks using Databricks SQL, without duplicating data.

Which steps are necessary to configure Lakehouse Federation in a secure and governed manner?

A. Mirror the external datasets into Delta Lake using Auto Loader, and govern them using Data Lineage and System Tables.
B. Configure connections and foreign catalog in Unity Catalog, then grant access to foreign catalogs, schemas, and tables using Unity Catalog permissions.
C. Use Partner Connect to create linked datasets, and apply table ACLs at the source system to govern access through Databricks.
D. Create external locations and storage credentials to connect to each database, then register foreign tables in Unity Catalog.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
Lakehouse Federation is configured by defining secure connections to external data sources and registering them as foreign catalogs in Unity Catalog. Access is then governed using Unity Catalog permissions at the catalog, schema, and table levels, enabling analysts to query external tables securely without data duplication.

**QUESTION 284**
Why are Pandas UDFs often preferred over traditional PySpark UDFs in performance-critical applications involving large datasets?

A. They minimize memory usage by streaming each row individually through a lightweight Python wrapper, avoiding batch processing overhead.
B. They leverage Apache Arrow to enable vectorized operations between the JVM and Python runtimes, reducing serialization costs and improving computational efficiency.
C. They allow row-level execution of functions in Python with native Spark optimization, removing the need for columnar execution.
D. They eliminate the JVM-Python boundary by bypassing serialization entirely, thereby avoiding data conversion overhead.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
Pandas UDFs use Apache Arrow to transfer data between the JVM and Python in a columnar, vectorized format. This significantly reduces serialization overhead and enables efficient batch processing, resulting in much better performance than traditional row-by-row PySpark UDFs on large datasets.

**QUESTION 285**

A data engineer is setting up a pipeline to ingest data from a message bus system that occasionally delivers duplicate messages. The duplicate messages can be a week apart. The target is a Databricks Delta Lake table where each record should appear exactly once.

Which Databricks ingestion pattern should be implemented to handle potential duplicates where events can arrive outside of the configured watermark?

A. Use Delta Lake's change data feed to filter duplicate records
B. Use Delta Lake time travel to identify and remove duplicates
C. Configure Structured Streaming with dropDuplicates transformation
D. Implement a write operation using MERGE INTO with a unique key

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Using MERGE INTO with a unique key enforces idempotent writes at the Delta Lake table level. This approach reliably handles duplicates even when events arrive far outside any streaming watermark, ensuring that each logical record is written exactly once regardless of arrival time.

**QUESTION 286**
A data engineer needs to design an efficient pipeline that automatically processes new CSV files as they arrive in S3 storage.

Which Databricks feature should the data engineer use to meet these requirements?

A. Streaming from cloud storage using standard Spark `readStream with format ("csv") and format ("json")`
B. COPY INTO SQL command with parameters to track processed files
C. Traditional batch processing with scheduled Databricks Jobs
D. Auto Loader with schema inference and evolution enabled



**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Auto Loader is designed to efficiently and incrementally process new files as they arrive in cloud object storage. It provides scalable file discovery, supports schema inference and evolution, and minimizes overhead compared to traditional batch or manual streaming approaches.

**QUESTION 287**
A data engineer is implementing liquid clustering on a Delta Lale table and needs to understand how it affects data management operations. The table will be updated frequently with new data. The table is an external table and not managed by Unity Catalog.

How does liquid clustering in Delta Lake handle new data that is inserted after the initial table creation?

A. New data is rejected if it doesn't match the clustering pattern.
B. New data is automatically clustered during write operations.
C. New data is written to a staging area and clustered during scheduled maintenance.
D. New data remains unclustered until the next OPTIMIZE operation.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
With liquid clustering, newly inserted data is written without being immediately reclustered. The clustering

layout is applied incrementally during subsequent OPTIMIZE operations, which reorganize both existing and newly added data to maintain an efficient data layout as the table evolves.

**QUESTION 288**
A data engineer is optimizing a MERGE operation on an 800GB UC-managed table that experiences frequent updates and deletions.

Which two actions should the engineer prioritize to improve MERGE performance? (Choose two.)

A. Apply liquid clustering using the merge join keys.
B. Enable deletion vectors on the table if not already enabled.
C. Partition the table by date.
D. Use ZORDER on high-cardinality columns.
E. Overwrite the table instead of Merge.

**Correct Answer:** AB
**Section:** (none)

**Explanation/Reference:**
Explanation:
Liquid clustering on the merge join keys improves data locality and reduces the amount of data scanned during MERGE operations, which is especially effective for large, frequently updated tables. Enabling deletion vectors avoids rewriting entire Parquet files for updates and deletes, significantly reducing I/O and improving MERGE performance on Unity Catalog–managed tables.

**QUESTION 289**
A data engineer is building a streaming data pipeline to ingest JSON files from cloud storage into a Delta Lake table. The pipeline must process files incrementally, handle schema evolution automatically, ensure exactly-once processing, and minimize manual infrastructure management.

How should the data engineer fulfill these requirements?

A. Use Lakeflow Spark Declarative Pipelines with a static DataFrame read, merge schema with `spark.conf.set (“spark.databricks.delta.schema.autoMerge.enabled”, “true”)`
B. Use Auto Loader in batch mode with a daily job to overwrite the Delta table.
C. Use Lakeflow Spart Declarative Pipelines with Auto Loader and enabling schema inference with `“cloudFiles.schemaEvolutionMode”= “addNewColumns”`
D. Use traditional Spark Structured Streaming with Auto Loader, manually configuring checkpoints location and enabling schema inference with `“mergeSchema”= “true”`

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Lakeflow Spark Declarative Pipelines combined with Auto Loader provide fully managed incremental file ingestion with exactly-once guarantees and minimal operational overhead. Enabling schema inference and evolution allows new columns in incoming JSON files to be incorporated automatically, satisfying the requirements for streaming ingestion, schema evolution, and reduced manual infrastructure management.

**QUESTION 290**
Given the following PySpark code snippet in a Databricks notebook:

```
filtered_df=spark.read.format(“delta”).load(“/mnt/daya/large_table”) \
    .filter (“event_date> ‘2024-01-01’”)
filtered_df.count ()
```

The data engineer notices from the Query Profile that the scan operator for `filtered_df` is reading almost all files, despite a filter being applied.

What is the probable reason for poor data skipping?

A. The Delta table lacks optimization that enables dynamic file pruning.
B. The filter condition involves a data type that is excluded from data skipping support.
C. The filter is executed only after the full data scan, which prevents data skipping from taking place.
D. The `event_date` column is outside the table's partitioning and Z-ordering scheme.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Delta Lake's data skipping relies on partitioning and clustering (such as Z-ordering) on the filtered columns. If event_date is neither a partition column nor included in the table's clustering strategy, Spark must scan most files because file-level statistics cannot be effectively used to prune irrelevant data.

**QUESTION 291**
When a new Databricks project starts, the central IP team provisions the required infrastructure using Terraform and a Service Principal. This includes creating a Databricks workspace, a Unity Catalog linked to an External Location, and a Databricks group containing all project team members. Project teams must store all assets – e.g., tables and volumes, as Managed assets in Unity Catalog. This model hides infrastructure complexity while giving teams autonomy within their catalog. They can create and manage schemas, tables, volumes, and related objects but cannot rename, delete, or change catalog permissions, those remain under IT's control.

Which rights should the project group be granted to enable this model?

A. The group needs to have USE CATALOG and USE SCHEMA on the catalog.
B. The group needs to have ALL PRIVILEGES and the MANAGE on the catalog.
C. The group needs to have ALL PRIVILEGES on the catalog.
D. The group should be made OWNER of the catalog.



**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Granting the project group USE CATALOG (and USE SCHEMA on existing schemas) allows team members to work within the catalog without giving them ownership or administrative control. This enables teams to create and manage schemas, tables, and volumes inside the catalog while preventing them from renaming, deleting, or modifying catalog-level permissions, which remain under central IT governance.

**QUESTION 292**
A data engineer needs to productionize a new Spark application written by teammate. This application has numerous external dependencies, including libraries, and requires custom environment variables and Spark configuration parameters to be set.

Which two methods will help the data engineer accomplish the task? (Choose two.)

A. Install libraries on DBFS
B. Add libraries to compute policies
C. Use secrets in init scripts to store configuration data
D. Use compute policies to set system properties, environment variables, and Spark configuration parameters.
E. Create init scripts on DBFS.

**Correct Answer:** DE
**Section:** (none)

**Explanation/Reference:**
Explanation:

Compute policies allow centrally defining and enforcing Spark configuration parameters, system properties, and environment variables required by the application, ensuring consistent production settings. Init scripts enable installing external dependencies and performing custom environment setup at cluster startup, making them essential for productionizing Spark applications with complex dependency and configuration requirements.

**QUESTION 293**
A healthcare analytics team is implementing a dimensional model in Delta Lake for patient care analysis. They have a date dimension table and are evaluating design options to ensure it supports a wide range of time-based analyses.

Which design approach for the date dimension will support efficient time-based querying and aggregation?

A. Store the date as string in ISO format (YYYY-MM-DD) for readability.
B. Pre-calculate attributes like fiscal_period, quarter, month_name, day_of_week, and holiday.
C. Store only the date value and calculate all time attributes in queries.
D. Create separate dimension tables for different calendar systems (fiscal, academic, etc.)

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
Pre-calculating commonly used time attributes in the date dimension enables efficient filtering, grouping, and aggregation without requiring repeated computations at query time. This design supports a wide range of time-based analyses and improves query performance and usability in dimensional models.

**QUESTION 294**
A data engineer is tasked with ensuring that a Delta table in Databricks continuously retains deleted files for 15 days (instead of the default 7 days), in order to permanently comply with the organization's data retention policy.

Which code snippet correctly sets this retention period for deleted files?

A. `spark.sql("ALTER TABLE my_table SET TBLPROPERTIES ('delta.deletedFileRetentionDuration' = 'interval 15 days')")`
B.
```
from delta.tables import *
deltaTable = DeltaTable.forPath(spark, "/mnt/data/my_table")
deltaTable.deletedFileRetentionDuration = "interval 15 days"
```
C. `spark.sql("VACUUM my_table RETAIN  HOURS")`
D. `spark.conf.set("spark.databricks.delta.deletedFileRetemtionDuration", "15 days")`

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
The deleted file retention period in Delta Lake is controlled by the table property delta.deletedFileRetentionDuration. Setting this property via ALTER TABLE ensures the retention policy is persistently enforced at the table level, extending deleted file retention to 15 days in compliance with organizational requirements.

**QUESTION 295**
A Data Engineer is building a fraud detection pipeline that calls out to Open AI, via a Python library, and needs to include an access token when using the API.

Which Databricks CLI command should the Data Engineer use to create the secret?

A. `databricks secrets put-secret KEY SCOPE; dbutils.secrets.get (KEY, SCOPE)`
B. `databricks tokens put-token SCOPE KEY; dbutils.tokens.get (SCOPE, KEY)`
C. `databricks secrets put-secret SCOPE KEY; dbutils.secrets.get (SCOPE, KEY)`
D. `databricks tokens put-token KEY SCOPE; dbutils.secrets.get (KEY, SCOPE)`

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
The Databricks CLI command databricks secrets put-secret <scope> <key> is used to securely store sensitive values such as API access tokens in a secret scope. The secret can then be retrieved at runtime using dbutils.secrets.get(scope, key), ensuring credentials are not hard-coded and remain protected.

**QUESTION 296**
A company has a task management system that tracks the most recent status of tasks. The system takes task events as input and processes events in near real-time using Lakeflow Spark Declarative Pipelines. A new task event is ingested into the system when a task is created or the status is changed. Lakeflow Spark Declarative Pipelines provides a streaming table (table name: tasks_status) for the BI users to query. The table represents the latest status of all tasks and includes 5 columns: `task_id`(unique for each task), `task_name, task_owner, task_status, task_event_time.` The table enables 3 properties: deletion vectors, row tracking, and change data feed.

A data engineer is asked to create a new Lakeflow Spark Declarative Pipelines to enrich the `"task_status"` table in near real-time by adding one additional column representing `task_owner's` department, which can be looked up from a static dimension table (table name: employee).

How should this enrichment be implemented?

A. Create a new Lakeflow Spark Declarative Pipelines: use readStream() function with option readChangeFeed to read `tasks_status` table CDF; enrich with the employee table; create a new streaming table as the result table and use `apply_changes()` function to process the changes from the enriched CDF.
B. Create a new Lakeflow Spark Declarative pipeline: use the readStream()function to read `tasks_status table`, enrich with the employee table; store the result in a new streaming table.
C. Create a new Lakeflow Spark Declarative Pipeline: use the readStream() function with the option skipChangeCommits to read the `tasks_status` table; enrich with the employee table; store the result in a new streaming table.
D. Create a new Lakeflow Spark Declarative Pipeline: use the read () function to read `tasks_status` table; enrich with employee table; store the result in a materialized view.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Because tasks_status represents the latest state and includes updates, the downstream pipeline must correctly propagate incremental changes. Reading the table's change data feed provides the insert/update/delete events needed for near-real-time incremental processing. Enriching those change events with the static employee dimension and using apply_changes() rebuilds the latest-state target correctly while preserving update semantics, rather than treating the source as append-only.

**QUESTION 297**
A data engineer is reviewing the PySpark code to copy a part of the production dataset to the sandbox environment, and needs to be sure that no PII(Personally Identifiable Information) data is being copied. After checking the sales table, the data engineer notices that it has user emails as the only PII data included as well as being the only column to identify the user.

from `pyspark.sql` import functions as F

```
# Load data from a table in the production environment
df = spark.read.table("production_catalog.production_schema.sales").sample(fraction=0.1)

# Anonymize the data before saving it to the sandbox
sandbox_df = << fill in >>

# Write the anonymized data to a table in the sandbox environment
sandbox_df.write.format("delta").mode("overwrite").saveAsTable("sandbox_catalog.sandbox_schema.sales")
```

Which anonymised code should be used to achieve the required outcome?

A. `df.withColumn (“user_emai”, F.expr(“uuid()”))`
B. `df.withColumn (“user_email”, F.sha2 (“user_email”))`
C. `df.withColumn (“hashed_email”, sha2 (“user_email”))`
D. `df.withColumn (“user_email”, F.regexp_replace (“user_eamail”, “@*”, “@anonymized.com”))`

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
Hashing the email column replaces the original PII with a deterministic, irreversible value while preserving its role as a unique identifier. This ensures no actual email addresses are copied to the sandbox environment, while still allowing consistent joins or user-level analysis if needed.

**QUESTION 298**
A data engineer wants to ingest a large collection of image files (JPEG and PNG) from cloud object storage into Unity Catalog-managed table for further analysis and visualization.

Which two configurations and practices are recommended to incrementally ingest these images into the table? (Choose two.)

A. Use Auto loader and set cloudFiles.format to "TEXT"
B. Use Auto loader and set cloudFiles.format to "BINARYFILE"
C. Use Auto loader and set cloudFiles.format to "IMAGE"
D. Use the pathGlobFilter option to select only image files (e.g.. "*.jpg, *.png")
E. Move files to a volume and read with SQL editor

**Correct Answer:** BD
**Section:** (none)

**Explanation/Reference:**
Explanation:
Using Auto Loader with the BINARYFILE format enables efficient ingestion of binary assets such as JPEG and PNG files into a Unity Catalog-managed table while preserving file metadata and supporting incremental processing.

Applying a pathGlobFilter ensures that only image files (e.g., .jpg, .png) are incrementally detected and ingested, preventing non-image objects from entering the pipeline and optimizing ingestion performance.

**QUESTION 299**
A data engineer is configuring Delta Sharing for a Databricks-to-Databricks scenario to optimize read performance. The recipient needs to perform time travel queries and streaming reads on shared sales data.

Which configuration will provide the optimal performance while enabling these capabilities?

A. Use the open sharing protocol instead of Databricks-to Databricks sharing for better performance.
B. Share tables WITHOUT HISTORY and enable partitioning for better query performance.

C. Share tables WITH HISTORY, ensure tables don't have partitioning enabled. and enable CDF before sharing.
D. Share the entire schema WITHOUT HISTORY and rely on recipient-side caching for performance.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Sharing tables with history is required to support time travel queries.

Enabling Change Data Feed (CDF) is necessary for streaming reads on shared Delta tables.

Avoiding partitioning improves performance in Databricks-to-Databricks Delta Sharing, as the sharing protocol is optimized for non-partitioned tables and predicate pushdown over shared metadata.

**QUESTION 300**
A data organization has adopted Delta Sharing to securely distribute curated datasets from a Unity Catalog-enabled workspace. The data engineering team is sharing large Delta tables with an internal partner via Databricks-to-Databricks and aggregated reports with an external client via Open Sharing. While testing new sharing workflows, the data engineering team encounters challenges related to access control, data update visibility, and shareable object types.

What is a limitation of the Delta Sharing protocol or implementation when used with Databricks-to-Databricks or Open Sharing?

A. Delta Sharing does not support Unity Catalog enabled tables; only legacy Hive Metastore tables are shareable.
B. With Databricks-to-Databricks sharing. Unity Catalog recipients must re-ingest data manually using `COPY INTO` or `REST APIs`.
C. Delta Sharing (both Databricks-to-Databricks and Open sharing) allows recipients to modify the source data if they have `SELECT` privileges.
D. With Open sharing, recipients cannot access Volumes, Models, or notebooks — only static Delta tables are supported.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Open Sharing supports sharing Delta tables only. It does not support sharing Volumes, ML models, notebooks, or other Unity Catalog object types, which are only supported in Databricks-to-Databricks sharing scenarios.

**QUESTION 301**
A data engineer is creating a daily reporting Job. There are two reporting notebooks one for weekdays and the other for weekends. An "If/else condition" task is configured as `"{{job.start_time.is_weekday}} == true"` to route the job to either weekday or weekend notebook tasks. The same job would be used in multiple time zones.

Which action should a senior data engineer take upon reviewing the job to merge or reject the pull request?

A. Reject. As they should use `{{job.trigger_time.is_weekday}}` instead.
B. Reject. As the `{{job.start_time.is_weekday}}` is not a valid value reference.
C. Reject. As the `{{job.start_time.is_weekday}}` is for the UTC timezone.
D. Merge. As the job configuration looks good.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
The job.start_time.is_weekday expression is evaluated using the job's start time in UTC, not local time zones.

Since the same job is used across multiple time zones, this would route incorrectly for regions where the local date differs from UTC, making the logic unreliable for weekday/weekend routing.

**QUESTION 302**
How are the operational aspects of Lakeflow Spark Declarative Pipelines from Spark Structured Streaming different?

A. Lakeflow Spark Declarative Pipelines automatically handle schema evolution, while Structured Streaming always requires manual schema management.
B. Structured Streaming can process continuous data streams, while Lakeflow Spark Declarative Pipelines cannot.
C. Lakeflow Spark Declarative Pipelines manage the orchestration of multi-stage pipelines automatically, while Structured Streaming requires external orchestration for complex dependencies.
D. Lakeflow Spark Declarative Pipelines can write to Delta Lake format while structured streaming cannot.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Lakeflow Spark Declarative Pipelines automatically manage orchestration, task dependencies, retries, and multi-stage pipeline execution, providing built-in operational management.

Spark Structured Streaming focuses on stream processing execution only and requires external orchestration tools (e.g., Jobs, Airflow, workflows) to manage complex multi-stage pipelines and dependencies.

**QUESTION 303**
A workspace admin has created a new catalog called `'finance_data'` and wants to delegate permission management to a finance team lead without giving them full admin rights.

Which privilege should be granted to the finance team lead?

A. `GRANT_OPTION` privilege on the `finance_data` catalog.
B. Make the finance team lead a metastore admin.
C. MANAGE privilege on the `finance_data` catalog.
D. ALL PRIVILEGES on the `finance_data` catalog.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Granting the MANAGE privilege on the finance_data catalog allows the finance team lead to manage permissions and objects within that catalog (schemas, tables, grants) without requiring metastore admin or full administrative rights, enabling delegated governance in a least-privilege model.

**QUESTION 304**
A data engineer is configuring a Lakeflow Spark Declarative Pipelines to process CDC data from a source. The source events sometimes arrive out of order, and multiple updates may occur with the same `update_timestamp` but with different `update_sequence_id`.

What should the data engineer do to ensure events are sequenced correctly?

A. Use `SEQUENCE BY STRUCT(event_timestamp, update_sequence_id)` in `AUTO CDC` APIs.

B. Set `track history column list to [event timestamp, event id]` in `AUTO CDC` APIs.
C. Use `dropduplicate()` to remove out-of-order and duplicate records in LDP.
D. Use a window function to `sort update_sequence_id` within the same partition, i.e., `update_timestamp` in the LDP pipeline.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Using SEQUENCE BY STRUCT(update_timestamp, update_sequence_id) in the AUTO CDC APIs defines a deterministic ordering key for CDC processing.

This ensures correct event sequencing when records arrive out of order and when multiple updates share the same timestamp by using the sequence ID as a tie-breaker, guaranteeing consistent and correct change application in the pipeline.

**QUESTION 305**
A streaming video analytics team ingests billions of events daily into a Unity Catalog-managed Delta table `video_events`. Analysts run ad-hoc point-lookup queries on columns like `user_id`, `campaign_id`, and `region`. The team manually runs OPTIMIZE `video_events` Z-ORDER `user_id`, `campaign_id`, `region`, but still sees poor performance on recent data, and dislikes the operational overhead. The team wants a hands-off way to keep hot columns co-located as query patterns evolve.

Which Delta capability should the team leverage on `video_events`?

A. Enable auto-compaction (optimizeWrite & autoCompact).
B. Schedule OPTIMIZE/ZORDER to run after each job to improve recent file performance.
C. Utilize Liquid Clustering CLUSTER BY AUTO and Predictive Optimization.
D. Enable Delta caching.



**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Liquid Clustering with CLUSTER BY AUTO continuously adapts data layout based on observed query patterns, keeping frequently queried (“hot”) columns co-located without manual Z-ORDER management.

Predictive Optimization automatically handles optimization operations in the background, eliminating operational overhead while ensuring recent data is also optimized, which directly addresses both performance and hands-off maintenance requirements.

**QUESTION 306**
A data engineer is designing a data pipeline in Databricks that needs to process records from a Kafka stream where late-arriving data common.

Which approach should the data engineer use?

A. Use a watermark to specify the allowed lateness to accommodate records that arrive after their expected window, ensuring correct aggregation and state management.
B. Use an Auto CDC pipeline with batch tables to simplify late data handling.
C. Implement a custom solution using Databricks Jobs to periodically reprocess all historical data
D. Use batch processing and overwrite the entire output table each time to ensure late data is incorporated correctly.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**

Explanation:
Using a watermark in Spark Structured Streaming defines the maximum allowed lateness for late-arriving events, enabling correct windowed aggregations, state management, and event-time processing while still incorporating late data without unbounded state growth.

**QUESTION 307**
A data engineering team is configuring access controls in Databricks Unity Catalog. They grant the `SELECT` privilege on the `sales` catalog to the `analyst_group`, expecting that members of this group will automatically have `SELECT` access to all current and future schemas, tables, and views within the `sales` catalog.

What describes the privilege inheritance behavior in Unity Catalog?

A. Privileges in Unity Catalog do not cascade; SELECT most be explicitly granted on each schema and table, even if granted at the catalog level.
B. Granting SELECT on a catalog automatically applies SELECT to all current and future schemas, tables, and views within that catalog.
C. Granting SELECT at the catalog level applies to existing schemas and tables but not to those created in the future.
D. Privileges granted at the schema level override any catalog-level privileges and prevent access unless explicitly revoked.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
In Unity Catalog, data access privileges like SELECT do not cascade from the catalog level down to schemas, tables, or views.

SELECT must be explicitly granted on each schema and each table/view. Catalog-level privileges are used primarily for namespace access (USAGE), not for automatic data access inheritance.

**QUESTION 308**
An organization processes customer data from web and mobile applications. Data includes names, emails phone numbers, and location history. Data arrives both as Batch files from an SFTP drop (daily) and Streaming JSON events from Kafka (real-time).

To comply with internal data privacy policies, the following requirements must be met:
- Personally identifiable information (PII) like email, phone_number, and ip_address must be masked or anonymized before storage
- Both batch and streaming pipelines must apply consistent PII handling
- Masking logic must be auditable and reproducible
- The masked data must still be usable for downstream analytics

How should the data engineer design a compliant data pipeline on Databricks that supports both batch and streaming modes, applies data masking to PII, and maintains traceability of transformations for audits?

A. Ingest both batch and streaming data using Lakeflow Spark Declarative Pipelines, and apply masking via Unity Catalog column masks at read time to avoid modifying the data during ingestion.
B. Load batch data with notebooks and ingest streaming data with SQL Warehouses; use Unity Catalog column masks on Silver tables to redact fields after storage.
C. Use Lakeflow Spark Declarative Pipelines for batch and streaming ingestion define a PII masking function, and apply it during Bronze ingestion before writing to Delta Lake.
D. Allow PII to be stored unmasked in Bronze for lineage tracking, then apply masking logic in Gold tables used for reporting.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

Explanation:
Using Lakeflow Spark Declarative Pipelines for both batch and streaming ingestion ensures a single, consistent processing framework.

Defining a PII masking/anonymization function and applying it during Bronze ingestion guarantees that sensitive data is never stored in raw form, meeting privacy-by-design requirements.

Declarative pipelines provide versioned, auditable, and reproducible transformations, ensuring full traceability for compliance and audits, while the masked data remains structured and usable for downstream analytics.

**QUESTION 309**
Which method can be used to determine the total wall-clock time it took to execute a query?

A. In the Spark UI, take the job duration of the longest-running job associated with that query.
B. In the Spark UI, take the sum of all task durations that ran across all stages (or all jobs associated with that query.
C. Open the Query Profiler associated with that query and use the Total wall-clock duration metric.
D. Open the Query Profiler associated with that query and use the Aggregated task time metric.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
The Query Profiler provides a dedicated metric called Total wall-clock duration, which directly represents the real elapsed time from query start to completion, making it the correct and reliable way to measure actual execution time.

**QUESTION 310**
A data architect is designing a Databricks solution to efficiently process data for different business requirements.

In which scenario should a data engineer use a materialized view compared to a streaming table?

A. Precomputing complex aggregations and joins from multiple large tables to accelerate BI dashboard performance.
B. Processing high-volume, continuous clickstream data from a website to monitor user behavior in real-time.
C. Ingesting data from Apache Kafka topics with sub second processing requirements for immediate alerting,
D. Implementing a CDC (Change Data Capture) pipeline that needs to detect and respond to database changes within seconds.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
A materialized view is designed for precomputing and storing results of complex transformations, joins, and aggregations so that downstream queries (e.g., BI dashboards) read from precomputed data instead of recomputing expensive logic, providing faster and more consistent analytical performance.

**QUESTION 311**
A query is taking too long to run. After investigating the Spark UI, the data engineer discovered a significant amount of disk spill. The type of compute instance the data engineer used only provides a core-to-memory ratio of 1:2.

What are the two steps the data engineer should take to minimize spillage? (Choose two.)

A. Increase spark.sql.files.maxPartitionBytes.

B. Choose a compute instance with more disk space.
C. Choose a compute instance with a higher core-to-memory ratio
D. Reduce spark.sql.files.maxPartitionBytes.
E. Choose a compute instance with more network bandwidth.

**Correct Answer:** CD
**Section:** (none)

**Explanation/Reference:**
Explanation:
Use a compute instance that provides more memory per core so shuffle, joins, and aggregations can keep more data in memory and avoid spilling to disk.

Reduce spark.sql.files.maxPartitionBytes to create smaller input partitions so each task processes less data at once, lowering peak memory pressure and reducing spill during downstream shuffles.

**QUESTION 312**
A data engineering team has a time-consuming data ingestion job. It has three data sources, and each data ingestion notebook takes about one hour to load new data. The job had been working fine, loading data every midnight. However, one day they received an alert message stating the job had failed. By investigating the cause, they figured out that the failed notebook code was updated, and a new required configuration was introduced to parameterize a hardcoded value without notice. Now, the team has to fix the issue as quickly as possible to load the latest data from the failing data source.

Which action should the team take in this scenario?

A. Update the task by adding the missing task parameter, and manually run the job.
B. Share the analysis with the failing notebook owner so that they can fix it quickly.
C. Repair the run with the new parameter.
D. Repair the run with the new parameter, and update the task by adding the missing task parameter.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Repairing the run with the new parameter allows the team to immediately recover and load the missing data without waiting for the next scheduled execution.

Updating the task configuration to include the missing parameter ensures the job is permanently fixed, preventing future failures from the same configuration change.

**QUESTION 313**
A data engineer is masking the provided data column containing the email address. The goal is to have an output of the same length for all rows, while keeping different outputs for different values.

Which SQL function should be used to achieve this?

A. `hash(email)`
B. `mask(email, '?')`
C. `sha1('email')`
D. `sha2(email,0)`

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
A cryptographic hash function like SHA-1 produces a fixed-length output for every input value while generating different outputs for different inputs, which satisfies both requirements: consistent length and

deterministic uniqueness for different email values.

**QUESTION 314**
A company wants to implement Lakehouse Federation across multiple data sources, but is worried about data consistency and ensuring all teams access the same authoritative version of their data.

Which statement is applicable for Lakehouse Federations to maintain data consistency?

A. Federation creates local copies that must be manually refreshed.
B. Federation implements change data capture from all sources.
C. A separate data synchronization service must be deployed.
D. Federation provides read-only access that reflects the current state of source systems.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Lakehouse Federation provides read-only, live access to external source systems, meaning queries always reflect the current state of the authoritative source data without copying or synchronizing data, ensuring consistency across teams.

**QUESTION 315**
A data engineer needs to implement column masking for a sensitive column in a Unity Catalog-managed table. The masking logic must dynamically check if users belong to specific groups defined in a separate table (`group_access`) that maps groups to allowed departments.

Which approach should the engineer use to efficiently enforce this requirement?

A. Create a view without selecting the sensitive column
B. Apply a column mask that references the `group_access` mapping table in its UDF
C. Create a UDF that hardcodes allowed groups and apply it as a column mask.
D. Use a row filter to restrict access based on the user's group.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
Use a Unity Catalog column mask implemented as a SQL UDF that evaluates the current user’s group membership (for example via group membership functions) against the group_access mapping, so masking is enforced centrally, stays auditable, and automatically reflects updates to the mapping table without redeploying code.

**QUESTION 316**
Two data engineers are working on the same Databricks notebook in separate branches. Both have edited the same section of code. When one tries to merge the other’s branch into their own using the Databricks Git folders UI, a merge conflict occurs on that notebook file. The UI highlights the conflict and presents options for resolution.

How should the data engineers resolve this merge conflict using Databricks Git folders?

A. Use the Git CLI in the cluster's web terminal to force-push the conflicted merge (git push --force), overriding the remote branch with local version and discarding changes.
B. Use the Git folders UI to manually edit the notebook file, selecting the desired lines from both versions and removing the conflict markers, then mark the conflict as resolved.
C. Abort the merge, discard all local changes, and try the merge operation again without reviewing the confiding code.
D. Delete the conflicted notebook file via the Databricks workspace UI, commit the deletion, and recreate the notebook from scratch in a new commit to bypass the conflict entirely.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
In Databricks Git folders, merge conflicts are resolved by opening the conflicted notebook, manually reconciling the conflicting sections by keeping the desired changes from each version, removing the conflict markers, and then marking the file as resolved so the merge can be completed and committed.

**QUESTION 317**
A data engineer wants to automate job monitoring and recovery in Databricks using the Jobs API. They need to list all jobs, identify a failed job, and rerun it.

Which sequence of API actions should the data engineer perform?

A. Use the jobs list endpoint to list jobs, check job run statuses with jobs runs list, and rerun a failed job using jobs run-now.
B. Use the jobs get endpoint to retrieve job details, then use jobs update to rerun failed jobs.
C. Use the jobs list endpoint to list jobs, then use the jobs create endpoint to create a new job, and run the new job using jobs run-now.
D. Use the jobs cancel endpoint to remove failed jobs, then recreate them with jobs create endpoint and run the new ones.

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
The correct flow is to first list jobs to obtain job identifiers, then list runs to find a run with a failed status, and finally trigger a rerun by invoking the job again using the run-now endpoint.

**QUESTION 318**
A data engineering team is setting up deployment automation. To deploy workspace assets remotely using the Databricks CLI command, they must configure it with proper authentication.

Which authentication approach will provide the highest level of security?

A. Use a service principal and its Personal Access Token
B. Use a service principal with OAuth token federation
C. Use a service principal ID and its OAuth client secret
D. Use a shared user account and its OAuth client secret

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
OAuth token federation with a service principal provides the highest security by using short-lived, dynamically issued access tokens instead of long-lived secrets or PATs.

This approach minimizes credential exposure risk, supports enterprise-grade identity management, and aligns with zero-trust and least-privilege security models, making it the most secure authentication method for automated Databricks CLI deployments.

**QUESTION 319**
A data engineer treated a daily batch ingestion pipeline using a cluster with the latest DBR version to store banking transaction data, and persisted it in a MANAGED DELTA table called `prod.gold.all_banking_transactions_daily`. The data engineer is constantly receiving complaints, from business units that constantly read this table in an ad hoc way using a SOL Serverless Warehouse, regarding the poor performance of their queries. Analyzing the queries, the data engineer

slated that these areas use high cardinality columns as filters, to perform their analysis. The data engineer is studying possibilities to implement a data layout optimization technique for that table, which is incremental, easy to maintain, and can easily evolve over time.

Which command should the data engineer implement?

A. Alter the table to use Hive Style Partitions + Z-ORDER and implement a periodic `OPTIMIZE` command.
B. Alter the table to use Hive Style Partitions and implement a periodic `OPTIMIZE` command.
C. Alter the table to use Z-ORDER arid implement a periodic `OPTIMIZE` command.
D. Alter the table to use Liquid Clustering and implement a periodic `OPTIMIZE` command

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Liquid Clustering is designed for high-cardinality columns, provides incremental optimization, and evolves automatically over time as data and query patterns change.

It eliminates rigid partitioning strategies and heavy manual maintenance, making it easy to operate, adaptive, and well-suited for ad-hoc analytics on large Delta tables with evolving access patterns.

**QUESTION 320**
A data engineer is designing an append-only pipeline that needs to handle both batch and streaming data in Delta Lake. The team wants to ensure that the streaming component can efficiently track which data has already been processed.

Which configuration should be set to enable this?



A. `checkpointLocation`
B. `overwriteSchema`
C. `mergeSchema`
D. `partitionBy`

**Correct Answer:** A
**Section:** (none)

**Explanation/Reference:**
Explanation:
Setting a checkpointLocation enables Spark Structured Streaming to persist progress information (offsets and state) so it can reliably track which data has already been processed, support exactly-once semantics with Delta, and resume correctly after restarts.

**QUESTION 321**
A data engineer is implementing Unity Catalog governance for a multi-team environment. Data scientists need interactive clusters for basic data exploration tasks, while automated ETL jobs require dedicated processing.

How should the data engineer configure cluster isolation policies to enforce least privilege and ensure Unity Catalog compliance?

A. Configure all dusters with `NO_ISOLATION_SHARED` access modes since Unity Catalog works with any cluster configuration
B. Allow all users to create any duster type and rely on manual configuration to enable Unity Catalog access modes.
C. Create compute policies with `STANDARD` access mode for interactive 'workloads and dedicated access mode for automated jobs.
D. Use only `DEDICATED` access mode for both interactive workloads and automated jobs to maximize security isolation.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
Using STANDARD access mode for interactive clusters enables secure, compliant access for data scientists while supporting Unity Catalog governance.

Using DEDICATED access mode for automated ETL jobs enforces stronger isolation, least-privilege access, and controlled execution for production workloads, aligning with Unity Catalog compliance and security best practices.

**QUESTION 322**
A data engineer, while designing a `Pandas UDF` to process financial time series data with complex calculations that require maintaining state across rows within each stock symbol group must ensure the function is efficient and scalable.

Which approach will solve the problem with minimum overhead while preserving data integrity?

A. Use a `SCALAR_ITER Pandas UDF` with iterator-based processing, implementing state management through persistent storage (Delta tables) that gets updated after each batch to maintain continuity across iterator chunks.
B. Use a `GROUPED_AGG UPF` that processes each stock symbol group independently, maintaining state through intermediate aggregation results that get passed between successive UDF calls via broadcast variables.
C. Use a `SCALAR Pandas UDF` that processes the entire dataset at once, implementing custom partitioning logic within the UDF to group by stock symbol and maintain state using global variables shared across all executor processes.
D. Use `ApplyInPandas` method on spark dataframe that receives all rows for each stock symbol as a pandas DataFrame, allowing processing within each group while maintaining state variables local to each group's processing function.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
applyInPandas (grouped map) sends all rows for each stock_symbol group to the function as a single pandas DataFrame, so you can maintain per-symbol state naturally within the group's processing logic.

This avoids cross-batch external state management and avoids unsafe/global state across executors, making it efficient, scalable, and deterministic for group-wise stateful calculations.

**QUESTION 323**
A data engineer is developing an LDP pipeline using a Databricks notebook that is directly connected to their pipeline. After adding new table definitions and transformation logic in their notebook, they want to check for any syntax errors in the pipeline code without actually
processing any data or running the pipeline.

How should the data engineer perform this syntax check?

A. Open the web terminal from the notebook and run a shell command to validate the pipeline code.
B. Disconnect the notebook from the pipeline and reconnect it to a compute cluster to access code validation features
C. Use the "Validate" option in the notebook to check for syntax errors.
D. Switch to a workspace file instead of a notebook to access the validation and diagnostics tools.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**

Explanation:
The “Validate” option performs a static syntax and configuration check of the Lakeflow Spark Declarative Pipeline code, allowing the engineer to detect errors without executing the pipeline or processing any data, making it the correct approach for fast, safe validation.

**QUESTION 324**
A data engineering team is in the process of migrating off its legacy Hadcop platform. As part of the process, they are evaluating the different storage formats to see which offers the best query performance. Their legacy platform utilizes ORC and RCFile formats. They converted
a subset of their data to Delta Lake and ran a side-by-side benchmark, and found that their queries performed significantly better. Upon investigation, they discovered that the query plans were different, and the queries that read from the Delta Lake tables leveraged a Shuffle Hash
Join, whereas the legacy formats used Sort Merge Joins. The queries that read from the Delta Lake tables also read less data.

Which reason could be attributed to the difference in query performance?

A. The queries against the ORC tables leveraged the dynamic data skipping optimization but not the dynamic file pruning optimization.
B. The queries against the Delta Lake tables were able to leverage the dynamic file pruning optimization.
C. Delta Lake enables data skipping and file pruning using a vectorized Parquet reader.
D. Shuffle Hash Joins are always more efficient than Sort Merge Joins.

**Correct Answer:** B
**Section:** (none)

**Explanation/Reference:**
Explanation:
Dynamic file pruning allows Delta Lake queries to avoid reading irrelevant files at runtime based on join/ filter conditions.

This reduces the amount of data scanned and changes the physical query plan, enabling more efficient join strategies (such as shuffle hash joins instead of sort-merge joins), which directly explains both the lower I/O and better performance observed in Delta Lake queries.

**QUESTION 325**
A data team is automating a daily multitask ETL pipeline in Databricks. The pipeline includes a notebook for ingesting raw data, a Python wheel task for data transformation, and a SQL query to update aggregates. They want to trigger the pipeline programmatically and see previous
runs in the GUI. They need to ensure tasks are retried on failure and stakeholders are notified by email if any task fails.

Which two approaches will meet these requirements? (Choose two.)

A. Trigger the job programmatically using the Databricks Jobs REST API (`/jobs/run-now`), the CLI (`databricks jobs run-now`), or one of the Databricks SDKs.
B. Use Databricks Asset Bundles (DABs) to deploy the workflow, then trigger individual tasks directly by referencing each task’s notebook or script path in the workspace.
C. Create a multi-task job using the UI, DABs, or the Jobs REST API (`/jobs/create`) with notebook, Python wheel, and SQL tasks. Configure task-level retries and email notifications in the job definition.
D. Use the REST API endpoint `/jobs/runs/submit` to trigger each task individually as separate job runs and implement retries using custom logic in the orchestrator.
E. Create a single notebook that uses `dbutils.notebook.run()` to call each step in sequence. Define a job on this orchestrator notebook and configure retries and notifications at the notebook level.

**Correct Answer:** AC
**Section:** (none)

**Explanation/Reference:**
Explanation:
Triggering an existing Databricks Job via /jobs/run-now, the CLI, or an SDK supports programmatic

execution while still registering runs so they are visible in the Jobs GUI with run history.

Defining a multi-task job (UI, DABs, or /jobs/create) with notebook, Python wheel, and SQL tasks allows configuring task-level retries and email notifications on failure directly in the job definition, meeting the operational requirements without custom orchestration.

**QUESTION 326**
A data engineer is designing a system to process batch patient encounter data stored in an S3 bucket, creating a Delta table (`patient_encounters`) with columns `encounter_id`, `patient_id`, `encounter_date`, `diagnosic_code`, and `treatment_cost`. The table is queried frequently by `patient_id` and `encounter_date`, requiring fast performance. Fine grained access controls must be enforced. The engineer wants to minimize maintenance and boost performance.

How should the data engineer create the `patient_encounters` table?

A. Create an external table in Unity Catalog, specifying an S3 location for the data files. Enable predictive optimization through table properties, and configure Unity Catalog permissions for access controls.
B. Create a managed table in Unity Catalog. Configure Unity Catalog permissions for access controls, schedule jobs to run OPTIMIZE and vacuum command daily to achieve best performance.
C. Create a managed table in Hive metastore. Configure Hive metastore permissions for access controls, and rely on predictive optimization to enhance query performance and simplify maintenance.
D. Create a managed table in Unity Catalog. Configure Unity Catalog permissions for access controls, and rely on predictive optimization to enhance query performance and simplify maintenance.

**Correct Answer:** D
**Section:** (none)

**Explanation/Reference:**
Explanation:
Creating a managed table in Unity Catalog ensures fine-grained access control, centralized governance, and full security integration.

Relying on Predictive Optimization provides automatic, hands-off performance tuning (file compaction, data layout optimization, clustering) without manual maintenance.

This combination delivers strong governance, high query performance on frequent filters, and minimal operational overhead, which directly matches the requirements.

**QUESTION 327**
A data engineer has a delta table order with deletion vectors enabled for it. The engineer is attempting to execute the below code:

```
DELETE FROM orders WHERE status = 'cancelled'
```

What should be the behaviour of deletion vectors when the command is executed?

A. Files are physically rewritten without the deleted row.
B. Rows are marked as deleted both in metadata and in files.
C. Rows are marked as deleted in metadata, not in files
D. Delta automatically removes all cancelled orders permanently.

**Correct Answer:** C
**Section:** (none)

**Explanation/Reference:**
Explanation:
With deletion vectors enabled, Delta Lake records deletions in metadata (deletion vectors) rather than rewriting the underlying data files.

The rows are logically marked as deleted and filtered out at read time, while the original files remain unchanged, enabling faster deletes and better performance for DML operations.